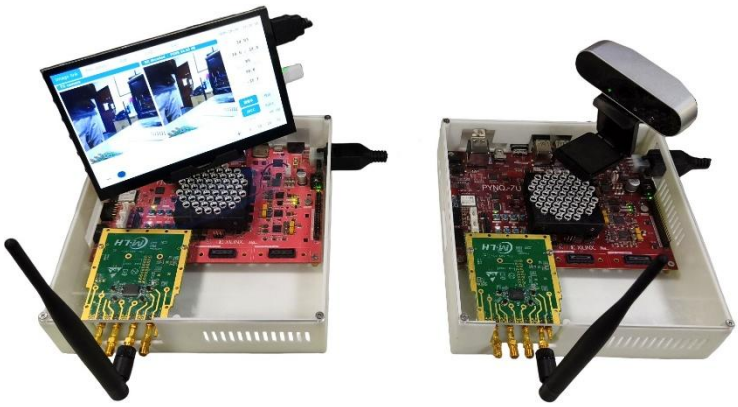
全国大学生嵌入式芯片与系统设计竞赛

2026　FPGA 赛道　•　自主选题（高级组）

基于 FPGA 的 DeepJSCC-Q 实时无线图像传输系统

GlideLink: Real-Time DeepJSCC-Q Image Transmission on AMD FPGA



设计报告

2026 年 10 月

目　　录

摘要.....1
术语与缩略语.....2
第一部分 作品概述.....5
1.1 功能与特性.....5
1.2 应用领域.....5
1.3 主要技术特点.....5
1.4 主要性能指标.....6
1.5 主要创新点.....6
1.6 设计流程.....6
第二部分 系统组成及功能说明.....7
2.1 整体介绍.....7
2.1.1 帧与数据量.....8
2.1.2 时钟域.....9
2.1.3 软硬件划分.....10
2.1.4 PS–PL 接口.....10
2.1.5 神经网络与物理层的接口约定.....11
2.2 硬件系统介绍.....12
2.2.1 硬件整体介绍.....12
2.2.2 机械设计介绍.....12
2.2.3 电路各模块介绍.....13
2.3 软件系统介绍.....45
2.3.1 软件整体介绍.....45
2.3.2 软件各模块介绍.....47
第三部分 完成情况及性能参数.....55
3.1 整体介绍.....55
3.2 工程成果.....56
3.2.1 机械成果.....56
3.2.2 电路成果.....57
3.2.3 软件成果.....59
3.3 特性成果.....61
3.3.1 功能验证.....61
3.3.2 画质与 SNR.....61
3.3.3 帧率与时延.....63
3.3.4 射频前端相对开源基线的改进.....65
3.3.5 与 DeepJSCC-Q 原网络的对比.....65
3.3.6 与 FPGA 实现的 DeepJSCC 对比.....67

第四部分 总结 68
4.1 可扩展之处 68
4.2 心得体会 69

第五部分 参考文献 70

附录 72
附录 A 大模型协作记录 72
附录 B 技能包 74
附录 C 复现说明 76
附录 D 浅层解码器的设计细节 78
D.1 Yang 与 Mandt 浅层解码器中的大核 78
D.2 直线插值的线性程度 79
D.3 各级的噪声 80
D.4 PWL 学到的激活 81
附录 E 基于本作品的通信算法验证平台 82
E.1 意义与基本组成 82
E.2 包格式 83
E.3 流交换 86
E.4 模块槽位 88
E.5 电脑端软件 92
E.6 板卡与电脑之间的传输 93
E.7 平台上的两个应用 93

基于 FPGA 的 DeepJSCC-Q 实时无线图像传输系统

摘要

传统的无线图像传输采用分离信源信道编码（SSCC）：先用 JPEG 等信源编码压缩，再加信道编码和调制。为了追求高压缩率，信源编码通常采用熵编码等变长编码，单个比特出错不仅会使当前码字译错，还会使后续码字连带出错，即“错误传播”。因此，信道 SNR 低于信道编码的纠错门限时，整幅图像无法解码，出现“悬崖效应”。深度联合信源信道编码（DeepJSCC）采用自编码器结构，用一对神经网络把图像直接映射为信道符号，画质随 SNR 平缓变化；DeepJSCC-Q 进一步把符号约束到 QAM 星座上，能够接入标准的数字通信物理层。但它的计算量大，接收端又必须使用未经判决的软符号，对硬件资源和物理层同步精度都提出了很高要求。

本作品用两块 PYNQ-ZU（Zynq UltraScale+ xczu5eg）和两块 AD9361 射频子卡，实现了一条实时无线图像链路。DeepJSCC-Q 编码器、解码器与类 802.11a OFDM 物理层全部在 PL 中实现：编码器在保持信道符号数不变的前提下缩小通道数与下采样倍数，解码器采用与编码器联合训练的非对称浅层结构，乘加量降为原网络的约 1/102，采用 W8A12 整数运算，由 64 个卷积引擎在 250 MHz 下运行，权重与激活全部存放在片上；物理层面向 683 个 OFDM 符号的长帧设计了二阶 SFO 跟踪与 PN 扰码，均衡后的连续 I/Q 直接送入解码器；PS 负责取图、射频控制与触摸屏显示，并调用 Mali-400 GPU 渲染信道三维图。

实测表明：系统能够以 30 fps 的帧率连续传输 256×256 彩色图像，PL 内端到端时延 67.5 ms；在每幅图同样 32,768 个 64QAM 符号、同一物理层的条件下，DeepJSCC-Q 在高 SNR 时 PSNR 比 JPEG + LDPC 基线高 2.8 dB，并在 SNR 从 36 dB 降到 2.9 dB 的过程中平缓退化，而基线在约 16~19.5 dB 处断崖式失效。与已发表的 FPGA DeepJSCC 实现（GLOBECOM 2025）相比，本作品的网络结构更复杂、卷积工作量是其 5.2 倍，而编码器与解码器的 DSP 和 BRAM 用量都明显低于后者。

关键词：深度联合信源信道编码；FPGA；OFDM；软件无线电；实时图像传输

术语与缩略语

缩略语	英文全称	说明
ADC	Analog-to-Digital Converter	模数转换器
AGC	Automatic Gain Control	自动增益控制
APP	A Posteriori Probability	后验概率；LDPC 译码中指变量的后验对数似然比
AWGN	Additive White Gaussian Noise	加性高斯白噪声
AXI	Advanced eXtensible Interface	ARM AMBA 总线协议；AXI4-Lite 用于寄存器读写，AXI-Stream 是带 valid / ready 握手的无地址数据流
BRAM	Block RAM	PL 中的块存储器；BRAM36 容量为 36 Kb，可拆成两块 BRAM18
CDC	Clock Domain Crossing	跨时钟域传输
CFO	Carrier Frequency Offset	载波频偏，两块板本振频率之差
CHDR	Condensed Hierarchical Datagram for RFNoC	RFNoC 中模块之间传输数据与控制命令的包格式
CNU	Check Node Unit	LDPC 译码器中的校验节点处理单元
CORDIC	COordinate Rotation DIgital Computer	只用移位和加法迭代计算三角函数、幅度与相角的算法
CP	Cyclic Prefix	循环前缀，OFDM 符号前复制的符号尾部采样
CPE	Common Phase Error	公共相位误差，一个 OFDM 符号内所有子载波共有的相位旋转
CRC	Cyclic Redundancy Check	循环冗余校验
DAC	Digital-to-Analog Converter	数模转换器
DDR	Double Data Rate SDRAM	板上的外部内存
DDS	Direct Digital Synthesizer	直接数字频率合成器
DeepJSCC	Deep Joint Source-Channel Coding	深度联合信源信道编码，用神经网络把图像直接映射为信道符号
DeepJSCC-Q	Constellation-Constrained DeepJSCC	信道符号限定在 QAM 星座点上的 DeepJSCC[2]
DMA	Direct Memory Access	直接存储器访问；MM2S 为内存到数据流，S2MM 为数据流到内存
DSP	DSP48E2 slice	PL 中的乘加单元，含一个 27×18 位乘法器
EGL / GBM	Embedded-System Graphics Library / Generic Buffer Management	Linux 下 OpenGL ES 与显示驱动之间的接口库
ENSM	Enable State Machine	AD9361 的收发状态机
EVM	Error Vector Magnitude	误差矢量幅度：接收星座点与理想星座点之差的均方根，相对信号功率以 dB 表示

缩略语	英文全称	说明
FDD	Frequency Division Duplex	频分双工，收发通道同时工作
FFT / IFFT	(Inverse) Fast Fourier Transform	（逆）快速傅里叶变换
FIFO	First In, First Out	先进先出缓冲
FIR	Finite Impulse Response	有限冲激响应滤波器
FMC	FPGA Mezzanine Card	FPGA 子卡连接器标准
fps	frames per second	每秒帧数
GDN / IGDN	Generalized Divisive Normalization / Inverse GDN	广义除法归一化[4]及其逆运算
GLES	OpenGL for Embedded Systems	嵌入式 OpenGL 图形接口
GMAC	Giga Multiply-ACcumulate	10^9次乘累加
GUI	Graphical User Interface	图形用户界面
I / Q	In-phase / Quadrature	复基带信号的同相（实部）与正交（虚部）分量
IIO	Industrial I/O	Linux 内核中管理 ADC、DAC 等数据采集设备的子系统
KL	Kullback–Leibler divergence	KL 散度，衡量两个概率分布的差异
LDPC	Low-Density Parity-Check	低密度奇偶校验码
LFSR	Linear Feedback Shift Register	线性反馈移位寄存器，用于产生伪随机序列
LLR	Log-Likelihood Ratio	对数似然比，比特为 0 与为 1 的概率之比的对数
LO	Local Oscillator	本振
LTF	Long Training Field	长训练字段：前导的后半部分，用于精定时与信道估计
LUT	Look-Up Table	PL 中的查找表逻辑单元；LUTRAM 为由 LUT 构成的分布式 RAM
LVDS	Low-Voltage Differential Signaling	低压差分信号
MAC	Multiply-ACcumulate	乘累加
MJPEG	Motion JPEG	逐帧 JPEG 编码的视频流
MMCM	Mixed-Mode Clock Manager	PL 中的时钟管理单元
MMIO	Memory-Mapped I/O	存储器映射的寄存器访问
MSE	Mean Squared Error	均方误差
NCO	Numerically Controlled Oscillator	数控振荡器
NHWC	—	张量的存放顺序：批次、高、宽、通道，同一像素的各通道连续存放
OFDM	Orthogonal Frequency Division Multiplexing	正交频分复用
OOC	Out-Of-Context synthesis	把一个模块脱离顶层单独综合，用于统计该模块的资源

缩略语	英文全称	说明
PAPR	Peak-to-Average Power Ratio	峰均功率比
PHY	Physical layer	物理层
PL	Programmable Logic	Zynq 器件中的可编程逻辑（FPGA 部分）
PN	Pseudo-Noise	伪随机（序列）
PRBS	Pseudo-Random Binary Sequence	伪随机二进制序列，用作测试数据源
PS	Processing System	Zynq 器件中的处理器系统（ARM 处理器、内存控制器与外设）
PSNR	Peak Signal-to-Noise Ratio	峰值信噪比，8 位图像为 $10 \cdot \lg(255^2/\mathrm{MSE})$
PWL	Piecewise Linear	分段线性；本作品中指逐通道可学习的分段线性激活函数
Q10	—	定点格式，小数部分 10 位，1.0 表示为 1024
QAM	Quadrature Amplitude Modulation	正交幅度调制；64QAM 每个符号携带 6 比特
QPSK	Quadrature Phase Shift Keying	正交相移键控，每个符号携带 2 比特
RFNoC	RF Network-on-Chip	Ettus Research 的 FPGA 信号处理框架，模块之间以数据包通信
RTL	Register Transfer Level	寄存器传输级，指用硬件描述语言编写的电路
SFO	Sampling Frequency Offset	采样频偏，两块板采样时钟频率之差
SNR	Signal-to-Noise Ratio	信噪比
SPI	Serial Peripheral Interface	串行外设接口
SSCC	Separate Source-Channel Coding	分离信源信道编码，本作品中为 JPEG + LDPC
SSIM	Structural Similarity	结构相似性
STF	Short Training Field	短训练字段：前导的前半部分，用于帧检测与载波频偏估计
TCP	Transmission Control Protocol	传输控制协议
TX / RX	Transmitter / Receiver	发射端 / 接收端
UDP	User Datagram Protocol	用户数据报协议
URAM	UltraRAM	PL 中的大容量存储块，每块 288 Kb
UVC	USB Video Class	USB 摄像头的标准设备类
W8A12	—	权重 8 位、激活 12 位的整数量化
WNS / WHS	Worst Negative Slack / Worst Hold Slack	建立时间 / 保持时间的最差时序裕量，非负表示时序收敛
XPM	Xilinx Parameterized Macros	Vivado 提供的参数化原语库（如异步 FIFO）

第一部分 作品概述

1.1 功能与特性

本作品是一条基于FPGA的实时无线图像传输系统。发射端把USB摄像头、预设视频或图像裁成 256×256 彩色图像，由 PL 中的 DeepJSCC-Q[2]编码器直接映射为32,768个64QAM符号，经自行设计的OFDM物理层和AD9361发射；接收端完成同步与均衡后，把连续 I/Q 软符号送入 PL 中的解码器重建图像，在屏幕上实时显示。主要特性：

- 30 fps 实时传输，编码、物理层基带部分、解码全部在 PL 中完成，PL 内端到端时延约 67.5 ms；
- 信道变差时画质平缓下降，没有悬崖效应；
- 内置同物理层、同符号数的 SSCC 基线（JPEG + LDPC），可在触摸屏上切换对比，并实时调节 TX 衰减；
- 触摸屏显示 PSNR、时延、星座图、信道三维图（Mali-400 GPU 渲染）等链路状态；
- 上电约 1 分钟自动建立链路，无需上位机。

1.2 应用领域

DeepJSCC[1]的画质随信道质量平缓变化，不需要根据信道预先选择压缩率和码率，适合信道多变、对实时性要求高、允许画质随距离逐渐降低的图像传输场景：

- 无人机、机器人和车辆的实时图传：移动中信道快速变化，传统方案在信号减弱到一定程度后会突然黑屏，本方案仍能输出可辨认的画面；
- 应急通信与野外监测：链路质量差、无法重传时，仍能持续得到图像；
- 嵌入式视觉终端：编码器是结构固定的流水线，权重全部在片上，可以移植到更小的 FPGA 或专用芯片。

本作品采用标准的 OFDM 物理层与 QAM 星座，DeepJSCC-Q 可以替换现有数字图传系统的信源与信道编码部分，物理层与射频前端无需改动。

1.3 主要技术特点

1. 面向 FPGA 的神经网络改造：编码器在保持每幅图 32,768 个信道符号不变的前提下，主通道数从 256 减为 32、下采样倍数从 16 减为 4；解码器采用与编码器不对称的浅层结构（3×3 卷积、4 个 PWL 残差块、两级 3×3 亚像素上采样），与编码器联合训练，乘加量只有对称结构解码器的 1/6，10 dB 下 PSNR 差在 0.1 dB 以内。整个网络的乘加量和参数量分别降为原网络的约 1/102 和 1/177；W8A12 整数量化损失不超过 0.03 dB。
2. 逐层流水的神经网络硬件：卷积引擎、行缓存、GDN、PWL 等均为参数

化 RTL 模块，67 个卷积算子映射为 64 个独立卷积引擎；每层按帧率预算选择最小并行度，权重 ROM 按尺寸在三种组织方式中选择，激活缓存放入 URAM，全部存储放在片上。

3. 面向长帧软符号的 OFDM 接收机：按晶振标称偏差的最坏情况设计二阶 SFO 跟踪与 CPE 补偿，接收端不做星座判决；PN 扰码消除神经网络输出相关性带来的 PAPR 升高。

1.4 主要性能指标

表 1-1　主要性能指标

指标	数值
图像 / 帧率	256×256 RGB，30 fps
物理层	OFDM，64QAM，32,768 符号 / 图
PL 时延	67.5 ms
实测 PSNR	32.2 dB（SNR 36 dB）至 23.5 dB（SNR 2.9 dB）
对比 SSCC（JPEG + LDPC）	高 2.8 dB，无悬崖效应
DSP（TX / RX）	243 / 133
功耗（实测，含 PS 与 DDR）	3.07 W / 3.44 W

1.5 主要创新点

1. DeepJSCC-Q 编解码器与 OFDM 物理层全部在 PL 中实现，经空口 30 fps 实时传输。
2. 与编码器联合训练的非对称浅层解码器：非线性集中在潜变量分辨率上，上采样用 3×3 亚像素卷积，乘加量降为对称结构的 1/6。
3. 逐层流水卷积引擎配合按层并行度与三种权重 ROM：每帧卷积乘累加量是文献[3]的 5.2 倍（按同一方法计数，3.3.5 节），编码器 DSP 只用其 15 %。
4. 面向长帧软符号的 OFDM 接收：二阶 SFO 跟踪、CPE 补偿与 PN 扰码。
5. 同物理层、同符号数的 SSCC 基线实测对比，802.11n LDPC 编译码器自行设计。

1.6 设计流程

设计流程见图 1-1。先确定神经网络与物理层的接口约定（2.1.5 节），再分两条线并行设计、各自验证：

1. 神经网络：浮点模型与浅层解码器联合训练 → W8A12 整数参考模型 → 子模块 RTL 与参考模型逐比特比对 → 规划并行度与存储 → 脚本生成顶层 → 整网逐比特仿真与 250 MHz 时序收敛；
2. 物理层与射频：算法仿真确定同步与跟踪参数 → RTL 编写与仿真 → AD9361 接口与校准 → 单板环回 → 两板空口测试；

3. 系统集成：对接编解码器与物理层，加入 PS 软件、GUI 与 SSCC 基线，进行空口实测。

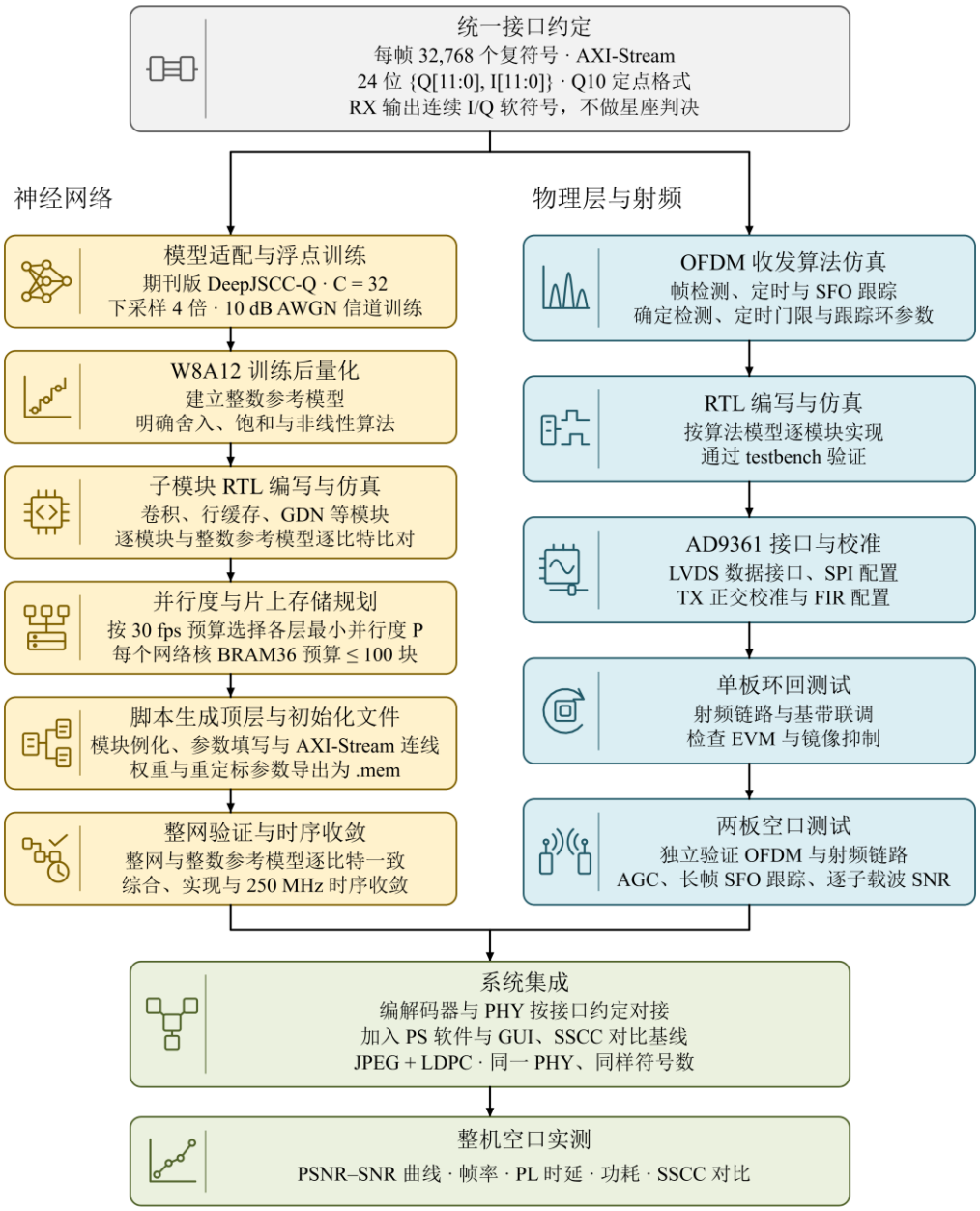


图 1-1　设计流程：神经网络与物理层两条线按同一接口约定并行设计、分别验证，再进行系统集成。

第二部分　系统组成及功能说明

2.1 整体介绍

本作品由两块 PYNQ-ZU 开发板（Zynq UltraScale+ xczu5eg）和两块 AD9361

射频子卡（FMC 接口）组成一条单向无线图像链路，系统整体结构如图 2-1。TX 板把摄像头、预设视频或图像裁成 256×256 RGB 图像，PL 中的 DeepJSCC-Q 编码器把一幅图直接映射为 32,768 个 64QAM 符号，经 OFDM 物理层调制后由 AD9361 发射。RX 板的 OFDM 接收机完成同步与均衡，把未经判决的连续 I/Q 软符号送入 PL 中的解码器，重建出的图像由 PS 显示在触摸屏上。两板之间的网线传输 TX 端的预览图（用于计算 PSNR）和控制命令，编码后的图像经空口传输。

为了与传统方案对比，两块板上还实现了一条 SSCC 基线链路（JPEG[15]+LDPC+64QAM），它与 DeepJSCC-Q 共用同一 OFDM 物理层、同一星座映射规则和每幅图同样的 32,768 个符号，运行时可在两种模式之间切换。

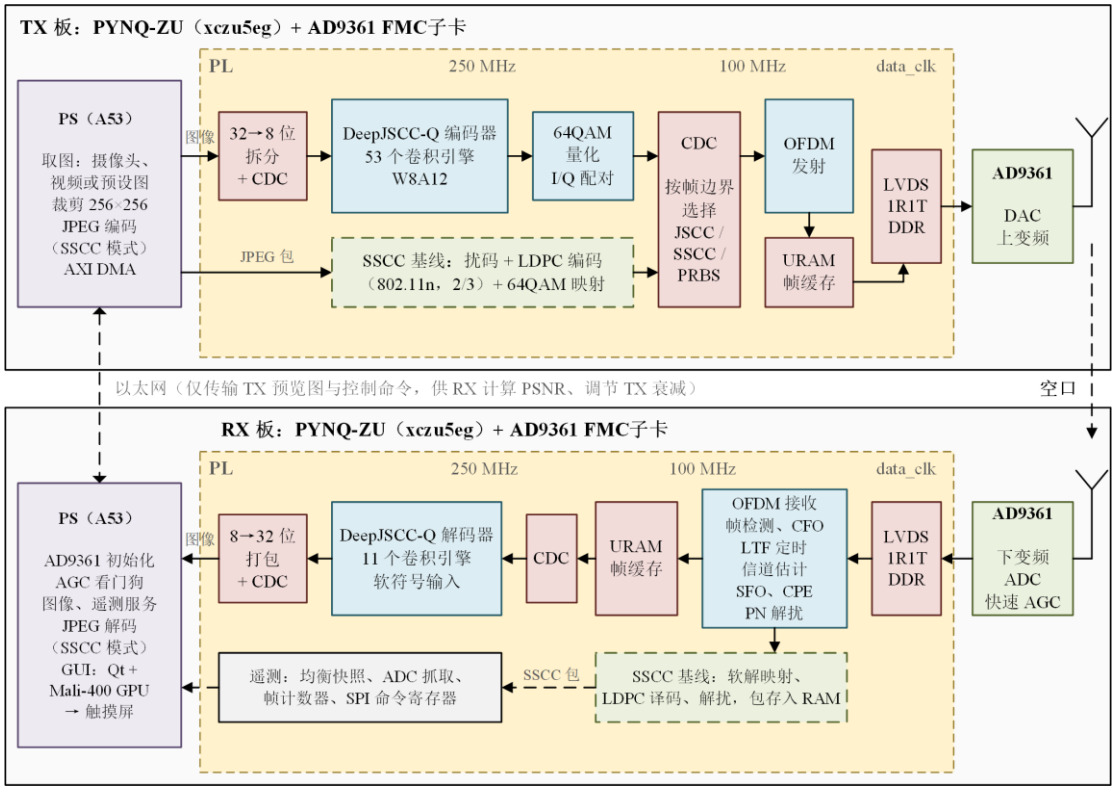


图 2-1　系统整体框图。上：TX 板，数据从左向右；下：RX 板，数据从右向左。黄色虚线框为 PL，紫色框为 PS 软件，绿色实线框为 AD9361 射频芯片，虚线边框的模块为 SSCC 基线。

2.1.1 帧与数据量

链路以“一幅图 = 一个 PHY 帧”组织，各级数据量见表 2-1。编码器每 33.3 ms 产出一帧符号，PHY 只用 2.748 ms 就能把它发完，空口占空比约 8 %，其余时间发射机静默。神经网络与 PHY 的速率差由 TX 端 DAC 前和 RX 端均衡后的两个 URAM 帧缓存吸收（2.2.3.4 节）。

表 2-1　一帧的数据量

位置	内容	数量
图像	256×256×3，uint8，NHWC 顺序	196,608 字节
编码器输出	64×64×16 潜变量，相邻通道配成(I, Q)	32,768 个 64QAM 符号（24 位）
OFDM 映射	每个 OFDM 符号 48 个数据子载波；32,768 = 682×48 + 32，末符号补 16 个零	683 个 OFDM 符号
时域	前导 320（STF 160 + LTF 160）+ 683×80（含 CP 16）	54,960 个采样 = 2.748 ms（20 MSPS）
帧率	编码器节奏决定	30 fps，帧间隔 33.3 ms

2.1.2 时钟域

板上 125 MHz 晶振经 PL 中的 MMCM 产生三个时钟，分别供神经网络编解码器、PHY 和 AD9361 控制模块使用；AD9361 另外提供 LVDS 随路时钟，共四个时钟域（表 2-2）。神经网络与 PHY 之间、PHY 与 AD9361 之间都只经过异步 FIFO，每个 FIFO 同时传送数据和帧边界（`tlast/tuser`），帧结构在跨时钟域后保持不变。

表 2-2　时钟域

时钟	频率	使用者
神经网络时钟	250 MHz	DeepJSCC-Q 编码器 / 解码器
物理层时钟	100 MHz	OFDM 物理层、帧缓存、SSCC 链路、AXI 寄存器与 DMA 接口
控制时钟	20 MHz	AD9361 SPI、初始化与校准状态机
`data_clk`（AD9361 接口时钟）	40 MHz（AD9361 输出）	LVDS 收发；1R1T 下每个采样占 2 个时钟周期

复位在 MMCM 锁定、PS 的 `pl_resetn0` 释放后，分别在 250 MHz 和 100 MHz 域内用两级触发器同步释放。XPM 异步 FIFO 只有一个复位输入，所以它的复位要等两个时钟域都完成同步后才释放。AD9361 一侧的 FIFO 读写时钟之一是 `data_clk`，而 `data_clk` 要等 AD9361 的 BBPLL 和 LVDS 配置完成才存在，因此这两个 FIFO 一直保持复位，直到初始化完成后再过 256 个控制时钟周期，保证复位期间两侧时钟都已存在。

2.1.3 软硬件划分

表 2-3　软硬件划分与依据

功能	位置	依据
DeepJSCC-Q 编码器 / 解码器	PL，250 MHz	每帧卷积乘累加量为 0.913 + 0.241 GMAC，30 fps 时约 35 GMAC/s，A53 无法实时完成
OFDM 物理层（IFFT/FFT、同步、信道估计、SFO / CPE 跟踪）	PL，100 MHz	20 MSPS 连续采样流，时延要确定
帧缓存、跨时钟域	PL（URAM、FIFO）	神经网络与 PHY 的速率不同，实时接口一侧必须整帧缓存
SSCC 扰码 / LDPC 编译码	PL	逐比特 / 逐符号的流式处理；LDPC 译码每个码字每次迭代要更新 7,128 条边、最多 16 次迭代，须在码字到达间隔（约 27 μs）内完成
AD9361 LVDS 接口、TX 初始化与正交校准相位扫描	PL	接口时序严格
取图、中心裁剪、DMA 送图	PS（TX）	依赖 Linux UVC 驱动与 OpenCV；每帧只有 196 KB
SSCC 的 JPEG 编 / 解码	PS	变长熵编码，控制流复杂
RX 的 AD9361 初始化、AGC 看门狗、TX 衰减控制	PS	慢速控制
图像转发、遥测、频谱、GUI 与三维显示	PS（RX）	人机交互；三维曲面用 Mali-400 GPU 绘制

2.1.4 PS－PL 接口

两块板的 PS 都只用两个 AXI 端口：PS 作为主机，经 HPM0_LPD 以 AXI4-Lite 访问 PL 的寄存器；DMA 作为主机，经 HP0 访问 DDR。地址映射见表 2-4。图像以整帧为单位传输：TX 的 MM2S 一次送出 196,608 字节，受编码器反压；RX 的 S2MM 一次收 49,152 个 32 位字。

寄存器块的主要内容如下：

- TX 控制寄存器组：
  - 控制寄存器中的两位：分别选择数据源（编码器或 PRBS 64QAM 测试源）和 SSCC 模式，两者都在帧边界生效；
  - 四个计数器：分别计数进入编码器的字节、进入 PHY 的符号和帧、发往 DAC 的帧，用于测量分段时延（3.3 节）；
  - 帧尾错位计数器：计数 `tlast` 不在第 32,767 个符号上的帧，检查接口约定；

  - SPI 命令与状态寄存器是 AD9361 的寄存器读写通道：PS 写入地址、数据和读/写位，读回传输计数与读出数据。
- RX 控制寄存器组：
  - 软复位位：复位 PHY 与帧缓存；
  - 图像转发控制：设置单帧或连续转发解码图像；
  - ADC 抓取控制：可以立即触发，也可以在下一帧开始时触发；
  - 五个帧计数器：依次计数 PHY 同步、PHY 输出、进入帧缓存、整帧丢弃和解码完成的帧；
  - AGC 状态寄存器：AD9361 `CTRL_OUT` 状态输出引脚的实时值（增益锁定位与增益索引）；
  - 射频控制寄存器（AD9361 复位、PS 控制模式、初始化完成）：与 SPI 命令、状态寄存器一起，用于 PS 初始化 AD9361。

跨时钟域的控制位和计数器都经两级同步；AD9361 的 SPI 命令以请求位翻转的方式发出，20 MHz 域对同步后的请求位做双边沿检测，每检测到一次翻转发起一次传输；PS 写命令后等待传输计数加一再发下一条。

表 2-4　PS 可见的地址空间

板	地址	模块	说明
TX	0x8000_0000	图像 DMA（MM2S，32 位）	图像或 SSCC 包 → PL
TX	0x8002_0000	TX 控制寄存器组	数据源选择、计数器、AD9361 SPI 命令
RX	0x8000_0000	图像 DMA（S2MM）	解码图像 → PS
RX	0x8001_0000	ADC 抓取 DMA（S2MM）	ADC 原始采样 → PS
RX	0x8002_0000	RX 控制寄存器组	软复位、图像转发、抓取、计数器、AD9361 控制
RX	0x8004_0000	遥测接口（AXI4）	均衡器各级快照、SSCC 包缓冲

2.1.5 神经网络与物理层的接口约定

神经网络与 PHY 之间的接口约定见表 2-5，两侧按相同的接口约定分别设计和验证，再进行对接。

表 2-5　神经网络与 PHY 的接口约定

项	约定
协议	AXI-Stream，双向 valid / ready，任一侧都可以反压
数据	`tdata[23:0] = {Q[11:0], I[11:0]}`，Q10 补码，每拍一个复符号
星座	64QAM 电平 158 • (2k − 7)，k = 0…7，平均功率 1.0
顺序	潜变量按 NHWC 顺序，通道(2j, 2j+1)配成(I, Q)
帧边界	每帧 32,768 个符号，最后一个符号上 `tlast = tuser[0] = 1`
接收端	输出均衡后的连续 I/Q（同一格式），不做判决

2.2 硬件系统介绍

2.2.1 硬件整体介绍

表 2-6　硬件组成

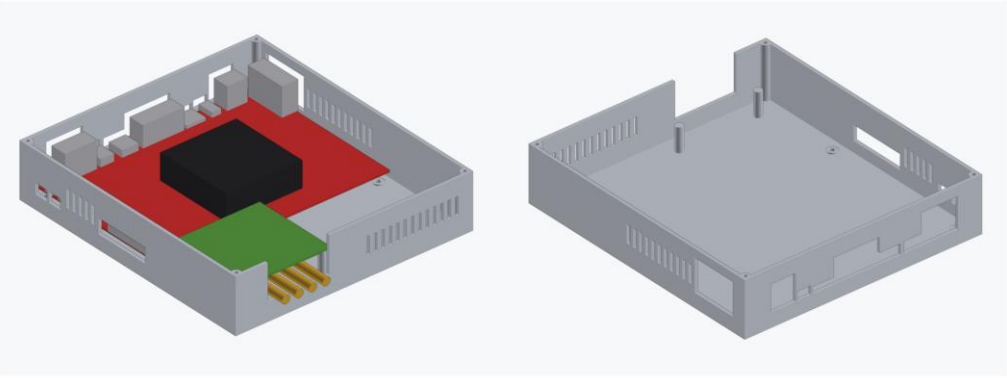
部件	数量	型号 / 规格	用途
FPGA 开发板	2	PYNQ-ZU，Zynq UltraScale+ xczu5eg-sfvc784-2-e（4×A53 + Mali-400 MP2，PL 117k LUT / 144 BRAM36 / 64 URAM / 1248 DSP），PYNQ 3.1	TX、RX 各一块，完成神经网络编解码与基带处理
射频子卡	2	AD9361 FMC 子卡（参考时钟 40 MHz）	射频收发，LVDS 1R1T，FDD
天线	2	胶棒天线	TX 子卡的发射口、RX 子卡的接收口各一根
USB 摄像头	1	UVC，MJPG 640×480 @ 30 fps	TX 取图（可选，也可用预设视频或图像）
触摸屏	1	1024×600，DisplayPort 输入，USB 触摸	RX 端显示 GUI
网线	1	两块板之间	传输 TX 预览图与控制命令
外壳	2	3D 打印底壳 + 3 mm 亚克力顶盖	固定板卡，引出全部接口

AD9361 与 PL 之间是 LVDS 数据接口：接收方向有随路时钟 `rx_clk_in`、帧信号 `rx_frame_in` 和 6 对数据线，发送方向有 `tx_clk_out`、`tx_frame_out` 和 6 对数据线；控制方向有 4 线 SPI（`spi_clk/csn/mosi/miso`），以及 `resetb`（复位）、`enable` / `txnrx`（收发状态控制）、`en_agc`（AGC 使能）、`ctrl_in[3:0]`（增益控制输入）、`ctrl_out[7:0]`（状态输出）等引脚。PL 在初始化完成后把 `enable` 和 `txnrx` 同时拉高，使 AD9361 进入 FDD 收发状态；`en_agc` 和 `ctrl_in` 固定为 0。增益由 AD9361 片内的快速 AGC 控制，PL 只把 `ctrl_out` 上的增益锁定位和增益索引读给 PS。

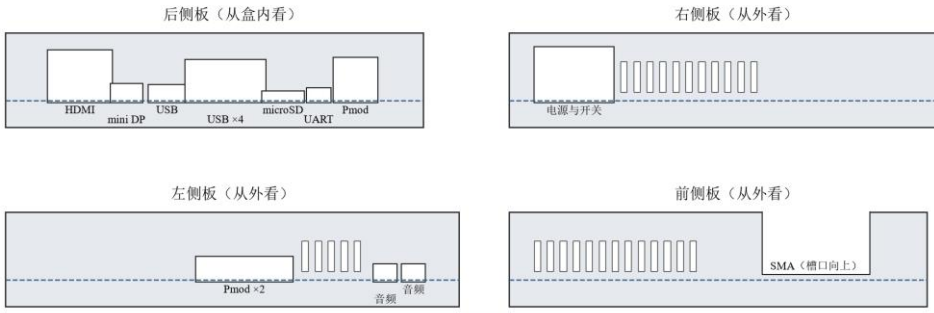
TX 和 RX 两个工程的 PL 顶层分别是 `jscc_tx_ps_top.sv` 和 `jscc_rx_ps_top.sv`，模块组成与图 2-1 一致，AD9361 接口、帧缓存和跨时钟域模块两端共用。

2.2.2 机械设计介绍

TX 和 RX 两块板各装在一个外壳中，两个外壳相同（图 2-2）。外壳由 3D 打印的底壳和 3 mm 透明亚克力顶盖组成，外形 193 mm × 209.4 mm × 43.5 mm（不含顶盖），顶盖透明，从上方可以看到开发板、射频子卡和风扇。



(a) 装入板卡（顶盖未画出）　　(b) 底壳



(c) 侧板开孔（同一比例，蓝色虚线为PYNQ-ZU的PCB上表面）

图 2-2　外壳设计：(a) 装入板卡后的底壳（顶盖未画出，板卡和接口以简化实体表示）；(b) 底壳；(c) 四个侧板的开孔。

外壳的尺寸由以下几点决定：

- 板卡固定：PYNQ-ZU（180 mm × 130 mm）用 4 根 8 mm 的 M3 铜柱固定在底壳上，铜柱高于板卡背面最高的元件（5.35 mm）。射频子卡插在 FMC 上，伸出开发板前缘 59.4 mm，底壳在它外端的两角下方设两根支撑柱，避免子卡只靠 FMC 接插件悬臂受力。
- 高度：板上最高的部件是带金属支架的 HDMI 接口（高出 PCB 28 mm），风扇散热器高 20.5 mm；顶盖下表面高出 PCB 上表面 31.4 mm。
- 天线：子卡的 4 个 SMA 接口从前侧板的通槽伸出，天线直接接在子卡上，不经过转接线；通槽向上开口，由顶盖封闭。
- 接口：开发板边缘的接口全部开孔（图 2-2(c)），开孔四周留约 2 mm 余量。后侧板相邻的开孔合并成一个台阶形开口，不留过窄的隔条；右侧板的电源插座和电源开关合用一个开口，留出插头卡扣和手指操作的空间。
- 散热：顶盖在风扇正上方开 55 个直径 5 mm 的蜂窝孔作为进风口，左、右、前三面的散热缝作为出风口。
- 顶盖固定：底壳四个内角各有一根直径 7 mm 的立柱，顶盖用 M3 螺丝固定，立柱不遮挡开发板的安装孔。

2.2.3 电路各模块介绍

PL 中的电路按功能分为六部分：

- DeepJSCC-Q 编码器与解码器（2.2.3.1 节）：编码器把 256×256 RGB 图像映射为 32,768 个 64QAM 符号，浅层解码器把均衡后的软符号重建为图像，均为逐层流水的整数运算；
- OFDM 发射链（2.2.3.2 节）：对符号流做 PN 扰码、插导频、IFFT 和功率回退，加上前导，形成 20 MSPS 的基带帧；
- OFDM 接收链（2.2.3.3 节）：完成帧检测与载波频偏纠正、LTF 精定时、信道估计与均衡、SFO 与 CPE 跟踪，输出连续的 I/Q 软符号；
- 帧缓存与跨时钟域（2.2.3.4 节）：吸收神经网络与 20 MSPS 采样流之间的速率差，并在各时钟域之间传递数据；
- AD9361 接口与射频控制（2.2.3.5 节）：LVDS 数据收发、SPI 配置与 TX 正交校准相位扫描；
- SSCC 基线链路（2.2.3.6 节）：发送端的扰码、LDPC 编码与 64QAM 映射，接收端的软解映射与 LDPC 译码。

其中第三方内容只有以下几项，其余 RTL 均为本作品自行设计：

- AD9361 的 LVDS 数据接口 `ad9361_lvds_mode.v`，源自 ADI HDL[17]，改为 PYNQ-ZU 引脚与 1R1T 模式；
- AD9361 的 SPI 主机 `ad9361_spi.v` 与初始化序列执行器 `ad9361_config.v`，源自 lzk2211 的 Zedboard_AD9361_radar 工程[16]，经原作者同意以 MIT 许可再发布，本作品修正了其握手错误（2.2.3.5 节）；
- AD9361 的初始化寄存器序列，由 ADI 配置工具生成；
- Vivado 自带 IP：FFT、FIR Compiler、CORDIC、DDS Compiler、除法器、XPM FIFO 与时钟向导。

2.2.3.1 DeepJSCC-Q 编码器与解码器

（1）神经网络结构与 RTL 实现方式

神经网络结构见图 2-3。编码器按 DeepJSCC-Q 期刊版[2]的结构实现，由 9 个块组成，把 256×256×3 图像映射为 64×64×16 潜变量，共 55 个卷积算子，块的类型有 RG（带 GDN 的残差块，可带步长 2 下采样）、RB（普通残差块）和 ATT（注意力块[5]）。解码器采用与编码器不对称的浅层结构，由 7 个块组成，共 12 个卷积算子：入口是一个线性 3×3 卷积；之后是 4 个 RP（激活函数为可学习分段线性函数 PWL 的残差块），全部工作在 64×64 的潜变量分辨率上；最后是两个 UP（亚像素上采样块，3×3 卷积输出 4 倍通道后接 PixelShuffle[6]），第一个 UP 的主路接 IGDN、旁路为线性，第二个 UP 后经 sigmoid 输出 8 位 RGB。解码器每帧的乘累加量为 0.250 GMAC（其中卷积 0.241 GMAC），约为与编码器对称、与论文同构的解码器（10 个块、59 个卷积算子、1.417 GMAC）的 1/6，设计依据见本节第 2 小节。

模块库（表 2-7）中的模块都是参数化 RTL 模块，两端网络共有 67 个卷积算子，各层的通道数、尺寸、并行度和存储方式各不相同，手工例化和连线工作量大且容易出错，因此顶层的例化与连线由脚本完成：脚本读取训练后导出的网

络描述，按网络拓扑例化模块库中的模块并连接 AXI-Stream 端口，把存储规划（本节第 8 小节）确定的并行度、ROM 组织方式和存储原语作为参数填入，并把权重、偏置和重定标参数写成.mem 初始化文件。

所有模块之间都是 AXI-Stream 握手。数据按 NHWC 顺序一拍传一个元素；tlast 标记一个像素的最后一个通道，tuser[0]标记一帧的最后一拍。整个神经网络是一条带反压的层间流水线：每一层都有自己的计算单元和局部缓存，层间无需缓存整幅特征图，下一帧的输入在上一帧尚未输出完时就可以开始进入。

表 2-7　神经网络模块库

模块	功能	关键参数
axis_line_buffer	K×K 卷积的行缓存与窗口读出	通道数、宽高、核尺寸、步长、缓存行数、窗口读取遍数、每字打包的元素数、存储原语
axis_pixel_buffer	1×1 卷积的像素缓存（双缓冲）	通道数、每个像素的读取遍数
conv_engine	P 路输出通道并行卷积 + 偏置 + 激活 + 重定标	输入 / 输出通道数、核尺寸、并行度 P、激活函数、权重 ROM 组织方式、重定标乘法的实现
weight_stream / rom_banked / rom	权重 ROM 的三种组织方式	并行度 P、32 位 ROM 列数、每个存储体的深度
axis_gdn + gdn_unit	GDN / IGDN 归一化	通道数、乘加通路数、开方与除法单元数、GDN 或 IGDN
axis_pwl	逐通道可学习的分段线性激活	通道数、第一个折点、段宽（2 的幂次）、斜率与折点纵坐标表
axis_sigmoid	32 段分段线性 sigmoid	分段数、输出 8 位还是 17 位
axis_gate	注意力门 y = x + a·σ(b)	两路的重定标乘数与移位
axis_add	残差相加并重定标	两路的重定标乘数与移位、是否加 ReLU
axis_pixel_shuffle	PixelShuffle（深度到空间变换[6]）	通道数、放大倍数、宽、是否在输出端带一行 FIFO
centre_tap	取 3×3 窗口的中心像素，供 1×1 旁路使用	通道数、核尺寸、主路每像素的读取遍数、转给旁路的遍数
axis_fifo_packed	残差旁路的延迟对齐 FIFO（打包存储）	每字打包的元素数、深度、存储原语
axis_fork / axis_split / axis_tag	一路分多路 / 按通道拆分 / 生成帧边界标记	分路数、拆分处的通道号
qam_tx / rx_frame	神经网络与 PHY 的边界（本节第 9 小节）	—

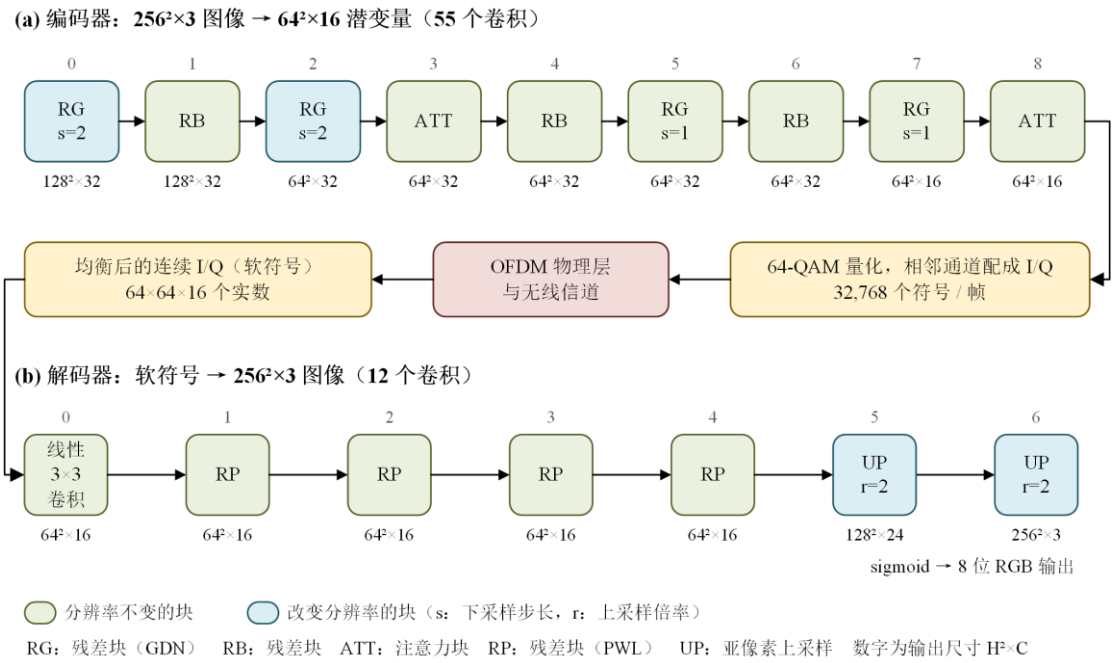


图 2-3　编码器与解码器的块结构，标出了块序号、步长 / 上采样倍率与各块的输出尺寸。

（2）解码器的设计依据

与论文同构、与编码器对称的解码器有 10 个块、59 个卷积算子，每帧 1.417 GMAC，单独综合（OOC）需要 50,178 个 LUT、312 个 DSP、100 块 BRAM36 和 18 块 URAM，比编码器还大。Yang 与 Mandt[19]指出，神经图像压缩中编码端与解码端的计算量可以不对称：训练好的深层解码器近似一个仿射映射，可以换成很浅的解码器，性能损失由更强的编码器补偿；他们的浅层解码器以一层大核转置卷积为主。本作品分析了其公开模型的合成大核（附录 D）：大核的能量在一个步长大小的块内大致均匀，用 3 层 3×3、步长 2 的线性转置卷积级联拟合时剩余误差只有核能量的 0.11 %，即大核可以由小核级联代替。

DeepJSCC-Q 的情况不同：64QAM 星座约束使编码器成为强非线性映射，解码器必须真正反解这种非线性。用直线插值检验各部分的线性程度：在两幅图像之间取 $x(t)=(1-t)x_0+t\cdot x_1$，比较输出沿 t 的路径长度与端点距离之比（完全线性时为 1，DIV2K 验证集 100 对图像的中位数）。编码器的比值为 1.88；浅层解码器从入口到第 4 个 RP 输出为 1.31，之后的上采样部分为 1.01，编码器与解码器串联后为 1.02。也就是说，编码器的非线性在 64×64 上被反解，两次上采样只做近似线性的插值与展开，用 3×3 亚像素卷积即可，不需要大核。因此非线性处理全部放在 64×64 上，这里每层卷积的计算量只有 256×256 的 1/16。插值曲线、各级噪声分析与 PWL 学到的激活形状见附录 D。

各部件的作用用消融实验确定：每次只去掉或替换最终解码器的一个部件，训练流程相同（冻结原编码器、无噪声输入训练解码器 300 轮，再在 10 dB AWGN

下联合训练 800 轮），结果见表 2-8 与图 2-4。

表 2-8　解码器消融结果（DIV2K 验证集 PSNR，dB；第一行为最终解码器，其余为与它的差）

改动	0 dB	4 dB	10 dB	20 dB	每帧乘累加量
最终解码器	20.91	26.69	31.06	32.82	0.250 G
去掉入口的线性 3×3 卷积	−0.34	−0.19	−0.04	−0.05	0.241 G
残差块中的 PWL 换成 LeakyReLU	−0.08	−0.13	−0.27	−0.39	0.250 G
去掉 4 个残差块	+0.42	−0.66	−1.10	−1.06	0.174 G
去掉 IGDN（第一个 UP 主路改为线性）	+0.12	−0.31	−0.55	−0.61	0.241 G
冻结原编码器，只训练解码器	−0.19	−0.55	−0.76	−1.00	0.250 G

- 入口的线性 3×3 卷积只在低 SNR 下起作用：0 dB 和 2 dB 分别贡献 0.34 dB 和 0.26 dB，10 dB 以上不到 0.05 dB。它在各 SNR 下都把输入的 SNR 提高 0.44 dB，这一增益保留到输出图像。
- 残差块、PWL 和 IGDN 负责反解编码器的非线性，去掉或替换后的损失随 SNR 升高而增大，高 SNR 下分别约为 1.1 dB、0.4 dB 和 0.6 dB。IGDN 是编码器中 GDN 的逆运算，每个输出由同一位置所有通道的能量共同归一化，逐通道的 PWL 不能代替这种跨通道的增益调整，因此第一个 UP 保留一个 IGDN。在 0 dB，去掉残差块或 IGDN 反而略有提高：接收潜变量几乎被噪声淹没时，非线性部件会放大噪声。
- 编码器与浅层解码器联合训练的增益同样随 SNR 升高而增大，高 SNR 下约 1 dB。联合训练后编码器本身更接近线性，路径长度比从 2.03 降到 1.88，即编码器把码字调整成浅层解码器容易反解的形式。训练中保留星座分布的 KL 正则，联合训练后编码器使用 64 个星座点的频率接近均匀（按使用频率计算的熵为 6.00 bit，最大值为 6 bit），平均功率为 1.00，物理层不受影响；编码器权重随之更新，TX 与 RX 的比特流一起更换。

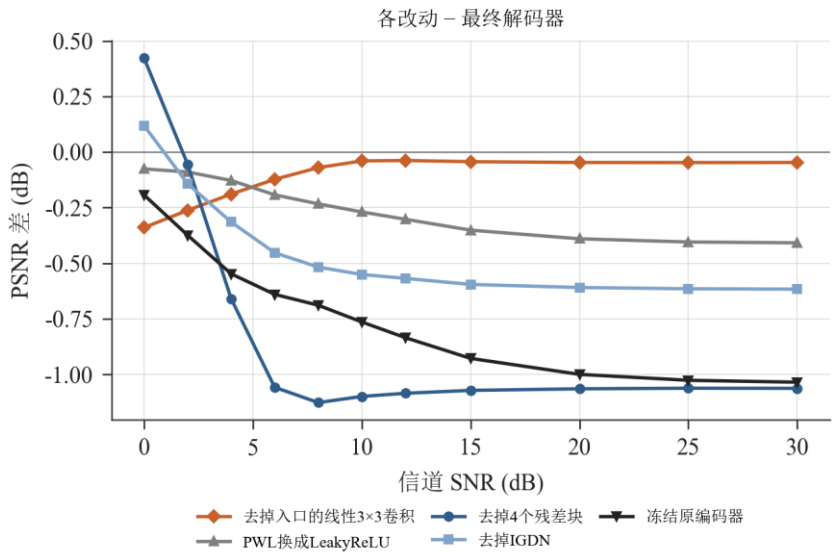


图 2-4　各改动与最终解码器的 PSNR 差随信道 SNR 的变化（浮点模型，同一组噪声）。

残差块的个数用扫描确定：残差块从 1 个到 6 个，其余结构与训练流程不变，结果见图 2-5。4 个块以前 PSNR 随块数上升，1 个块比 4 个块低 0.44 dB（10 dB）；4 个块以后变平，5 个、6 个块与 4 个块的差在−0.14～+0.08 dB 之间，与训练的波动相当（10 dB 下最后 20 轮验证 PSNR 的平均，5 个、6 个块与 4 个块之差为−0.03 dB 和+0.05 dB）。因此取 4 个残差块。

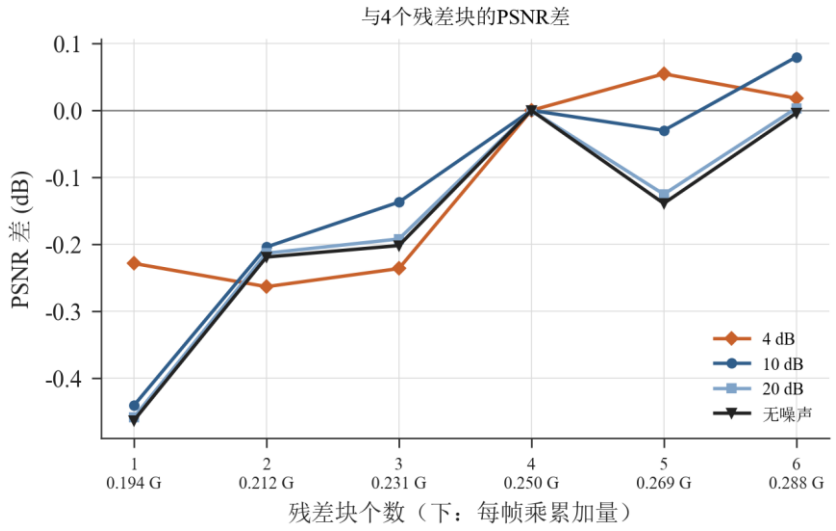


图 2-5　残差块个数为 1～6 时与 4 个残差块的 PSNR 差（DIV2K 验证集，浮点模型）。横轴下方为每帧乘累加量。

增加残差块同时增加了 PWL 的个数和感受野。两者的作用用两组对照分开（表 2-9）：A 在 1 个 3×3 残差块后接 3 个 1×1 残差块，PWL 个数与 4 个块相同、感受野与 1 个块相同；B 只有 1 个残差块、卷积改为 9×9，PWL 个数与 1 个块相同、感受野与 4 个块相同。在 10 dB、20 dB 和无噪声下，A 追回了 1 个块与 4 个块之差的 59 %、72 %和 74 %，B 几乎没有追回，计算量却比 4 个块多 38 %。按有效感受野（输出对潜变量梯度能量的 90 %）计，4 个残差块的残差部分也只用到 3×3 个潜变量像素。可见中高信噪比下的增益来自非线性的层数，而不是感受野。只在 0 dB，B 最好、A 最差：接收潜变量接近纯噪声时，大的感受野可以在更大范围内平均噪声，这与入口线性 3×3 卷积主要在低信噪比下起作用一致。A 与 4 个块之间仍差约 0.15 dB，说明非线性层之间的 3×3 空间混合仍有作用，因此每个残差块保留 3×3 卷积。

表 2-9　PWL 个数与感受野的对照（DIV2K 验证集 PSNR，dB）

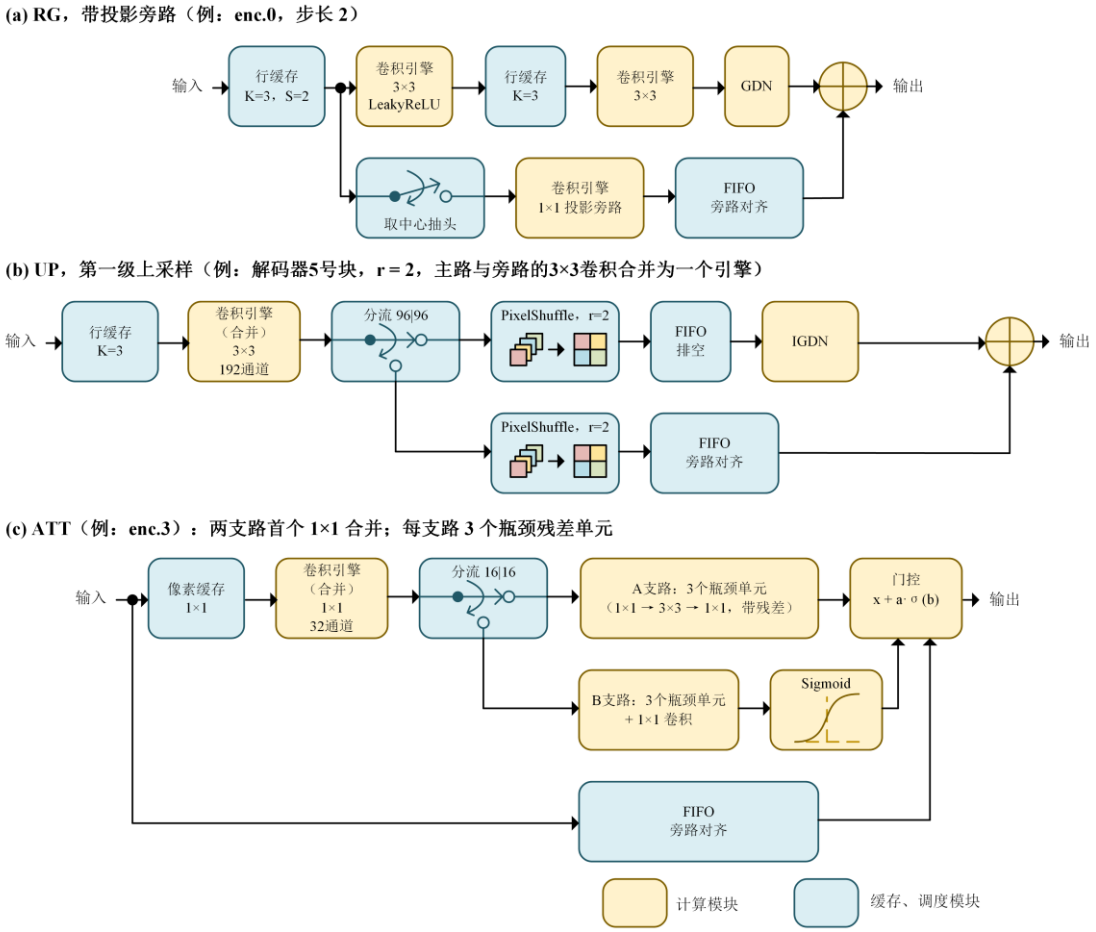
解码器	PWL 个数	理论感受野	每帧乘累加量	0 dB	4 dB	10 dB	20 dB	无噪声
1 个残差块	2	7×7	0.194 G	20.94	26.46	30.62	32.36	32.59
A：1 个 3×3 残差块 + 3 个 1×1 残差块	8	7×7	0.200 G	20.39	26.33	30.91	32.69	32.94
B：1 个残差块，卷积为 9×9	2	19×19	0.345 G	21.10	26.55	30.64	32.28	32.50
4 个残差块（最终解码器）	8	19×19	0.250 G	20.91	26.69	31.06	32.82	33.06

最终的浅层解码器与原解码器相比，W8A12 整数模型在 10 dB 下的 PSNR 差为−0.09 dB（DIV2K）和−0.01 dB（Kodak），完整的 PSNR‑SNR 对比见 3.3.5 节。

（3）块的 RTL 组成

各种块在 RTL 中的组成如图 2-6。

- RG：主路依次是行缓存 → 3×3 卷积（leaky ReLU，步长 1 或 2）→ 行缓存 → 3×3 卷积 → GDN。输入输出通道数或尺寸变化时，旁路是 1×1 投影卷积，它的输入是从主路第一个卷积的窗口中取出的中心像素，不另设缓存。主路与旁路按 y = rsr(a·MA + b·MB, SH)相加，MA、MB 是两路的重定标常数。
- RB：两个 3×3 卷积，旁路是恒等映射，经过 FIFO 对齐。
- RP（解码器）：结构与 RB 相同，两个 3×3 卷积后各接一个 PWL 激活（`axis_pwl`）。
- UP（解码器）：第一个 UP 的主路卷积和旁路卷积读同一个输入，按输出通道拼成一个引擎（192 个输出通道，P = 18），再拆成两路。两路各经 PixelShuffle 上采样 2 倍；主路先进入一个能存放 PixelShuffle 一行输出的 FIFO，再进入 IGDN，旁路进入对齐 FIFO，最后相加。IGDN 逐像素处理较慢时，有了这个 FIFO，合并后的引擎仍可先算完当前行而不被反压。第二个 UP 只有一路：3×3 卷积 → PixelShuffle → sigmoid。
- ATT：特征支路 A 与门控支路 B 各由 3 个瓶颈残差单元组成（1×1 → 3×3 → 1×1）。两支路的第一个 1×1 卷积读同一输入，同样合并为一个引擎。B 支路末端再经 1×1 卷积和 sigmoid 得到门控值，由门控单元计算 y = x + a·σ(b)。



每个卷积引擎自带权重 ROM 与重定标参数；所有连线都是 AXI-Stream（valid / ready / tlast / tuser）。RB 的结构与 (a) 相同，但第二个卷积后没有 GDN，旁路为恒等（只有对齐 FIFO）；解码器的 RP 与 RB 相同，两个卷积后各接一个 PWL 激活。

图 2-6　RG、UP、ATT 块的 RTL 组成（以 enc.0、解码器 5 号块、enc.3 为例）。

（4）行缓存 axis_line_buffer

每个 K×K 卷积前有一个行缓存，它保存最近若干行输入，并按卷积引擎需要的顺序读出窗口（S 为步长，G 为输出通道组数，填充宽度为(K−1)/2）：

```
for (oy, ox)：每个输出像素，光栅顺序，按步长 S 步进
  for g = 0 .. G-1：输出通道分组，G = ⌈Cout/P⌉
    for ky, kx = 0 .. K-1；c = 0 .. C-1

      输出 x[oy·S+ky−(K−1)/2][ox·S+kx−(K−1)/2][c]，越界为 0
```

每个窗口元素读一次就广播给卷积引擎的 P 条通路，所以激活的读带宽与并行度 P 无关；每个输出通道组要把整个窗口重读一遍，一个输出像素共读 G·K·K·C 拍，这就是并行度规划的约束量（本节第 8 小节）。行缓存的输出边带信号为：tlast 表示一遍窗口的最后一个元素；tuser[0]表示第一个元素；tuser[1]表示这一遍属于该像素的最后一组；tuser[2]表示该像素是一帧中的最后一个。

这些输入行以环形方式存放在一块 RAM 中，每 6 个 12 位元素打包成一个 72 位字，正好是 BRAM 和 URAM 一个字的宽度。写入采用整字写：打包寄存器累积当前字的元素，每收到一个元素就把整个字写一次，所以一行可以从字的中间开始，不需要读—改—写。流控规则有两条：

- 一个输入元素只有在它要写入的位置不再被当前输出行使用时才被接收，否则反压上游；
- 一个窗口只有在它需要的输入行、列都已写入时才开始读出；读请求按输出 FIFO 的空位发出，下游可以在任意时刻拉低 `tready`。

128×128 的层缓存 K + 1 = 4 行，使写入下一行与读出当前窗口能够重叠；64×64 的层取 3 行。1×1 卷积不需要行缓存，用 `axis_pixel_buffer` 代替：两个像素槽轮换，一个像素重复读出 G 遍，同时写入下一个像素。

（5）卷积引擎 `conv_engine`

卷积引擎的结构见图 2-7。输入窗口流的每个元素广播给 P 条乘累加通路；第 g 遍时，通路 p 累加输出通道 g • P + p，它的权重来自 P×8 位宽权重字中的第 p 个字节。一遍窗口结束时，P 个累加和被复制到结果寄存器组，再依次送入唯一的一个重定标单元串行处理，累加阵列马上可以开始下一遍，重定标乘法器也只需要一个。

数值处理方式为：

$$a=\sum x\,w+b,\qquad y=\mathrm{sat}_{12}\Big(\mathrm{rsr}\big[\mathrm{sat}_{27}(\mathrm{rsr}(a,p))\cdot M_c,\,s_c\big]\Big)$$

- 激活为 12 位补码、权重为 8 位补码（W8A12），偏置已换算到累加器的尺度；累加器位宽按 $B_{\mathrm{acc}}=B_x+B_w+\lceil\log_2(K^2C_{in})\rceil+1$ 确定。
- 激活函数在 a 上执行：ReLU 取 max(a, 0)；leaky ReLU 在 a < 0 时取 rsr(a, 7)，只需移位。
- $\mathrm{rsr}(v,k)=(v+2^{k-1})\gg k$ 为四舍五入的算术右移。
- 预移位 p 是层常数；M_c（17 位无符号）与 s_c 是逐通道参数，s_c 只存相对于层内最小移位的偏移（2~3 位）。先饱和到 27 位，是为了让乘法正好放进 DSP48E2 的 27×18 乘法器。

重定标乘法按该层的结果产生间隔选择实现方式：结果间隔短的层用 1 个 DSP；间隔足够长的层（P 路结果依次串行相乘所需的总拍数不超过 K •K •C_in）用 `mul_serial`，即基 4 的串行乘法器，每拍处理乘数的 2 位，只用 LUT。

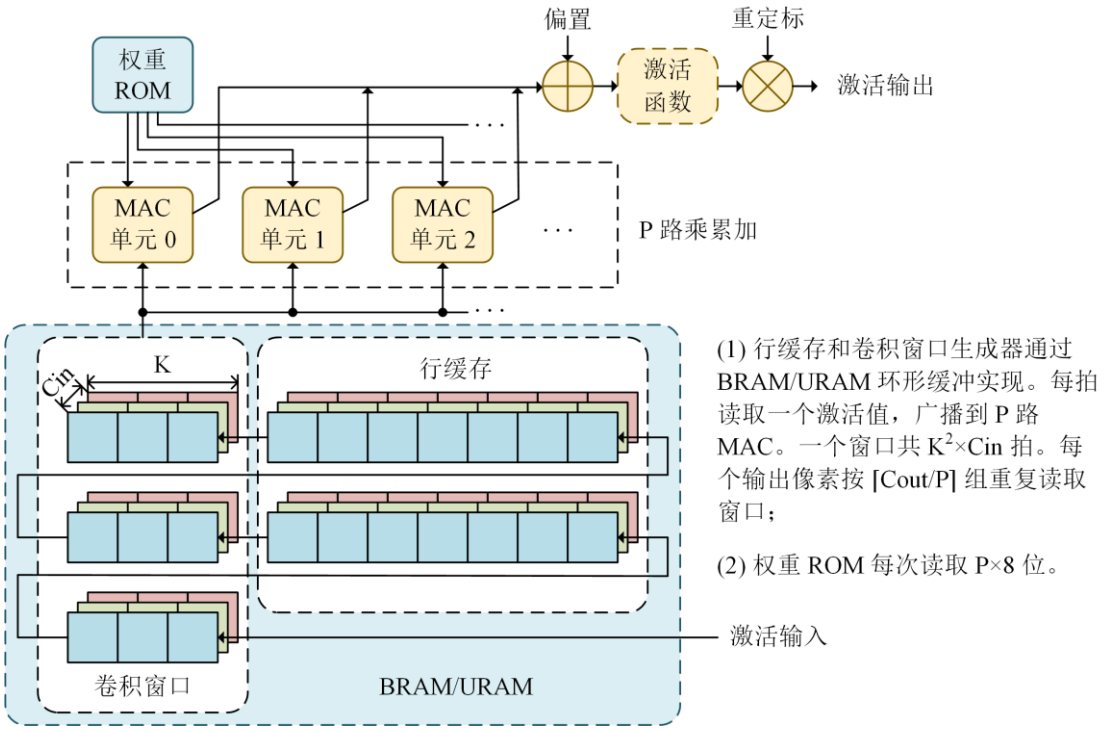


图 2-7　输出通道并行卷积引擎：窗口广播、并行累加与串行重定标。

（6）权重 ROM 的三种组织方式

卷积引擎在第 g 遍读第(ky, kx, c)个元素时需要 P 个权重同时到达，所以权重 ROM 的逻辑形状是“宽 P×8 位、深 G·K·K·C_in”。各层的 P 从 1 到 32 不等，按 ROM 的尺寸在三种方式中选择：

- LUT ROM：小 ROM 用分布式 LUT 实现，例如 ATT 中的 1×1 卷积和 P = 1~3 的层。
- BRAM 宽字 ROM（`rom_banked`）：按每 9 个通路一列（9×8 = 72 位，对应 BRAM 的 512×72 配置）切成若干列，深度超过 512 时再按 512 分体，读延迟固定。适合 P 为 9 的倍数或接近 9 的倍数的层，例如 P = 32、深 288 的层用 4 列共 8 个 BRAM18。
- 顺序读 ROM + 位宽转换（`weight_stream`）：卷积引擎对权重 ROM 只做顺序读（地址 0 到末尾，然后循环），因此权重可以首尾相接地紧凑存放在 NCOL 列 32 位 ROM 中（每列一个 512×32 的 BRAM18），读出端用一个位宽转换器（gearbox）把 32·NCOL 位的 ROM 字流转换成每拍 P×8 位的权重向量，条件是 4·NCOL ≥ P。这种方式下 ROM 的形状与 P 无关。例如 P = 7、深 1440 的层只用 5 个 BRAM18，而按宽字方式需要 3 个 512×72 的存储体（6 个 BRAM18）。

三种方式结合后，每一层的 P 都可以取满足帧率预算的最小整数，不受 BRAM 端口位宽的限制。编码器与解码器各层的配置见表 2-10。

表 2-10　各层的并行度与权重 ROM（同类层合并）

层	C_in→C_out，K，输出尺寸	P	组数	ROM 方式	重定标乘法
enc.0.conv1（步长 2）	3→32，3×3，128^2	3	11	LUT	DSP
enc.0.conv2、enc.1.conv1/2	32→32，3×3，128^2	32	1	BRAM 宽字，4 列	DSP
enc.2、4、5、6 的 3×3 卷积	32→32，3×3，64^2	7	5	顺序读，5 个 BRAM18	LUT
enc.3 各瓶颈 3×3 卷积	16→16，3×3，64^2	2	8	顺序读，2 个 BRAM18	LUT
enc.3、enc.8 的 1×1 卷积	32/16→16/32，1×1，64^2	1	16~32	LUT	LUT / DSP
enc.7.conv1	32→16，3×3，64^2	4	4	BRAM 宽字，1 列 ×3 体	LUT
解码器线性 3×3 卷积、RP 的 3×3 卷积（9 个）	16→16，3×3，64^2	2	8	顺序读，2 个 BRAM18	LUT
解码器第一个 UP（主路与旁路合并）	16→192，3×3，64^2	18	11	BRAM 宽字，2 列 ×4 体	DSP
解码器第二个 UP	24→12，3×3，128^2	12	1	BRAM 宽字，2 列	LUT

（7）GDN / IGDN 与其他算子

GDN[4]的计算为 $y_i = x_i / \sqrt{\beta_i + \sum_j \gamma_{ij} x_j^2}$，IGDN 把除法换成乘法。它需要同一像素的全部 C 个通道到齐后才能计算，又含开方和除法，因此单独做成 `axis_gdn` 模块（图 2-8）：

1. 输入：两个像素槽（LUTRAM）轮换，保存当前像素的 x 和 x^2，平方用 LUT 实现。
2. 乘加：L 条 DSP 通路，第 g 遍时通路 l 计算通道 $g \cdot L + l$ 的分母 D_i：依次读 $x^2[j]$（$j = 0 \cdots C-1$），乘以按通路打包存放的 γ，读到 $j = i$ 时同时锁存 x_i。
3. 迭代运算：每遍得到的 L 个分母连同对应的 x_i 存入分母寄存器，由派发逻辑轮流交给 N 个 `gdn_unit`（开方与除法单元）。开方用逐位的恢复余数法求 $r = \lfloor \sqrt{D} \rfloor$，每拍得到 2 位；GDN 再用恢复余数除法求 $\mathrm{sign}(x) \lfloor |x| 2^F / r \rfloor$，IGDN 用移位相加计算 $x \cdot r$。结果按派发顺序回收，输出保持通道顺序。

4. 重定标：与卷积引擎相同的公式，常数乘法用 LUT 实现。

通路数 L 和单元数 N 按该层的像素率配置，例如 128×128 的 `enc.0.gdn` 用 3 条通路、5 个迭代单元，64×64 的层只用 1 条通路；解码器第一个 UP 中的 IGDN（128×128，24 通道）用 2 条通路。

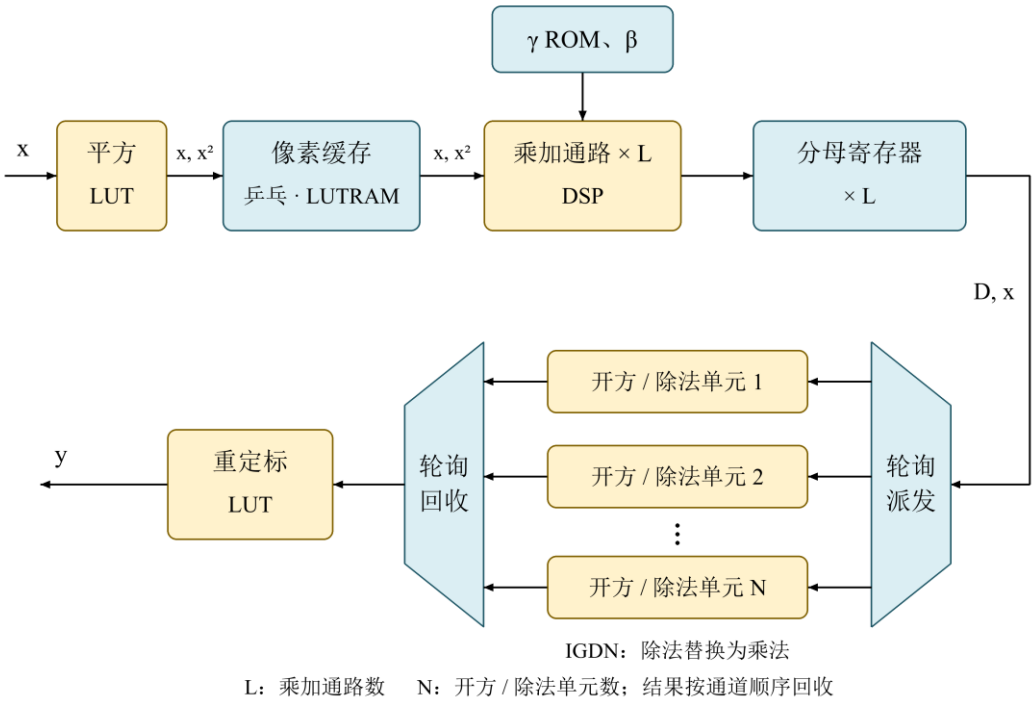


图 2-8　GDN / IGDN 模块的硬件结构（L 为乘加通路数，N 为开方 / 除法单元数）。

其他算子：

- sigmoid（`axis_sigmoid`）：32 段分段线性近似。取 u = |z|，用高位作段号、低位作段内偏移，查两组段系数（存在寄存器中），计算 g = c0 + rsr(c1·off, SHIFT)；负半轴利用 σ(−z) = 1 − σ(z)得到。门控值用 17 位无符号 Q0.16 表示，以便精确表示 1.0。神经网络最后一层的 sigmoid 直接输出 8 位像素 clamp(rsr(g • 255, 16), 0, 255)。乘法用串行 LUT 乘法器，约 12 拍出一个元素，与 ATT 门控支路的元素率匹配。
- 注意力门（`axis_gate`）：y = sat(rsr(x·MA + rsr(a·g, 16)·MB, SH))。
- 残差相加（`axis_add`）：y = sat(rsr(a·MA + b·MB, SH))，可选 ReLU；MA、MB、SH 都是层常数，用 LUT 实现常数乘法。
- PixelShuffle（`axis_pixel_shuffle`）：实现 out[R • y+i, R·x+j, c] = in[y, x, c • R² +i • R+j]。子行 i = 0 的 R 个输出像素直接输出，子行 i ≥ 1 写入行缓冲，一行输入处理完后顺序读出，恰好就是输出行 R • y+1 … R • y+R−1 的光栅顺序。

- 中心抽头（`centre_tap`）：只保留 3×3 窗口流中位于窗口中心（ky = kx = 1）的 C_in 个元素，并按 1×1 卷积的格式重写边带信号。
- 分段线性激活（`axis_pwl`）：解码器 RP 中的激活，每个通道一条分段线性曲线，9 个折点均匀分布在$[-2, 2]$，共 10 段，训练时以 LeakyReLU 初始化。定点实现时，PWL 输入的 12 位定标取为 2 的幂次，使折点落在整数上、段宽为2^L，每个通道存每段的 8 位斜率和每个折点的纵坐标：

$$s = \mathrm{clamp}\left(\left\lfloor (x - T_0)/2^L \right\rfloor + 1, 0, 9\right), \qquad k = \max(s-1, 0)$$

$$\mathrm{acc} = Y_c[k] + S_c[s]\,(x - T_0 - k\,2^L)$$

其中T_0为第一个折点，$Y_c[k]$为通道 c 第 k 个折点的纵坐标（累加器尺度），$S_c[s]$为第 s 段的斜率，之后接与卷积相同的重定标。段内偏移$x - T_0 - k\,2^L$不超过一个段宽，斜率的量化误差不随|x|增大，折点处天然连续。段号只需减法、算术右移和钳位；段内偏移乘斜率、重定标乘法都用串行 LUT 乘法器，不占 DSP，约 21 个时钟周期处理一个元素。64×64×16 的张量在 30 fps 下每个元素有约 127 个周期可用，满足要求。

（8）并行度与片上存储规划

整个神经网络是层间流水线，帧率由最慢的一层决定，比帧间隔快得多的层，多出来的乘累加通路只会空转。因此并行度的原则是让每一层都恰好跟上帧率，即每层取满足帧率要求的最小整数 P。行缓存每拍读出一个窗口元素并广播给 P 条通路，所以一层每帧需要的时钟周期数就是它要读的窗口元素数：

$$P_l = \min\left\{ p : H_o W_o\, K^2 C_{in} \left\lceil \frac{C_{out}}{p} \right\rceil \le 0.8\,\frac{f_{\mathrm{clk}}}{F_{\mathrm{target}}} \right\}, \qquad f_{\mathrm{clk}} = 250\ \mathrm{MHz},\ F_{\mathrm{target}} = 30\ \mathrm{fps}$$

不等式左边是该层每帧要读的窗口元素数，右边是每帧的时钟周期数，留 20 % 余量给流水启动和反压。按此规划，编码器、解码器的乘累加通路分别为 217 和 48 条（图 2-9(a)）。分支合并把 67 个卷积算子映射为 64 个引擎（编码器 53 个、解码器 11 个），合并卷积入口不改变乘累加计算量，只减少重复的窗口读取和引擎数量（合并方式见图 2-6）。

片上存储按每个神经网络核不超过 100 块 BRAM36 的预算分配：xczu5eg 共 144 块 BRAM36，除神经网络外，物理层、SSCC 基线和 PS 接口还要占用约 24 块，神经网络核取 100 块时整片占用约 86 %，留出约 20 块余量。不把 BRAM 用满，一是为物理层和调试逻辑的修改留出空间；二是 BRAM 在器件中按列分布，

用量接近全部时，部分存储只能放到离计算逻辑较远的列上，连线变长，250 MHz 的时序更难收敛。最终实现中 TX、RX 整片 BRAM36 分别占用 72.6 %和 45.5 %，两端时序均收敛（表 3-1）。

为满足这一预算，编码器的部分缓存放进 URAM。URAM 单块容量是 BRAM36 的 8 倍，但只有 72 位×4096 深一种形状，也不能在配置时带初值。权重 ROM 必须带初值，所以权重留在 LUT 和 BRAM 中；激活缓存（行缓存、对齐 FIFO、PixelShuffle 行缓冲）则按“每块 URAM 能替代的 BRAM 数”从大到小放进 URAM，直到 BRAM 用量回到预算以内：大缓存放进 URAM 能省下多块 BRAM，小缓存放进 URAM 则浪费大部分容量。最终编码器规划用 97.5 块 BRAM36 和 4 块 URAM；浅层解码器规模小，规划用 60.5 块 BRAM36，不需要 URAM（图 2-9(c)）。

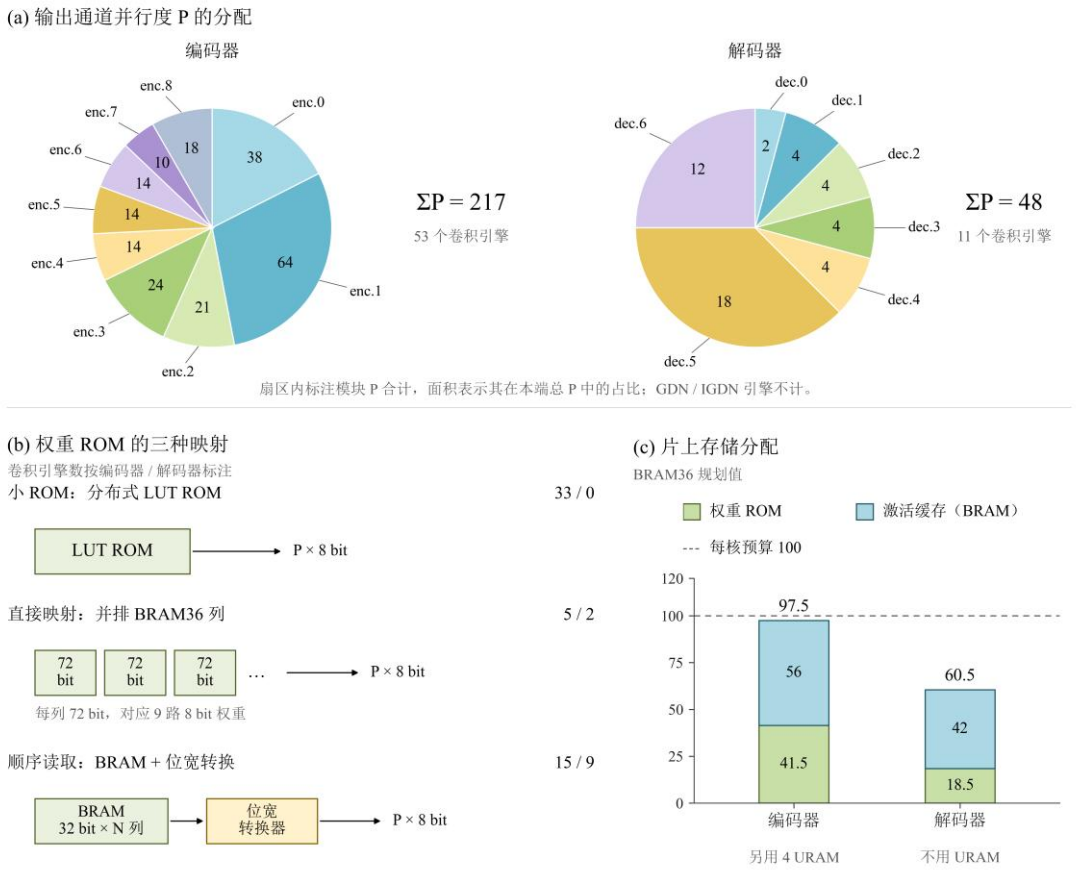


图 2-9　(a)各网络模块的输出通道并行度 P 合计及占比；(b)权重 ROM 的三种组织方式及各自的卷积引擎数；(c)片上存储分配（BRAM36 规划值）。

以存储与计算量都最大的编码器为例，把上述几处设计逐一换成不做该项优化时的常规做法，按同样的规则计算资源，结果见表 2-11（规划值，最终实现的资源见 3.2.2 节）。按层取最小并行度决定了 DSP 用量；三种 ROM 组织方式与 URAM 放置规则一起，使 BRAM36 不超过 100 块，同时 URAM 用量最少。解

码器按同样的规则配置，若各层统一取最大 P（18），乘累加通路为 198 条，是本设计 48 条的 4.1 倍。

表 2-11　本设计与常规做法的资源对比（编码器，规划值）

设计项	常规做法	常规做法的资源	本设计
并行度	各层统一取最大 P（32）	通路 1120 条，为本设计的 5.2 倍	每层取满足帧率的最小 P，乘累加通路 217 条
权重 ROM	全部用 BRAM 宽字 ROM	ROM 共 67.5 块 BRAM36；为保持 100 块 BRAM36，需把更多激活缓冲移入 URAM，URAM 14 块	含 GDN 的 γ ROM：LUT ROM 37 个、BRAM 宽字 5 个、顺序读加位宽转换 15 个，ROM 共 41.5 块 BRAM36
权重 ROM	不用位宽转换（其余不变）	ROM 共 49 块 BRAM36，URAM 6 块；省去约 5.8k LUT	同上
激活缓冲	全部放在 BRAM	BRAM36 共 114 块	按每块 URAM 能替代的 BRAM 数从大到小移入 URAM，BRAM36 共 97.5 块，URAM 4 块
重定标乘法	全部用 DSP	DSP 多 43 个	43 个引擎用串行 LUT 乘法器（约 3.0k LUT），其余 10 个用 DSP

（9）神经网络与物理层的边界

- 编码器出口 `qam_tx`：编码器最后一层输出 12 位整数潜变量。`qam_tx` 用 7 个门限（−343、−229、−115、0、114、228、342，即相邻星座电平的中点换算到整数尺度）把每个元素量化为 8 个电平之一，再映射为 Q10 电平 ±158、±474、±790、±1106。每两个相邻元素配成一个 {Q, I} 符号。模块只有一个暂存 I 的寄存器和一个输出寄存器，突发缓冲由 PHY 负责。
- 解码器入口 `rx_frame`：把每个 {Q, I} 符号按 I、Q 两拍拆回两个通道，并根据张量尺寸重新生成像素级的 `tlast` 和帧级的 `tuser`。入口不做星座判决，均衡后的噪声和残差原样进入解码器。

2.2.3.2 OFDM 发射链

物理层以 802.11a[7]为参考自行设计，主要区别是：调制方式固定为 64QAM、帧长固定为 683 个 OFDM 符号，因此去掉了 SIGNAL 字段；DeepJSCC-Q 模式下神经网络直接输出星座点，比特级处理（比特扰码、LDPC 编码、QAM 映射）只在 SSCC 基线中使用（2.2.3.6 节）。发射链全部工作在 100 MHz，结构见图 2-10，帧格式与子载波分配见图 2-11。

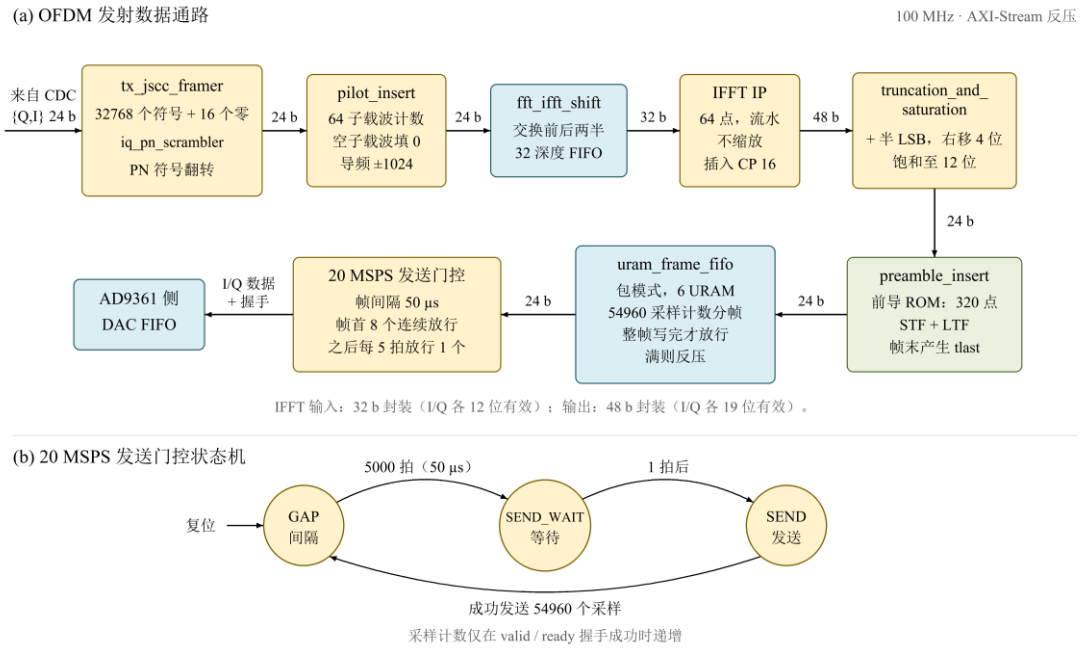


图 2-10　OFDM 发射链（`tx_baseband_jscc.sv`）与 20 MSPS 发送控制。所有模块之间都是带反压的 AXI-Stream，整条链路只在 DAC 前有一个帧缓存。

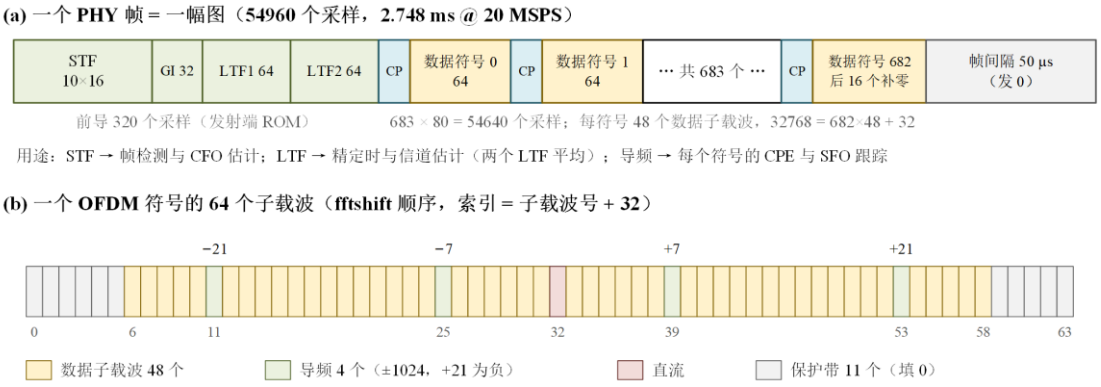


图 2-11　PHY 帧格式（上）与一个 OFDM 符号的子载波分配（下，fftshift 顺序，索引 = 子载波号 + 32）。

各模块依次为：

1. 分帧与 PN 扰码（`tx_jscc_framer`、`iq_pn_scrambler`）：按计数分帧，每帧先放行 32,768 个输入符号，再插入 16 个零符号，凑满 683×48 个数据子载波；输入的 `tlast` 只用来检查接口约定。DeepJSCC 的输出沿 NHWC 顺序是相关的：每个像素的 16 个通道配成 8 个符号，左右相邻像素同一对通道的符号（相隔 8 个符号）相关系数为 0.25~0.42，上下相邻像素（相隔 512 个符号）为 0.36~0.56，直接映射到子载波会使 OFDM 的 PAPR（99 %分位）比随机 64QAM 高约 1~2.5 dB。因此用 802.11 扰码多项式 $x^7 + x^4 + 1$（初值 1011101）生成的伪随机序列对 I、Q 分别做符号翻转（取反）：每个符号 LFSR 走两步，第一位决定 I 是否取反，第二位决定 Q；LFSR 每帧复位。符号翻转不改变星座集合，也不改变噪声

分布，不影响神经网络编解码器，RX 用同一模块再翻转一次即可恢复。

2. 插导频（`pilot_insert`）：对每个 OFDM 符号的 64 个子载波计数。索引 0~5、32（直流）、59~63 填 0；索引 11、25、39、53（子载波 −21、−7、+7、+21）填导频 ±1024，其中 +21 为 −1024；其余 48 个位置依次取数据。插入空子载波和导频时拉低对上游的 `tready`。
3. 顺序变换与 IFFT：`fft_ifft_shift` 用 32 深的 FIFO 交换前后两半，把居中排列的子载波换成 IFFT 的自然顺序，再送入 FFT IP。IP 配置为 64 点、流水线结构、不缩放，并由 IP 插入 16 点循环前缀。
4. 数字功率回退（`truncation_and_saturation`）：OFDM 是多个子载波的叠加，时域峰值远高于平均值，加 PN 扰码后 PAPR 的 99 %分位仍约为 9.6 dB。DAC 只有 12 位，峰值一旦超出满量程就会被削顶，产生带内失真和带外辐射，因此 IFFT 的 19 位输出要按“峰值不削顶”来定标。设计取右移 4 位（加半个最低位舍入，再饱和到 12 位）：数据的每轴 RMS 约为 322，比满量程 2047 低约 16 dB，大于 PAPR 的尾部，实测三张测试图的最大峰值为 1765。右移是 2 的幂次，不需要乘法器；回退损失的发射功率由 AD9361 的 TX 衰减补偿。
5. 插前导（`preamble_insert`）：320 个前导采样（STF 160 + LTF 160）存在 BRAM ROM 中，与数据按相同定标输出，帧的最后一个采样上置 `tlast`。
6. 帧缓存（`uram_frame_fifo`，包模式）：见 2.2.3.4 节。
7. 20 MSPS 发送控制：顶层的三状态机：帧间隔 → 等待整帧 → 发送。发送状态每 5 个 100 MHz 时钟放行 1 个采样（20 MSPS），共 54,960 个；帧开头的 8 个采样连续放行，预先填充 AD9361 一侧的 FIFO。帧间隔状态在两帧之间插入至少 50 μs（1000 个采样）的空闲：RX 的帧检测器确认一帧后要输出一帧长度加 100 个余量的采样，再回到搜索状态，并在下一帧的 STF 中重新累计 64 个满足条件的采样；若两帧背靠背发送（如 PRBS 测试源满速发送时），下一帧的 STF 就会落在上一帧的输出窗口内而被漏检。正常运行时帧间隔由编码器决定（33.3 ms），帧间隔状态只起保底作用。

2.2.3.3 OFDM 接收链

接收链结构见图 2-12。ADC 采样流不能反压，所以接收链各级的平均吞吐都高于 20 MSPS，并在帧检测、定时同步和 CPE 处各有一个小 FIFO 吸收短时的流水停顿。

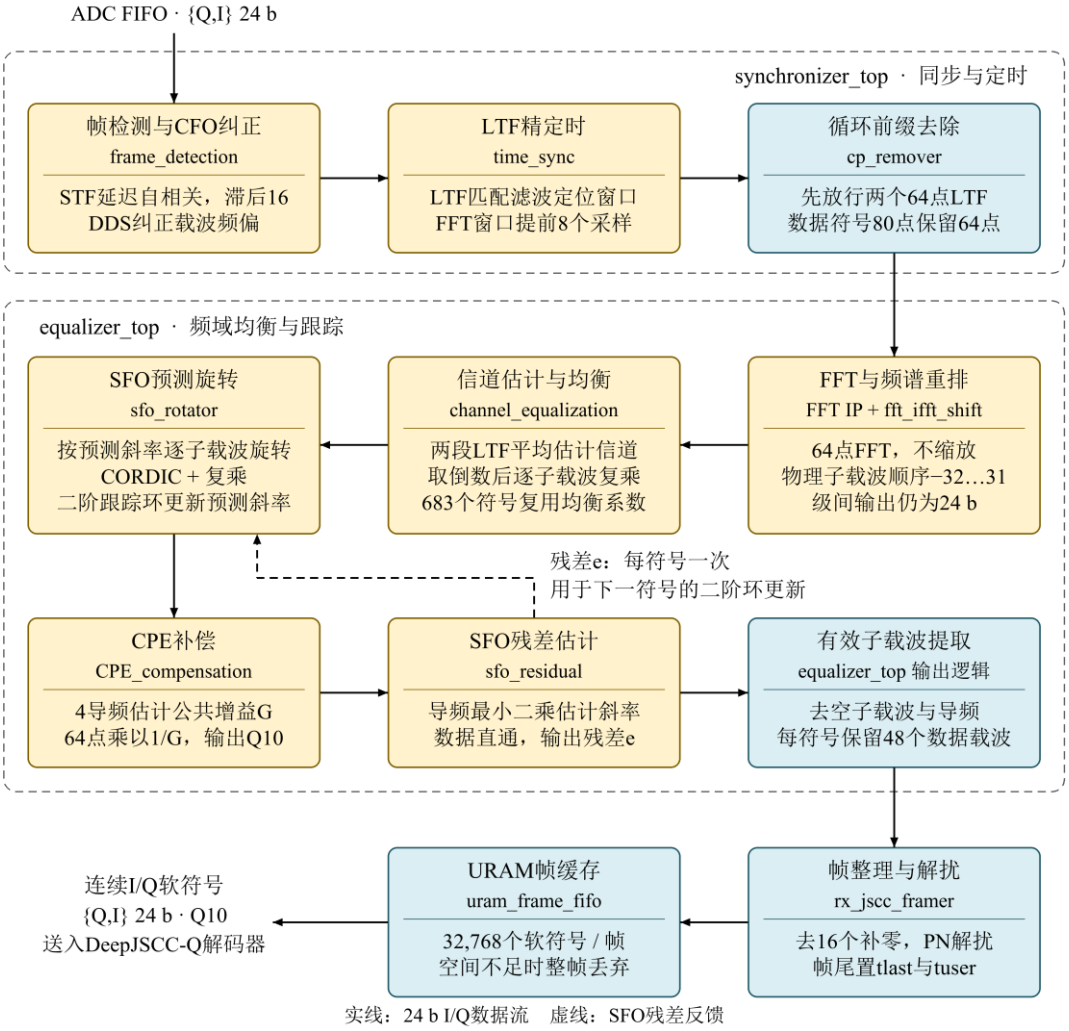


图 2-12　OFDM 接收链（`rx_baseband_top.sv`）。

（1）帧检测与载波频偏纠正 `frame_detection`

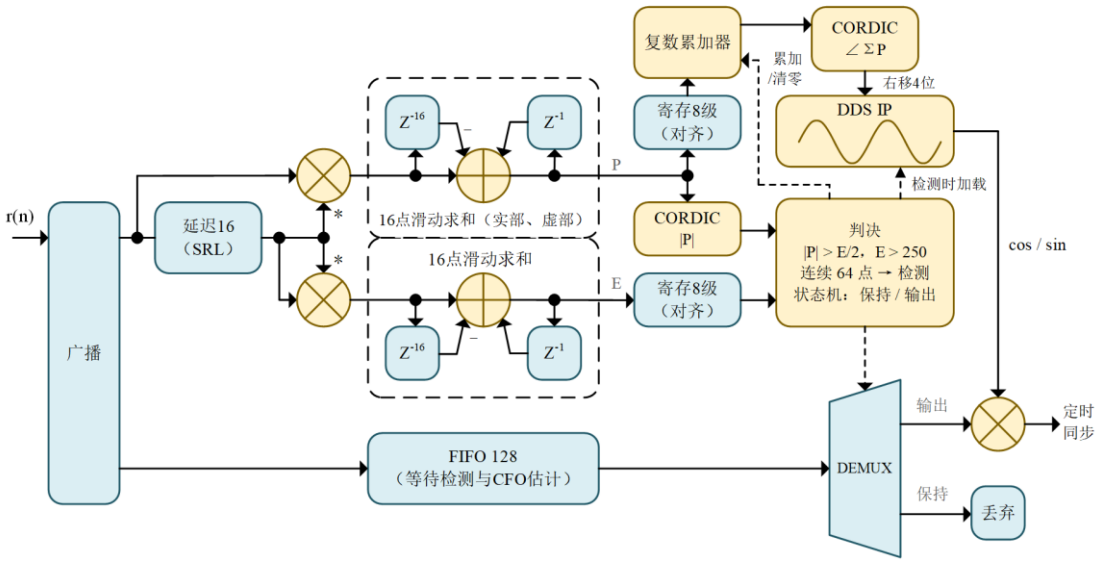
帧检测利用 STF 以 16 个采样为周期重复的特性，采用延迟自相关方法[8]（图 2-13）。输入同时进入 16 点延迟线和 128 深 FIFO；两个复数乘法器分别计算 $r_n r_{n-16}^{*}$ 和 $|r_{n-16}|^2$，再各做 16 点滑动求和，得到自相关 P_n 和能量 E_n；CORDIC IP 求出 P_n 的幅度。判决条件为：

- $|P_n| > E_n/2$，即归一化自相关大于 0.5，E/2 只需右移一位；
- $E_n > 160$（对应 RMS 约 101），只用来排除无信号时的噪声，取为 TX 衰减加大到刚好还能稳定收帧时噪声能量的约 2.6 倍（此时 AGC 增益索引约 56，ADC 处噪声 RMS 约 63，即 −30 dBFS）。

两个条件需连续满足 64 个采样才确认检测。图 2-14 的仿真条件为两板晶振标称偏差的最坏情况（各 ±20 ppm，相对频偏 40 ppm）、半个采样的定时偏移和

10 dB SNR：噪声段和数据段的归一化自相关也会偶尔超过 0.5，但都持续不到 64 个采样；STF 内则连续满足，在帧起点后约 100 个采样处确认。

P_n 的相位是 16 个采样内的载波相位差。频偏由确认检测的 64 个采样上的 P_n 复数累加后估计：P_n 与 E_n 一起经过 8 级对齐寄存器，在判决的同时累加，确认时把累加和的平均送入第二个 CORDIC 求相位，右移 4 位即每个采样的相位增量，配置 DDS IP 产生纠正因子，与 FIFO 中缓存的采样相乘完成频偏纠正。累加和覆盖约 80 个 STF 采样的乘积；第二个 CORDIC 约 25 个时钟（5 个采样）后给出结果，这期间通过的只是 STF 的采样，不影响 LTF。估计精度直接影响 EVM：残余频偏 ε（相对子载波间隔）使各子载波之间产生干扰，其功率约为$(\pi\varepsilon)^2/3$，CPE 跟踪只能去掉公共相位，去不掉这部分干扰。按算法仿真（AWGN、半个采样的定时偏移、±36.6 kHz 的随机频偏），频偏估计的均方根误差在 6 dB SNR 下为 9 kHz（子载波间隔的 2.9 %，干扰约 −26 dB），15 dB 下为 3.5 kHz。滞后 16 的自相关可估计的频偏范围为 ±fs/32 = ±625 kHz，即使在 AD9361 最高的 6 GHz 载频下，40 ppm 的频偏也只有 240 kHz，在此范围之内；估计的残余误差由后面的 CPE 跟踪逐符号补偿。复数乘法的结果先取 14 位、再饱和到 12 位，防止满量程附近的样点溢出反号。



* 表示取共轭；复乘结果先取 14 位、再饱和到 12 位；∠ΣP 是 16 个采样内的载波相位差，右移 4 位即每个采样的相位增量。

图 2-13　帧检测与载波频偏纠正的硬件结构（`frame_detection.sv`）。

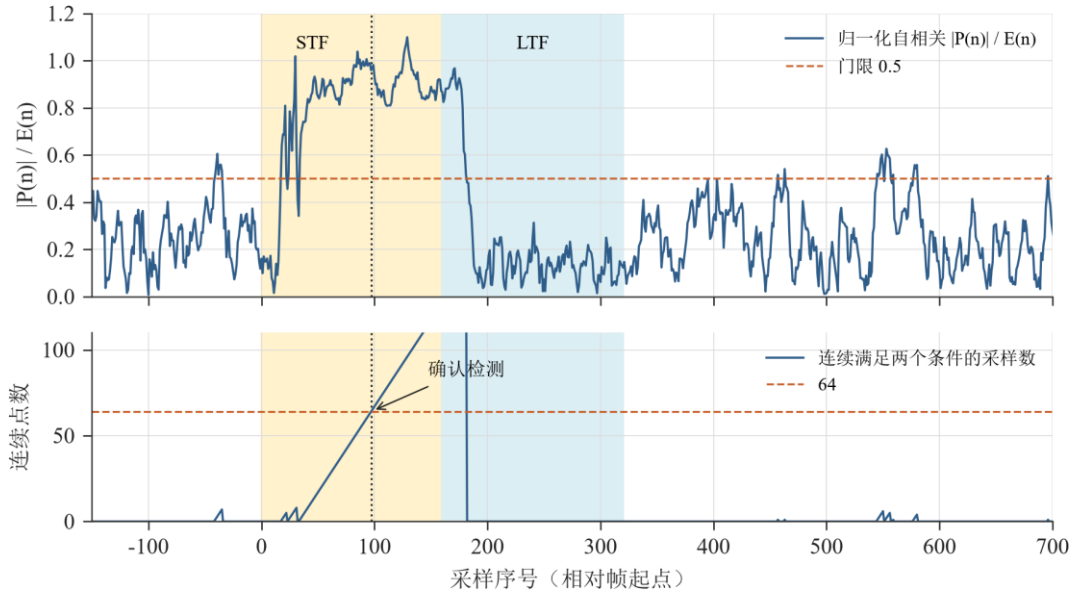


图 2-14　帧检测算法仿真：上为归一化自相关与门限，下为连续满足两个条件的采样数（40 ppm 频偏、半个采样定时偏移、10 dB SNR）。

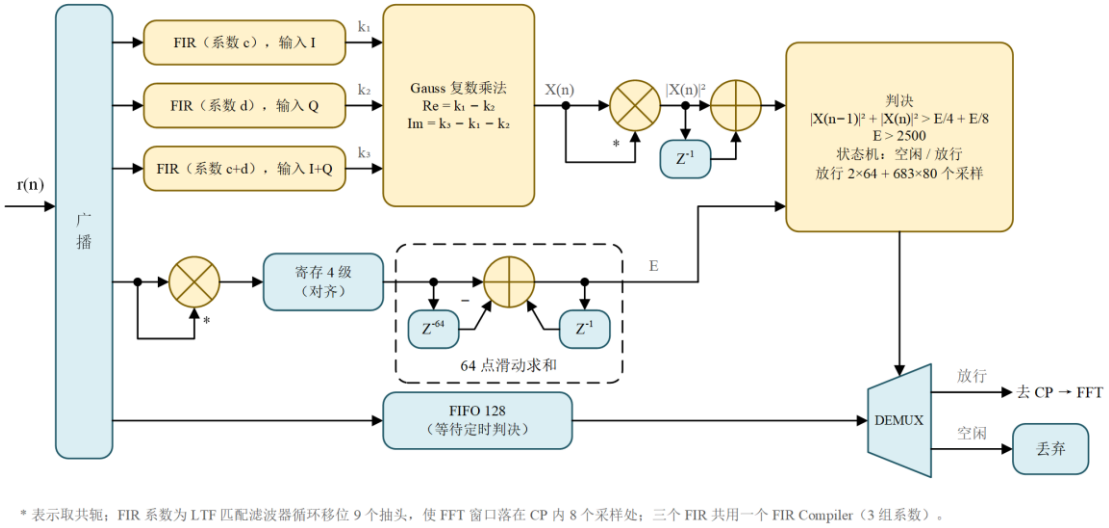
（2）LTF 精定时 `time_sync`

精定时用已知的 LTF 序列做匹配滤波（图 2-15）。复数匹配滤波用 Gauss 复数乘法算法以 3 个实数 FIR 实现：$k_1 = \mathrm{FIR}_c(I)$，$k_2 = \mathrm{FIR}_d(Q)$，$k_3 = \mathrm{FIR}_{c+d}(I+Q)$，则实部为 $k_1 - k_2$，虚部为 $k_3 - k_1 - k_2$，比直接实现少用 1/4 的乘法器。三个 FIR 共用一个 FIR Compiler IP 配置（3 组系数）。互相关的模平方先与前一个采样的模平方相加，再与 64 点滑动能量比较，判决条件为：

$$|X_{n-1}|^2 + |X_n|^2 > 0.375\,E_n, \qquad E_n > 750$$

其中 0.375 = 1/4 + 1/8，只需移位相加。能量门限 750 对应 RMS 约 110，约为同一条件下（TX 衰减加大到刚好还能稳定收帧时）噪声能量的 3 倍。它不能设得接近前导电平：前导能量在门限附近起伏时，LTF1 处可能不满足而 LTF2 处满足，定时晚 64 个采样，整帧解错。判决量取相邻两点之和，是因为两块板的采样相位互不相关：最坏时 LTF 位于两个采样的正中间，相关峰被分到相邻两个采样上。图 2-16 的仿真中，整数采样对齐时 LTF1 的归一化相关峰 ρ^2 为 0.91；偏移半个采样时两点分别只有 0.54 和 0.44（见图中局部放大），与门限对应的 $\rho^2 \approx 0.46$ 相差不到 0.1，SNR 稍低就可能都低于门限；两点之和为 0.98，与采样相位无关，数据段的两点和不超过 0.15。按算法仿真（每个 SNR 500 帧，随机定时偏移与频偏），2 dB 以上全部帧都定在 LTF1 上。两点和只多一个寄存器和一个加法器，触发时刻最多晚一个采样，在后述 8 个采样的余量之内。

匹配滤波器的系数整体循环移位了 9 个抽头，使检测提前 9 个采样触发，LTF1、LTF2 和所有数据符号的 FFT 窗口都落在 CP 内 8 个采样处。这一固定偏移只在各子载波上引入固定相位，会被信道估计吸收；同时给采样频偏引起的定时漂移留出 8 个采样的余量（见本节第 4 小节）。



* 表示取共轭；FIR 系数为 LTF 匹配滤波器循环移位 9 个抽头，使 FFT 窗口落在 CP 内 8 个采样处；三个 FIR 共用一个 FIR Compiler（3 组系数）。

图 2-15　LTF 精定时的硬件结构（`time_sync.sv`）。

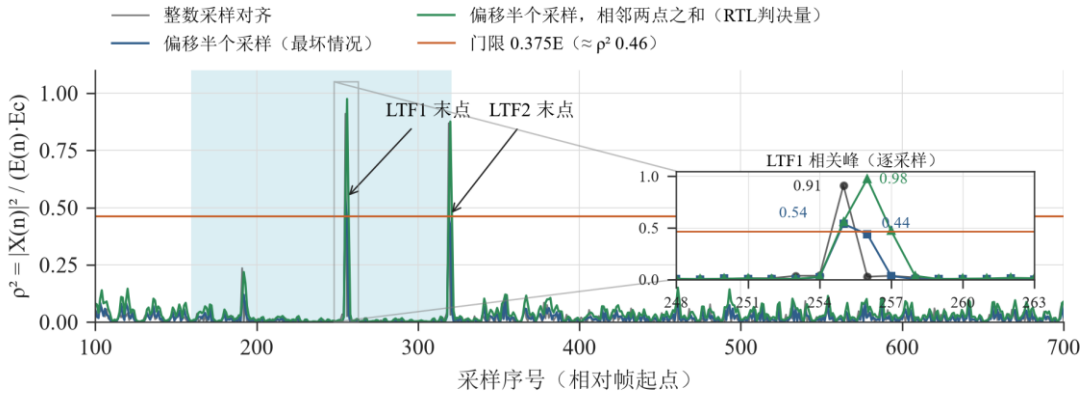


图 2-16　LTF 匹配滤波的归一化相关（10 dB SNR、40 ppm 频偏）：整数采样对齐、偏移半个采样，以及偏移半个采样时相邻两点之和。

（3）去 CP、FFT 与信道估计

`cp_remover` 先放行 LTF1、LTF2 的两个 64 点窗口，此后每 80 个采样丢弃前 16 个。FFT IP 为 64 点、不缩放，输出为自然顺序：序号 0~31 对应子载波 0~31（DC 与正频率），序号 32~63 对应子载波 −32~−1（负频率），而物理频谱是按负频率 → DC → 正频率排列的。若直接使用自然顺序，按子载波号线性变化的量会在序号 31 与 32 之间跳变。例如定时偏移或采样频偏使子载波 k 的相位旋转 $2\pi k\delta/64$（δ 为偏移的采样数），按物理顺序是一条过 DC 的直线，按自然顺序却

断成两段，对 4 个导频做直线拟合得到的斜率甚至与真实斜率反号（图 2-17）；`sfo_rotator` 逐子载波累加相位，也要求子载波按物理顺序连续。因此 FFT 输出先经 `fft_ifft_shift` 交换前后两半，换回物理顺序，再做信道估计与跟踪。

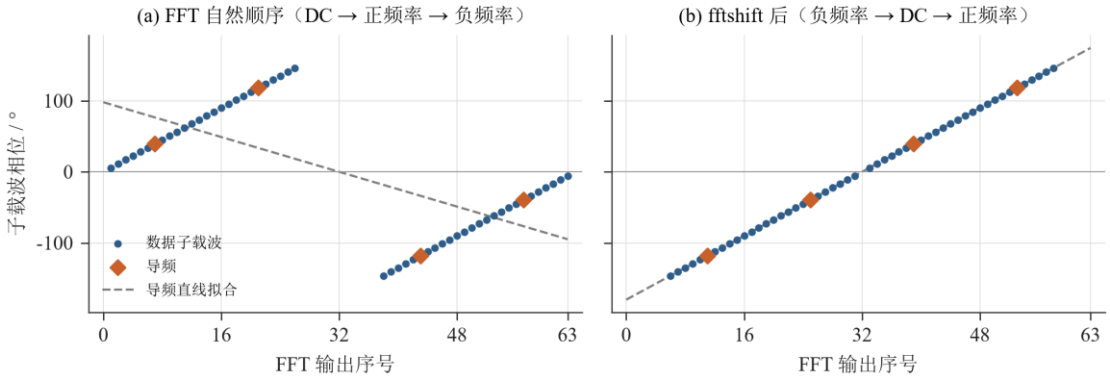


图 2-17　1 个采样的定时偏移在各子载波上产生的相位（仿真）：(a) FFT 自然顺序，相位在中间跳变，导频直线拟合的斜率与真实值反号；(b) fftshift 后相位为一条直线，拟合斜率正确。

`channel_equalization` 做最小二乘信道估计：

1. LTF1 乘以已知的 ±1 后存入 64×24 的分布式 RAM；
2. LTF2 到达时与同一子载波的 LTF1 求平均，得到 $\hat{H}_k$，两次平均使估计噪声降低 3 dB；
3. `reciprocal_cplx` 计算 $1/\hat{H}_k = \hat{H}_k^* / |\hat{H}_k|^2$，两个除法器 IP 并行求实部、虚部；
4. 64 个倒数在整帧 683 个数据符号上重复使用，与每个数据符号逐子载波相乘（迫零均衡）。

帧内的时变由后面的 CPE 和 SFO 跟踪补偿。

（4）SFO 跟踪与 CPE 补偿

每块射频子卡的参考晶振标称精度为 ±20 ppm，两块板之间最坏的相对偏差为 40 ppm。AD9361 的采样时钟与本振来自同一参考，因此采样频偏（SFO）也按 40 ppm 设计。一帧有 54,960 个采样，40 ppm 的 SFO 到帧尾累积 2.2 个采样的定时漂移，在边缘子载波（k = ±26）上对应 $2\pi \cdot 26 \cdot 2.2/64 \approx 5.6$ rad 的相位旋转，远超 64QAM 的判决容限（最外角点约 7.7°）。DeepJSCC 不做判决，但这一旋转会直接成为解码器输入的误差，所以必须逐个 OFDM 符号跟踪。前述 8 个采样的 FFT 窗口提前量大于 2.2 个采样，保证整帧内窗口不移出 CP。

跟踪方法参考文献[9]，跟踪环由三个模块组成（图 2-18）：

- `sfo_rotator`（在 CPE 之前）：对第 n 个数据符号的第 k 个子载波（k = −

32…31）乘以 $e^{-jkA_{\text{pred}}}$，$A_{\text{pred}} = A + R$ 是本符号预测的相位斜率（rad/子载波）。

相位由累加器逐子载波递减 A_{pred} 生成，经 CORDIC 得到 cos/sin。相位以

π 为单位定点表示（2^{29} 对应 π），截取高位送 CORDIC，相位超过 ±π 时自然回绕，所以累积漂移可以超过一个采样。

- `cpe_compensation`：4 个导频按已知极性求和，得到公共增益 G 的估计；`reciprocal_cplx` 求 1/G 后与该符号的 64 个子载波相乘（数据在 64 点 FIFO 中等待 1/G），同时补偿公共相位和缓慢的幅度变化。输出直接是 Q10，即解码器的输入尺度。
- `sfo_residual`（在 CPE 之后）：数据直通，只在导频经过时累加残余相位斜率的最小二乘估计

$$L = -21\,\mathrm{Im}\,p_{11} - 7\,\mathrm{Im}\,p_{25} + 7\,\mathrm{Im}\,p_{39} - 21\,\mathrm{Im}\,p_{53}$$

导频极性已并入符号，$21 = 16 + 4 + 1$，$7 = 8 - 1$，只需移位相加。这里用导频的虚部代替相角，前提是：CPE 已去掉公共相位并把导频幅度归一到 1024，跟踪环又已补偿了预测斜率，剩下的残余相角 θ 很小，于是 $\mathrm{Im}\,p = 1024\sin\theta \approx 1024\,\theta$，不需要再用 CORDIC 求相角。

收到残差 e 后，`sfo_rotator` 按二阶环更新：

$$R \leftarrow R + e/256, \qquad A \leftarrow A_{\text{pred}} + e/8$$

R 是每个符号的斜率增量，对应 SFO 本身。旋转放在 CPE 之前，是因为斜率大时 4 个导频的相位彼此不同，直接求和会使 CPE 估计变小甚至反号（漂移 0.71 个采样时下降 6 dB，1.15 个采样时反号）。`sfo_rotator` 等上一个符号的残差返回后才开始下一个符号，所以第 n 个符号的残差从第 n+1 个符号起生效。A 和 R 在每帧第一个数据符号处清零，以 LTF 时刻的信道估计作为本帧的定时参考。e 换算到相位单位需要乘 170.29，用 $128 + 32 + 8 + 2 = 170$ 近似（误差 −0.17 %），也只需移位相加。整个跟踪环只用一个 CORDIC 和一个复数乘法器。

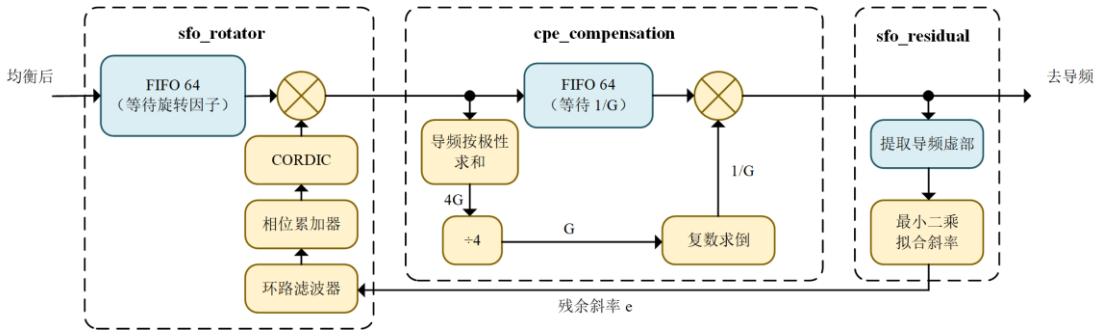


图 2-18　SFO 旋转、CPE 补偿与残余斜率测量构成的二阶跟踪环。

跟踪环的性能用逐符号仿真评估（图 2-19）：仿真按上述 RTL 的运算进行（导频求 CPE、残余斜率最小二乘估计、二阶环更新、残差下一符号生效、每帧开头清零），不加噪声。CPE 补偿是逐符号的前馈：每个符号用自己的 4 个导频估计并除掉公共相位，60° 的 CPE 阶跃在当前符号即被完全消除，且对称导频使 CPE 与斜率估计互不影响，斜率环输出保持为 0。斜率阶跃（定时偏移 0.25~1 个采样）下，二阶环的超调约 14 %，5 %调节时间约 65 个符号，三种幅度的响应几乎重合，说明用导频虚部近似相角在此范围内基本保持线性；线性化后闭环极点约为 0.92 和 0.95，接近临界阻尼。40 ppm 的 SFO 斜坡输入下，不跟踪时帧尾 k = 26 子载波的相位旋转约 320°；跟踪环补偿的相位紧跟实际斜坡，只在起始段有可见的差距（见局部放大），误差在第 14 个符号达到最大 2.8°，第 100 个符号后不超过 0.1°，帧尾为 0，即二阶环对斜坡输入无稳态误差。起始瞬态的 2.8° 也小于 64QAM 最外角点约 7.7° 的判决容限。

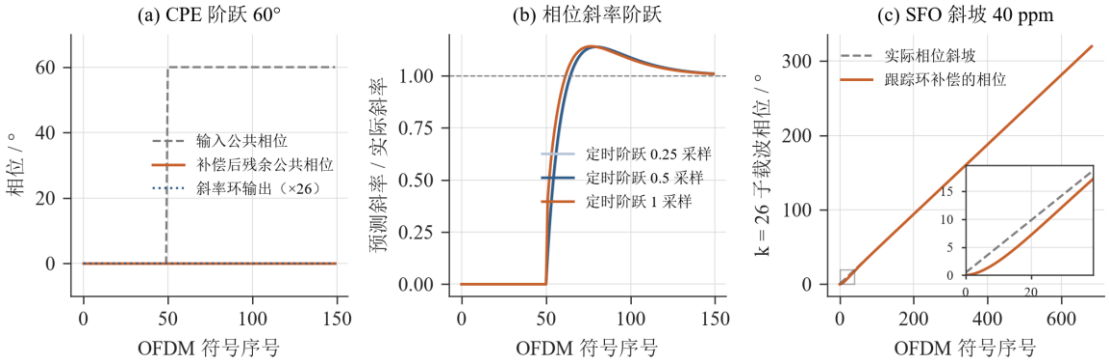


图 2-19　跟踪环的响应（仿真，无噪声）：(a) CPE 阶跃 60°；(b) 相位斜率阶跃，纵轴为预测斜率与实际斜率之比；(c) 40 ppm SFO 斜坡下 k = 26 子载波的实际相位与跟踪环补偿的相位，右下为前 40 个符号的局部放大。

（5）去导频、去补零与 PN 解扰

`equalizer_top` 的输出在空子载波和导频位置拉低 valid，只留下 48 个数据子载波。`rx_jscc_framer` 按计数丢弃每帧最后 16 个补零符号，用与 TX 相同的 `iq_pn_scrambler` 再翻转一次符号，并在第 32,767 个符号上置 `tlast` 与 `tuser`。

2.2.3.4 帧缓存与跨时钟域

`uram_frame_fifo` 用 6 块 URAM288（共 24,576 个 72 位字）存放采样，每个字打包 3 个 24 位采样，有两种工作模式：

- 包模式（TX，DAC 前）：输入满时反压上游；一帧完整写入后才允许读出，所以 20 MSPS 的读出在帧内不会断流。一帧 54,960 个采样等于 18,320 个字，容量约 1.34 帧。缓存放在这里，是因为编码器只能以约每秒 100 万个符号的平均速率产出，而 DAC 必须以 20 MSPS 连续取走 2.75 ms，必须先攒满一帧。
- 整帧丢弃模式（RX，均衡后、解码前）：输入不反压；每帧第一个符号到达时判断剩余空间能否放下整帧，放不下就整帧丢弃，并记入整帧丢弃计数器，输出直通，容量 2.25 帧（32,768 个符号等于 10,923 个字）。缓存放在均衡之后，是因为这里每帧只有 32,768 个有效符号；若放在 ADC 一侧，同样的作用需要存约 66 万个采样。

解码器每帧约需 26 ms（RTL 仿真的帧周期为 6,494,259 个 250 MHz 时钟周期），小于帧间隔 33.3 ms，所以 RX 帧缓存在当前链路下不会写满。跨时钟域统一使用 `axis_async_fifo`（基于 Xilinx XPM `xpm_fifo_axis`，独立时钟、两级同步、首字直通，深度 32，LUTRAM 实现），`tlast` 与 `tuser` 作为数据的一部分一起跨域。

2.2.3.5 AD9361 接口与射频控制

（1）LVDS 数据接口 `ad9361_lvds_mode`

接收方向，`rx_clk_in` 经 IBUFGDS 和 BUFG 成为 `data_clk`，6 对数据线和帧线经 IBUFDS、IDDRE1 转成单沿数据，帧信号的 4 位样式为 1100 时把两拍各 6 位拼成 12 位的 I 和 Q；发送方向由 ODDRE1 产生随路时钟和数据。1R1T 下每个采样占 2 个 `data_clk` 周期，20 MSPS 对应 `data_clk` 40 MHz。`data_clk` 域与 100 MHz 域之间各有一个异步 FIFO：

- DAC 侧 FIFO 为空时输出 0。AD9361 的收发状态机（ENSM）始终保持在 FDD 状态，帧间的“静默”靠发送 0 实现，而不是在发送、接收、待机状态之间切换。原因是 ENSM 的状态切换需要时间，`enable`、`txnrx` 引脚的阶跃也会耦合到射频输出上。
- ADC 侧 FIFO 满时丢样。RX 物理层各级的吞吐都高于 20 MSPS，对 ADC 侧始终保持 ready，所以这个 FIFO 实际不会写满。

（2）SPI 主机与总线归属

`ad9361_spi` 是 20 MHz 的 24 位 SPI 主机。原参考工程中，SPI 主机只要读或写请求为高就连续发起传输，而初始化模块在等待应答期间一直保持请求，于是每次传输结束后都会立刻重发上一条命令，应答与读回数据整体错后一个事务。所有轮询某寄存器直到校准完成的步骤读到的都是上一条命令的结果，轮询提前通过，TX 正交校准等步骤在未完成时就进入了后续配置，TX 的 I/Q 失衡得不到校正。本作品把 SPI 主机改为每个请求脉冲只发起一次传输。

SPI 总线在不同阶段属于不同的主控，由一个三选一的多路选择器切换（图 2-20(a)）：

1. 上电后属于初始化序列执行器 `ad9361_config`，按 ADI 配置工具生成的寄存器序列配置 BBPLL、LO、FIR、校准等；
2. 初始化完成后，TX 板交给 `ad9361_txq_search` 执行 TX 正交校准相位扫描；
3. 扫描完成（射频就绪，同时点亮 LED）后，交给 PS 寄存器驱动的命令通道，供 PS 读写任意寄存器（如 TX 衰减、AGC 状态）。

RX 板的初始化序列由 PS 执行（2.3.2.3 节），SPI 从一开始就交给 PS 命令通道。

（3）TX 正交校准相位扫描 `ad9361_txq_search`

AD9361 的 TX 正交校准借助 RX 回环测量 TX 的 I/Q 失衡，校准能否收敛取决于 RX NCO 的相位设置，而收敛的相位范围随 LO 频率和 FIR 群时延变化，不能使用固定值。ADI 的 no-OS 驱动[11]为此在软件中扫描 32 个相位、取收敛范围的中点（`ad9361_tx_quad_phase_search()`）。本作品把这一过程做成 PL 状态机（流程见图 2-20(b)），上电自动执行，TX 板不依赖软件即可完成射频初始化：

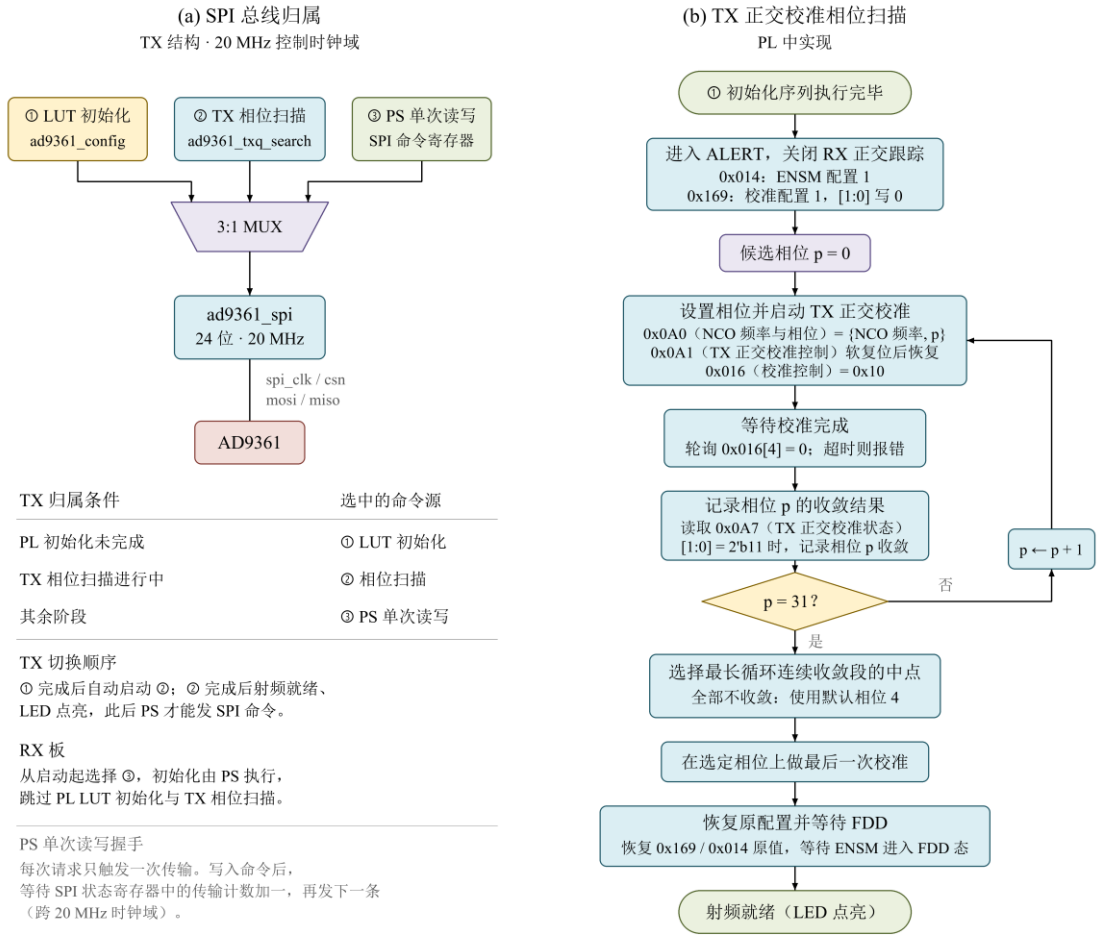


图 2-20　AD9361 控制：(a) SPI 总线的三段归属；(b) TX 正交校准相位扫描流程。

1. ENSM 进入 ALERT 状态，关闭 RX 正交跟踪；
2. 对相位 p = 0…31 依次设置 NCO 相位、启动 TX 正交校准、轮询完成标志，读校准状态判断是否收敛，结果存入 32 位收敛图；
3. 在收敛图中找最长的循环连续段，取其中点（没有任何相位收敛时取默认值 4）；
4. 在选定相位上做最后一次校准，恢复 RX 正交跟踪和 ENSM 状态。

（4）可编程 FIR

AD9361 的 TX 插值滤波链与 RX 抽取滤波链末端各有一个可编程 FIR。本作品重新设计了这两个 FIR（Kaiser 窗，TX 64 抽头、RX 48 抽头），目标是在 52 个子载波占用的 ±8.1 MHz 带宽内保持平坦，同时把滤波器冲激响应的能量集中在 CP（16 个采样）以内，避免符号间干扰。

2.2.3.6 SSCC 基线链路

SSCC 基线在同一物理层、同一星座电平与 TX 衰减、每幅图同样 32,768 个符号的条件下与 DeepJSCC-Q 对比：信源编码为 JPEG（PS），信道编码为 IEEE 802.11 的 HT LDPC 码[7]（即 802.11n 的 LDPC 码，码长 1944、码率 2/3），编译码全部在 PL 中实现（表 2-14）。

（1）信道码的选择

TX 没有反馈，不知道信道 SNR，只能固定一个码率。在 AWGN 信道下，用 DIV2K 验证集 100 幅图、与系统相同的帧（每幅图 32,768 个 64QAM 符号，即 196,608 个编码比特）仿真了几种候选（结果见图 2-21）：JPEG 取能放进载荷的最大质量，帧内任一码字未纠正就可能损坏 JPEG。

- 约束长度 7 的卷积码（码率 1/2，Viterbi 软判决）与 LDPC 2/3 的悬崖都在 15 dB 左右（整帧正确率达到 99 %），但 LDPC 2/3 的载荷多 33 %（16,360 字节对 12,272 字节），悬崖以上的 PSNR 高 1.7 dB（32.39 dB 对 30.65 dB）；
- LDPC 1/2 的悬崖只比 2/3 提前 0.75 dB，载荷却少 25 %；3/4、5/6 在高 SNR 下更高（33.17 dB、33.86 dB），悬崖却推后到 16.75 dB 和 18.5 dB。

按固定码率比较，2/3 在悬崖位置与载荷之间最均衡，因此采用 LDPC 2/3。802.11n 的码表取自 Vivado LDPC IP 自带的码表文件，即与该 IP 完全相同的码。

Vivado 的 LDPC IP 按 5G 的最大提升因子（384）和 Gbps 级吞吐设计，数据通路宽度固定：在 xczu5eg 上 OOC 综合，802.11 标准下的任何配置（全部码、单个码、码表固化、只出硬判决）都占 124.5 块 BRAM36（86 %）和约 55 k LUT，最小的 5G_Lite 架构也要 68.5 块 BRAM36，且只支持 5G NR 码。本系统每帧 196,608 个编码比特，30 fps 时平均只有 5.9 Mbit/s，一帧之内也只是每约 2,700 个 100 MHz 时钟到达一个码字，因此编码器与译码器都自行设计，不用 BRAM。

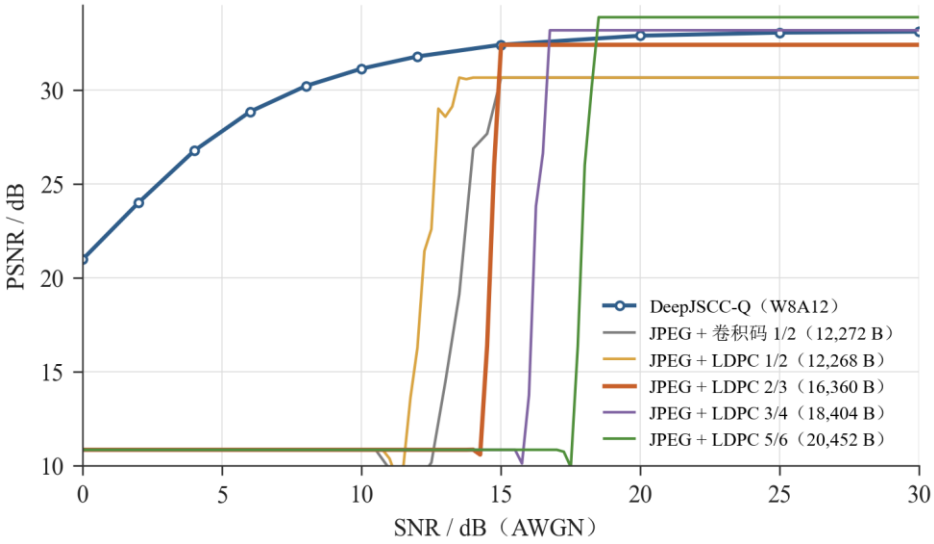


图 2-21　SSCC 候选信道码的 PSNR – SNR（AWGN 仿真，DIV2K 验证集 100 幅图，每幅图 32,768 个 64QAM 符号；括号内为每帧载荷，按 4 字节对齐）。悬崖以下 JPEG 损坏或无法解码，按实际重建图或中灰图计分。

（2）帧结构

表 2-12　SSCC 帧结构

项目	取值
码	IEEE 802.11n LDPC，n = 1944，k = 1296，提升因子 Z = 81，基矩阵 8×24（每行 11 个非零块，共 88 个）
每帧码字	101 个（196,344 个编码比特），其后 264 个零比特，合计 196,608 比特 = 32,768 个 64QAM 符号
每帧信息比特	101 × 1296 = 130,896 比特：SSCC 包 16,360 字节（12 字节包头 + 最多 16,348 字节 JPEG），其后 16 个零比特
扰码	802.11 扰码器（$x^7 + x^4 + 1$），每帧复位，只作用于 SSCC 包
比特到符号	编码比特按顺序每 6 个映射为一个 64QAM 符号（前 3 个为 I、后 3 个为 Q）
交织	不用：一个码字为 324 个符号，约 6.75 个 OFDM 符号，已覆盖全部 48 个数据子载波

包内的 12 字节包头为魔数、帧号、JPEG 长度、质量因子和 CRC32。RX 端的包 RAM 按 32 位字存储，16,360 字节正好是 4,090 个字。码的基矩阵见图 2-22。

（3）编码器 `ldpc_enc`

基矩阵中块(r, c)的移位为 s 时，校验 r • Z + i 与变量 c • Z + (i + s) mod Z 相连。802.11n 的码为准双对角结构：信息部分 16 列，校验部分第 16 列只有 3 个非零块（移位 1、0、1），其后 7 列为双对角。编码分三步：

$$\lambda_r = \sum_{c<16} P^{s(r,c)} u_c, \qquad p_0 = \sum_{r=0}^{7} \lambda_r, \qquad p_{r+1} = \lambda_r + P^{s(r,16)} p_0 + p_r$$

其中 P^s 为 Z×Z 的循环移位矩阵，p_r 为第 r 个 81 位校验块；r = 0 时式中没有 p_r 项，块(r, 16)为零块时没有 p_0 项。第 16 列的两个 P^1 相互抵消，三块之和为单位阵，所以 p_0 就是各 λ_r 之和，不需要求逆。编码过程如下，rot(x, s)表示 81 位向量循环移位，第 i 位取 x 的第(i + s) mod 81 位：

```
λ[0..7] ← 0
for 每个非零信息块 (r, c):            // 128 拍
  λ[r] ← λ[r] ⊕ rot(u[c], s(r,c))
p[0] ← λ[0] ⊕ … ⊕ λ[7]
for r = 0 .. 6:                       // 7 拍
  p[r+1] ← λ[r] ⊕ rot(p[0], s(r,16)) ⊕ p[r]
输出 u[0..15]、p[0..7]                 // 每拍 6 位
```

λ 的累加由 16 选 1 多路选择器、循环移位器和 81 位异或单元完成，每拍处理一个块；p[0]由 8 输入异或树求出，递推使用第二个循环移位器。输入移位寄存器在编码与输出当前码字的同时接收下一个码字；输出移位寄存器每拍送 6 个编码比特给 `qam64_map`，101 个码字后补 264 个零比特并给出帧尾标记。OOC 综合为 3.8 k LUT、5.9 k FF，不用 BRAM 和 DSP。

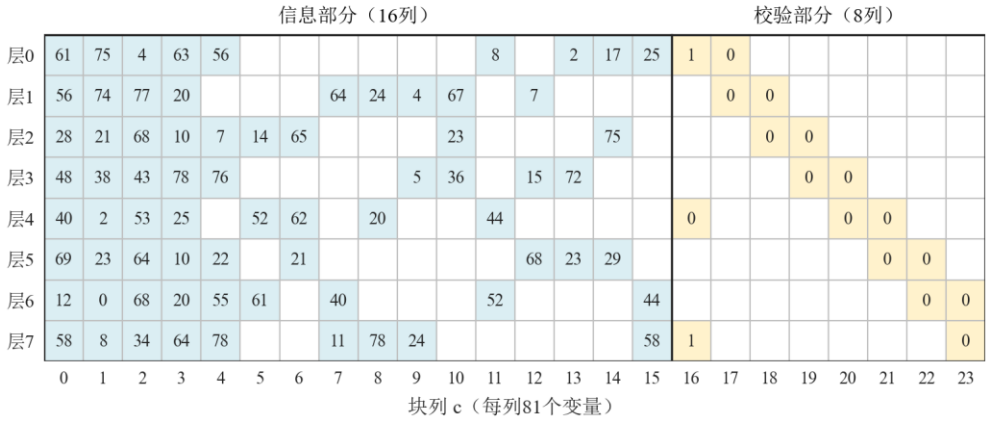


图 2-22　802.11n LDPC 码（码率 2/3，n = 1944）的基矩阵：每格为一个 81×81 的循环移位块，数字为移位值，空白为零块；译码时每行为一层。

（4）软解映射 `qam64_softdemap`

RX 端把均衡后的每个 64QAM 符号分成 I、Q 两路，每路 3 个比特，按格雷映射对应电平 A • {±1, ±3, ±5, ±7}（A = 158，与发送端相同），逐比特求对数

似然比 $L=\ln\dfrac{P(b=0\mid y)}{P(b=1\mid y)}$。按 max-log 近似，

$$L_i \approx \frac{1}{2\sigma^2}\left(\min_{s:\,b_i(s)=1}(y-s)^2-\min_{s:\,b_i(s)=0}(y-s)^2\right)$$

其中 y 为该轴的接收值，s 取该轴的 8 个电平，σ^2 为每轴噪声方差。格雷映射下，括号内的距离差对 3 个比特都是 y 的分段线性函数，各区间的斜率略有不同；按 Tosato 与 Bisaglia 的简化[20]，取统一斜率，只保留其形状：

$$D_0 = y, \qquad D_1 = 4A - |y|, \qquad D_2 = 2A - ||y| - 4A|$$

$D_i > 0$ 对应比特为 1。硬件输出 7 位 LLR：

$$L_i = -\mathrm{sgn}(D_i)\cdot\min\left(\left\lfloor |D_i|/8 \right\rfloor, 63\right)$$

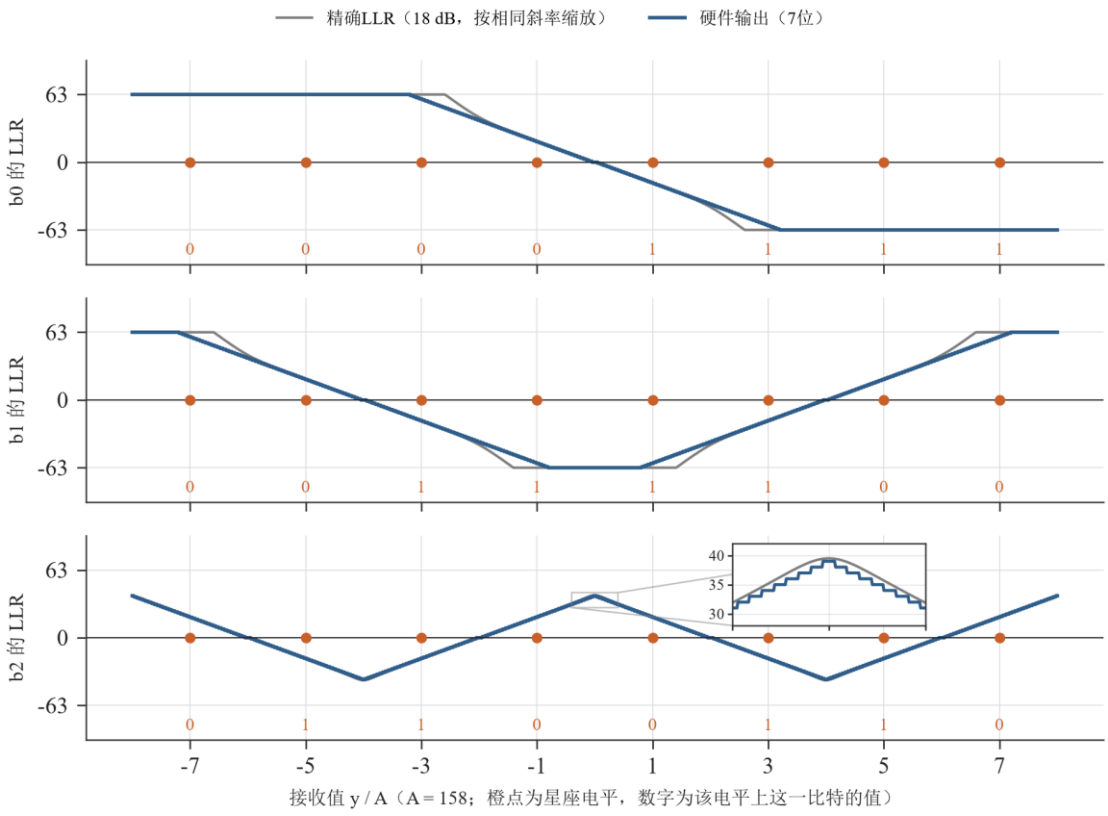


图 2-23　64QAM 单轴 3 个比特的 LLR 随接收值的变化：蓝色为硬件输出（7 位），灰色为 18 dB SNR 下的精确 LLR 按相同斜率缩放后的结果；橙点为星座电平，数字为该电平上这一比特的值；b2 中的小图放大了 y = 0 附近的峰顶。

计算只需绝对值、减法、移位和比较，每个符号在一个时钟内算出 6 个 LLR，

再按 I 路b_0~b_2、Q 路b_0~b_2的顺序逐个送出。省去的比例系数与 SNR 有关，而归一化最小和译码只比较消息的大小，所有 LLR 同乘一个正数时译码结果不变，因此不需要估计噪声方差。右移 3 位与限幅到±63 由定点仿真确定（见下文）。图 2-23 比较了硬件输出与按相同斜率缩放的精确 LLR：两者在判决边界附近重合；b0、b1 的差别只出现在远离边界、接近限幅的区域，b2 不会限幅，差别出现在峰顶和谷底的折点附近，不超过 2 个量化单位（见 b2 的局部放大）。

（5）译码器 `ldpc_dec`

译码采用分层归一化最小和算法：基矩阵的一行（81 个校验）为一层，81 个校验并行处理，每层按列逐块读写。对一层中第 k 个非零块对应的变量，设后验值为 APP、上次该层送来的校验消息为c_k^{old}：

$$v_k = \text{sat}_9(\text{APP}_k - c_k^{\text{old}}), \qquad c_k = \text{sgn}_k \cdot (m_k - \lfloor m_k / 4 \rfloor), \qquad \text{APP}_k \leftarrow \text{sat}_{11}(v_k + c_k)$$

其中m_k为本层其余 10 个v的绝对值中的最小值（即k为最小值位置时取次小值，否则取最小值），sgn_k为其余 10 个v的符号之积，$m - \lfloor m/4 \rfloor$为归一化系数 0.75。每个校验只需保存最小值、次小值（均已乘 0.75，各 8 位）、最小值位置（4 位）和 11 个符号，共 31 位。

定点位宽由逐位定点的 Python 模型仿真确定：信道 LLR 取 7 位（软解映射的 max-log 值右移 3 位后限幅到±63），APP 取 11 位，变量到校验消息取 9 位，在码字误码率 10^{-4}（对应整帧 99 %正确）处与浮点分层译码完全相同；软解映射的简化 max-log 近似也没有可见损失。最大迭代次数取 16：分层译码下 8 次比 16 次晚约 0.5 dB，12 次晚约 0.25 dB，16 次与浮点泛洪调度 40 次相同。某次迭代中每层读到的 APP 符号都满足本层全部校验、且本次迭代没有任何 APP 改变符号时，硬判决必为合法码字，译码提前结束；15 dB 时平均只需 4.6 次迭代。

译码过程见下面的伪代码，j 为块所在的列，c_old[l]为第 l 层上次送出的校验消息（第 1 次迭代为 0）。

第一阶段由循环移位器 A、81 路减法单元和 CNU 寄存器完成，c_old 从消息 RAM 读出后解压；第二阶段由 81 路加法单元和循环移位器 B 完成，v 从 v RAM 读回。存储见表 2-13。

下一层的第一阶段与本层的第二阶段同时进行；相邻两层共用的列（每层的前 4 列都在所有层中出现）必须在本层写回后才能读，用一个 24 位的记分板标记“已读出、尚未写回”的列，读到这样的列时第一阶段等待。层内的列按“与下一层共用的先写、与上一层共用的后读”排序，以减少等待。v RAM 和 CNU 寄存器各有两组，供相邻两层交替使用。一次迭代的实际时序见图 2-24：128 个

时钟中 88 个用于读出，其余 40 个是第一阶段在等待上一层写回共用的列。这样 16 次迭代约 2,100 个时钟，加上装入与输出，仍短于一个码字的到达间隔（约 2,700 个时钟）。LLR 输入为两个码字的乒乓缓存，译出的 1296 个信息比特在下一个码字译码的同时逐比特送出。

```
APP ← 信道 LLR                              // 24 拍
for it = 1 .. 16：
  for l = 0 .. 7：
    // 第一阶段，每拍一块
    for k = 0 .. 10（列 j，移位 s）：
      a ← rot(APP[j], s)
      v[k] ← sat9(a − c_old[l][k])
      更新 min1、min2、最小值位置、符号积
    // 第二阶段，与下一层的第一阶段重叠
    for k = 0 .. 10：
      c ← sgn_k · (m_k − ⌊m_k/4⌋)
      APP[j] ← rot(sat11(v[k] + c), −s)
    c_old[l] ← 本层的压缩消息
  if 各层校验全部满足且 APP 符号未变：break
输出 APP[0..15] 的符号
```

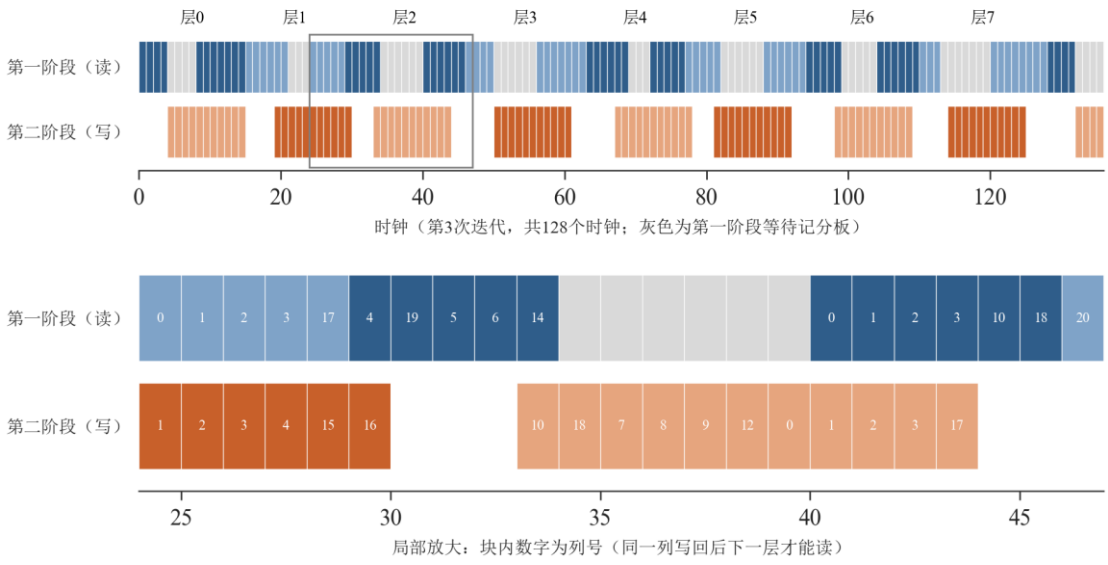


图 2-24　译码器一次迭代内的逐时钟时序（RTL 仿真，15.5 dB 的一个码字的第 3 次迭代）：上为整次迭代，下为局部放大。深浅两种颜色区分相邻的层，灰色为第一阶段等待记分板的时钟，块内数字为列号。

全部存储都用分布式 RAM（表 2-13）。OOC 综合为 27.4 k LUT（其中 3.6 k LUTRAM）、15.7 k FF，不用 BRAM 和 DSP，100 MHz 下时序余量 1.65 ns。

表 2-13　LDPC 译码器的存储

存储	规模	内容
LLR RAM	2 × 24 × 567 位	两个码字的信道 LLR（7 位），一个译码时另一个接收
APP RAM	24 × 891 位	每列 81 个后验值（11 位）；一个写口，两个读口（第一阶段、输出）
消息 RAM	8 × 2,511 位	每层 81 个校验的 min1、min2（已乘 0.75，各 8 位）、最小值位置（4 位）与 11 个符号
v RAM	2 × 11 × 729 位	相邻两层各 11 块的 v（9 位），第二阶段读回

表 2-14　SSCC 基线的 PL 实现

方向	模块	实现
TX	`sscc_tx_bits`	PS 送来的 16,360 字节包，高位先出，用 802.11 扰码器（每帧复位）扰码，再补 16 个零比特
TX	`ldpc_enc`	LDPC 编码，101 个码字后补 264 个零比特
TX	`qam64_map`	格雷映射，电平与 DeepJSCC 相同（$158 \cdot (2k-7)$）
RX	`sscc_sync_fifo`	1024 深，PHY 的输出不能被反压
RX	`qam64_softdemap`	简化的 max-log 软解映射，7 位 LLR
RX	`ldpc_feed`	每帧恰好送出 101×1944 个 LLR：帧提前结束时补 0（擦除），多余的与 264 个填充比特丢弃
RX	`ldpc_dec`	分层归一化最小和译码，最多 16 次迭代；统计未收敛的码字数与迭代次数
RX	`sscc_rx_bits`	解扰、拼字节，写入双缓冲包 RAM，PS 经遥测接口读取

TX 端由 `frame_src_mux` 在帧边界切换 JSCC 与 SSCC 两路符号；RX 端对每个 PHY 帧同时做两种处理，解码器照常解码，`sscc_rx` 同时把它当作 SSCC 帧译码，PS 根据 TX 当前的模式选用其中一种结果。

2.3 软件系统介绍

2.3.1 软件整体介绍

软件分为两部分：

- 板上软件：运行在两块板的 PS 上（PYNQ 3.1，Ubuntu 22.04，Python 3），负责取图、送图、收图、射频控制和显示，结构见图 2-25；
- PC 端工具链：负责模型训练、整数量化与导出、神经网络顶层例化与参数文件生成、比特流构建和测量数据分析，结构见图 2-31。

板上程序全部由 systemd 服务管理，上电约 1 分钟后链路自动建立，触摸屏

显示 GUI，无需连接上位机。

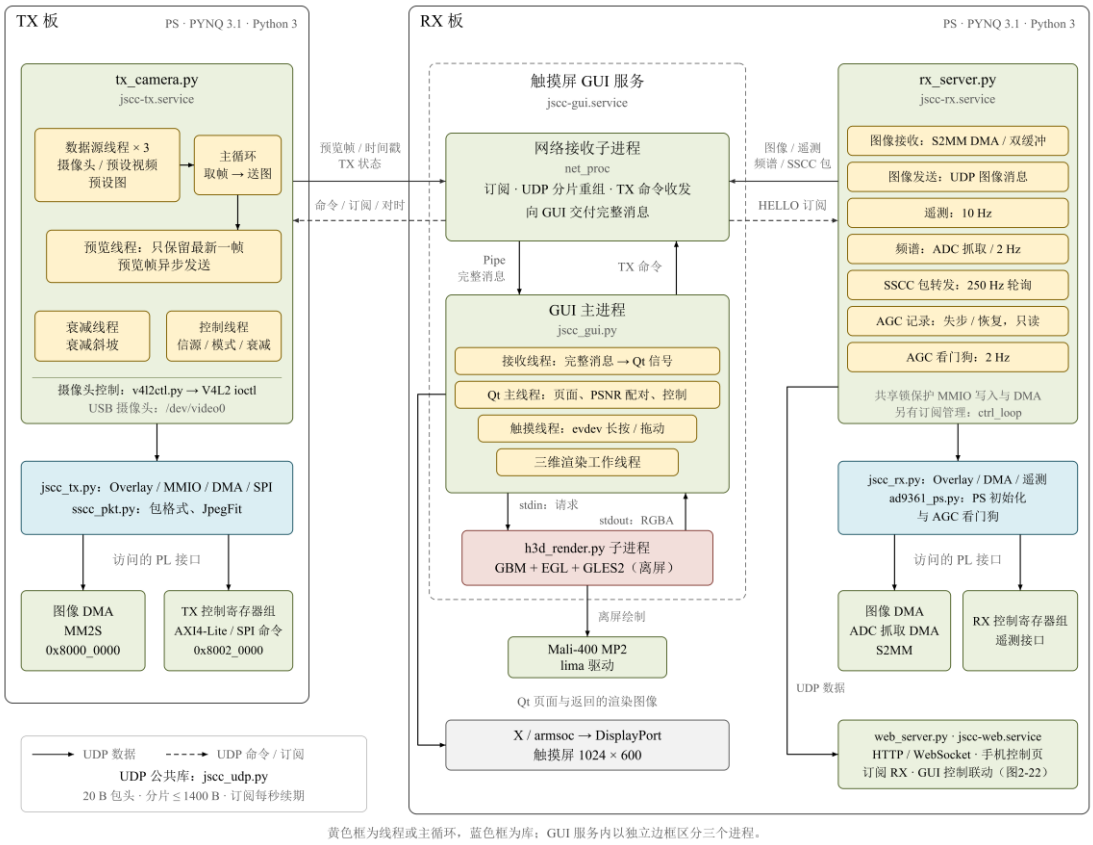


图 2-25　板上软件架构：两块板 PS 上的进程与线程、它们访问的 PL 接口，以及板间、板内的 UDP 连接。

表 2-15　板上软件

板	服务 / 进程	文件	功能
TX	`jscc-tx.service`	`tx_camera.py`	取图、送图、模式与衰减控制、预览转发、手机预览
TX	（库）	`jscc_tx.py`、`sscc_pkt.py`、`v4l2ctl.py`	PL 驱动（Overlay、寄存器、DMA、AD9361 SPI）、SSCC 包格式与 JPEG 质量自适应、UVC 控制
RX	`jscc-rx.service`	`rx_server.py`	收图、遥测、频谱、SSCC 包、AGC 看门狗，经 UDP 转发给 GUI
RX	（库）	`jscc_rx.py`、`ad9361_ps.py`	PL 驱动、AD9361 的 PS 初始化与 AGC 看门狗
RX	`jscc-gui.service`	`jscc_gui.py`	触摸屏 GUI（PyQt5 + pyqtgraph）
RX	`jscc-web.service`	`web_server.py`	手机控制页：两路画面、物理层指标与链路控制（2.3.2.5 节）
RX	GUI 的子进程	`h3d_render.py`	在 GPU 上离屏渲染信道三维曲面
两板	（库）	`jscc_udp.py`	UDP 协议：包头，消息分片与重组

板上进程之间、板与板之间都走 UDP。消息被切成不超过 1400 字节的分片，避免 IP 分片；订阅方每秒向数据源发一次订阅消息，数据源向最近 5 秒内发过订阅消息的地址推送数据，所以板上不需要预先配置对方的地址。

2.3.2 软件各模块介绍

2.3.2.1 TX 主程序 `tx_camera.py`

TX 主程序的流程见图 2-26，它由一个主循环和若干后台线程组成：

- 主循环：取一帧新图像 → 必要时切换模式 → 送入 PL → 转发预览。
- 数据源线程：三个线程分别维护摄像头、预设视频（30 fps 循环播放）和预设图（按 30 fps 重复）的最新一帧，并带序号；主循环只取序号比上次新的一帧，同一帧不会重复送入编码器。摄像头图像经 MJPG 解码、中心裁剪 480×480、用 OpenCV 的区域插值（`INTER_AREA`）缩小到 256×256、BGR 转 RGB，得到的 uint8 (H, W, C)数组正好是编码器要求的 NHWC 顺序；饱和度、对比度、gamma 用 `v4l2ctl.py` 直接调用 V4L2 ioctl 设置。摄像头为可选外设：未连接时信源为预设视频，摄像头线程每 5 秒检查一次设备，运行中插上即可切换，拔下时自动回到预设视频；GUI 选择的信源保存在 `tx_source.txt` 中，重启后沿用。
- 控制线程：接收 GUI 发来的切换信源、切换模式、设置 TX 衰减命令和对时请求，每条命令都回复当前状态。
- 预览线程：把交给编码器的那一帧原图发给 GUI，用于计算 PSNR；只保留最新一帧并在单独的线程中发送，主循环不等网络。

表 2-16　TX 主程序的关键输入输出

函数	输入	输出
`JsccTx.send_image(img)`	(256, 256, 3) uint8	MM2S DMA 送入 196,608 字节；DMA 受编码器反压，返回时编码器已收下整幅图
`JsccTx.send_packet(pkt)`	16,360 字节 SSCC 包	同一 DMA 送入 SSCC 发射链
`JsccTx.set_mode(m)` / `set_source(s)`	工作模式（DeepJSCC-Q / SSCC）；数据源（编码器 / PRBS 自测）	写控制寄存器中的模式位或数据源位，PL 在帧边界切换
`JsccTx.set_tx_atten(db)`	0~89.75 dB	经 SPI 写 AD9361 的 TX 衰减寄存器，立即生效
`JpegFit.encode(img)`	RGB 图像	不超过 16,348 字节的 JPEG、质量因子 q

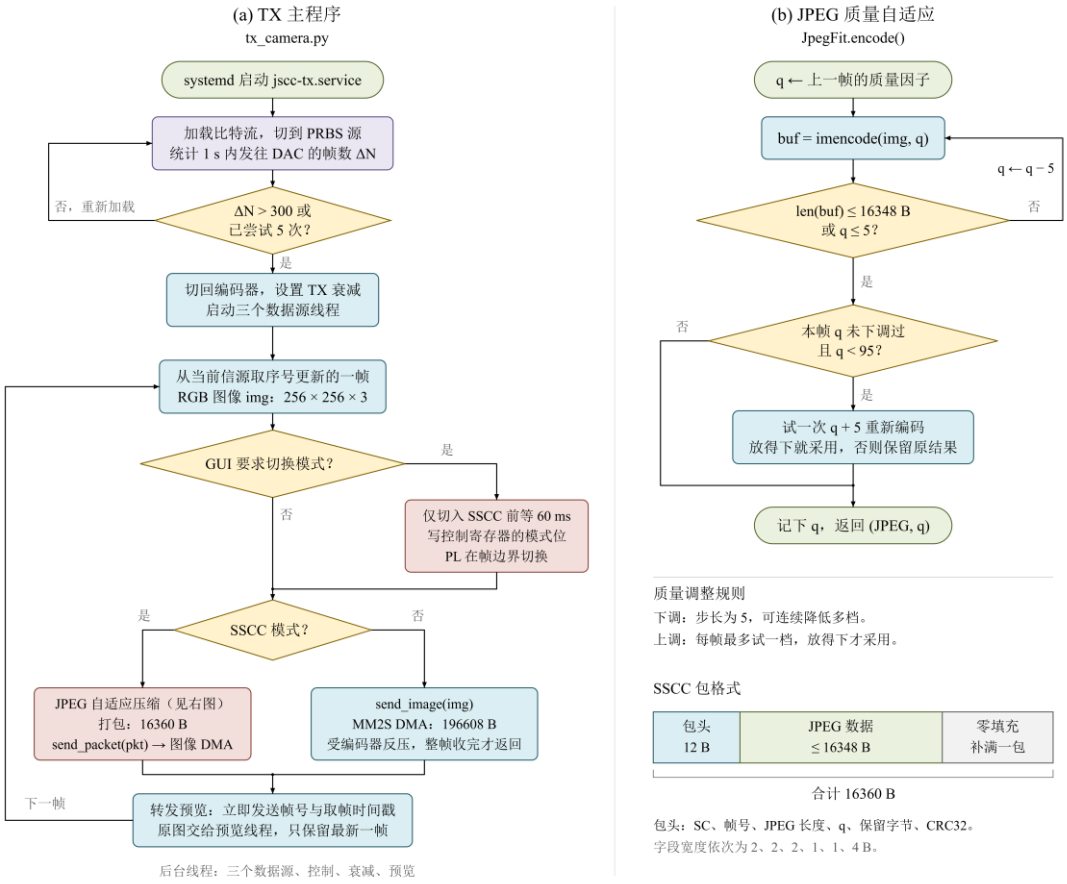


图 2-26　TX 主程序流程（左）与 JPEG 质量自适应（右）。

- 启动自检：TX 板的 AD9361 初始化与相位扫描在 PL 中自动完成，需要几秒。主程序加载比特流后先切到 PRBS 源，1 秒内发往 DAC 的帧计数增加超过 300 帧才认为 DAC 侧已就绪，然后切到编码器；否则重新加载，最多 5 次。
- 模式切换：切换到 SSCC 前先等 60 ms，让编码器把最后一帧（约 33 ms）送完，再写控制寄存器中的 SSCC 模式位；PL 中的 `frame_src_mux` 也只在帧边界切换，不会出现半帧。
- JPEG 质量自适应：为了公平对比，SSCC 每幅图必须和 DeepJSCC-Q 一样只占 32,768 个信道符号，即 JPEG 码流不能超过一个包的 16,348 字节。JPEG 的码长随画面复杂度变化，固定质量因子要么放不下、要么浪费带宽，所以每帧选择能放进包的最大质量因子：从上一帧的 q 开始，放不下就以 5 为步长下调；第一次就放得下且 q < 95，则试一次 q + 5。

2.3.2.2 RX 服务 `rx_server.py`

RX 服务由 7 个线程组成，共用一个锁保护 MMIO 写入和 DMA（表 2-17）。

PS 接收图像的速度即使跟不上，丢帧也只发生在 PL 的 URAM 帧缓存（整帧丢弃），PHY 不会被反压。DMA 等待超时后，用 AXI DMA 的软复位（S2MM 控制寄存器 bit 2）清除挂起的传输，再重新启动通道。

表 2-17　RX 服务的线程

线程	周期	输入	输出
图像接收	每帧	图像 DMA（S2MM），两个缓冲交替；图像转发设为连续模式	一帧 196,608 字节放入发送队列
图像发送	每帧	发送队列	UDP 图像消息
遥测	10 Hz	遥测快照（4 级 × 20 个符号 × 64）、寄存器计数	遥测消息：状态 + 星座数组
频谱	2 Hz	下一帧开始时触发抓取 9,728 个 ADC 采样	帧内与帧前噪声的 Welch 功率谱（256 点，Hann 窗，50 %重叠）
SSCC 包转发	250 Hz 轮询	遥测接口中的 SSCC 包序号与双缓冲；包长从遥测接口读出；每 5 s 记录 LDPC 未收敛码字数与平均迭代次数	SSCC 包消息：16,360 字节
AGC 记录	2 Hz	PHY 同步帧计数、增益	失步 / 恢复时记录 AGC 寄存器快照（只读）
AGC 看门狗	2 Hz	PHY 同步帧计数、增益	必要时复位 AGC（2.3.2.3 节）

2.3.2.3 AD9361 的 PS 初始化与 AGC 看门狗 `ad9361_ps.py`

PS 初始化（`AD9361.init`，图 2-27 左）：ADI 配置工具生成的初始化序列由转换脚本转换成 JSON 操作列表，操作有四种：`w`（写寄存器）、`r`（读寄存器）、`delay`（延时）和 `poll`（轮询某寄存器的若干位直到等于期望值，超时报错并指出是哪一步）。执行顺序为：射频控制寄存器写 001（切到 PS 控制，AD9361 复位引脚 RESETB 拉低）→ 2 ms 后写 011（释放复位）→ 逐条执行操作 → 写 111（初始化完成，PL 释放 AD9361 一侧的 FIFO，并把收发控制引脚 ENABLE / TXNRX 拉高）。RX 的初始化放在 PS，AGC 等射频参数修改后可以直接上板验证，不需要重新综合。

AGC 看门狗（`AgcGuard.check`，图 2-27 右）：AD9361 的快速 AGC（fast attack）[10]是为突发通信设计的：在一个包的前导期间快速调整增益并锁定，锁定后只在检测到过载、更强信号等事件时才调整；信号持续变弱时，它不会主动升高增益，而是由基带在包与包之间重新启动 AGC。本作品由 PS 在链路失步时重启：

- 每 0.5 s 读 PL 的 PHY 同步帧计数和当前增益索引（普通的寄存器读，不经过 SPI），计算最近 0.5 s 的同步帧率；
- 同步率低于 5 帧/s、增益索引小于增益表上限，且距上次复位已超过 3 s 时，经 SPI 复位 AGC 状态机（AD9361 的 0x0FA 寄存器即 AGC 配置 1（增益控制模式），先写 0x05 再写 0xE5）。

判据只用同步率和增益是否到顶两个量，与环境的底噪、接收功率无关；TX 正常发送时同步率约 30 帧/s，看门狗不动作。

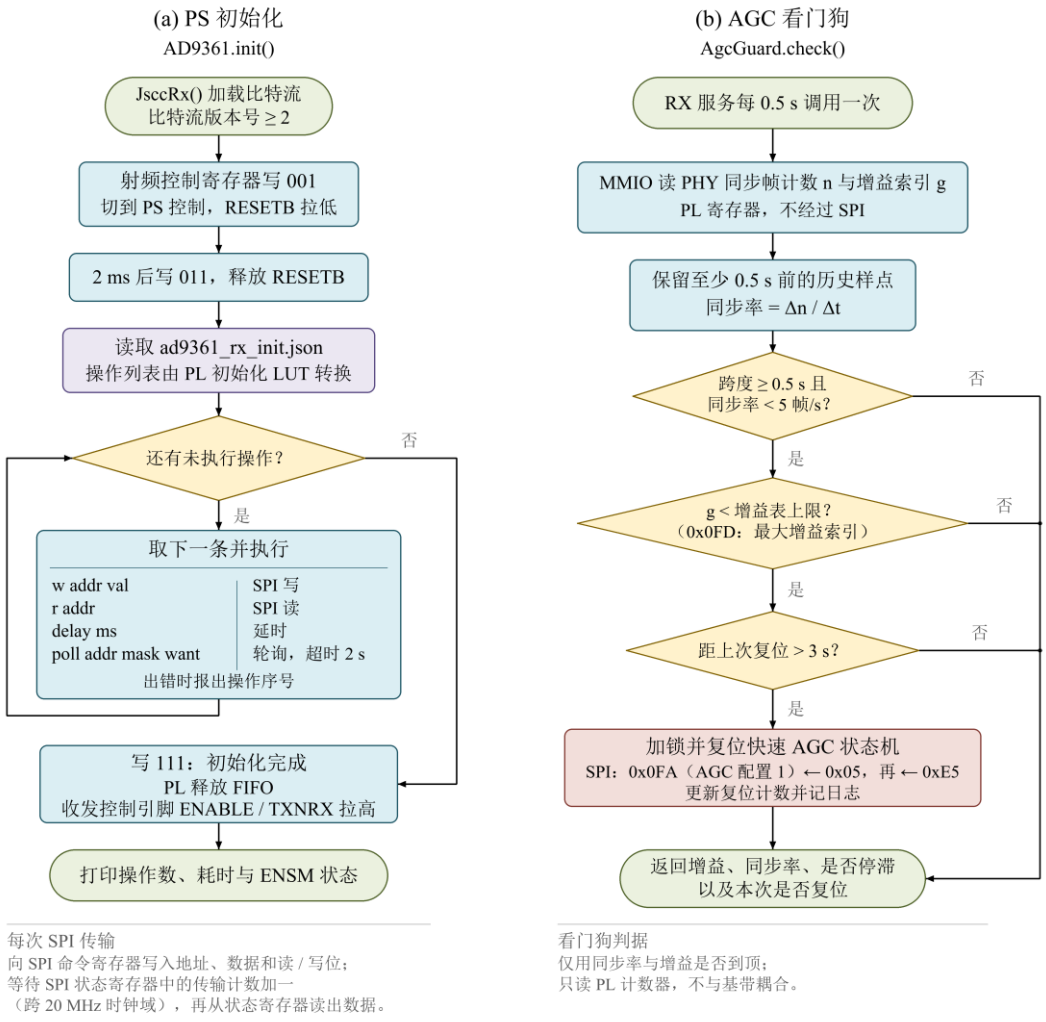


图 2-27　AD9361 的 PS 初始化（左）与 AGC 看门狗（右）。

2.3.2.4 触摸屏 GUI `jscc_gui.py`

GUI 用 PyQt5 + pyqtgraph 编写，在 RX 板上全屏运行，有五个页面：

- 图像链路页：并排显示 TX 原图与 RX 重建图，给出 PSNR、端到端时延和链路统计；页面上有 TX 衰减调节条和 JSCC / SSCC、信源切换按钮；
- 物理层页（图 2-28）：均衡各级的星座图与 EVM、信道 $|H|$ 随时间变化的三维曲面、逐子载波 SNR、EVM 历史、AGC 与同步状态；
- 主题页：切换配色主题；
- 介绍页：系统框图与原理说明；
- 手机页：手机控制页的地址与二维码（2.3.2.5 节）。

RX 解码图没有帧号，PSNR 的计算方法是：在最近 350 ms 收到的 TX 预览帧中，按 1/4 降采样后的均方误差找出最匹配的一帧，再算全分辨率 PSNR；SSCC 帧有帧号，直接配对。UDP 分片的接收与重组放在独立进程中完成，只把完整消息交给 GUI 进程，界面绘制不受网络收包影响。

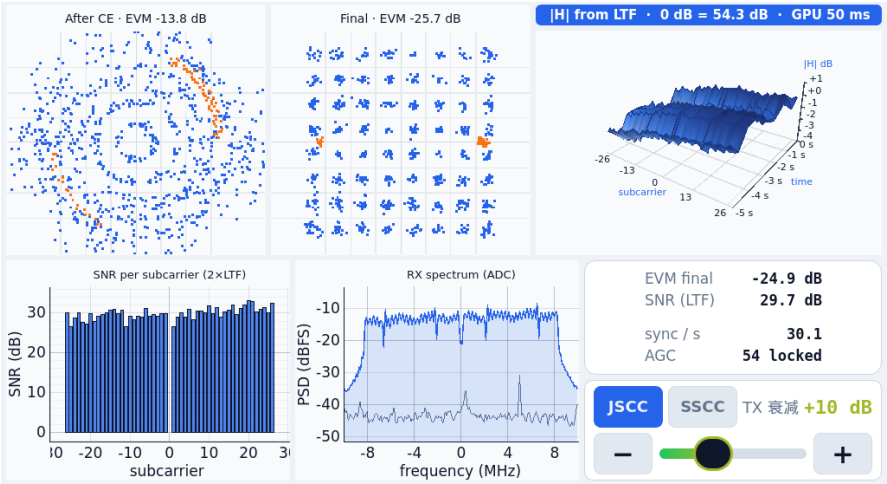


图 2-28　RX 板触摸屏上的物理层页（局部）：星座图、信道 |H| 三维曲面（Mali-400 渲染）、逐子载波 SNR 与链路状态。

2.3.2.5 手机控制页 `web_server.py`

测试远距离传输时，两板间不方便连网线，因此 RX 板上另运行一个网页服务（tornado，端口 8080），用手机浏览器即可查看两路实时画面和物理层指标，并控制链路（图 2-29）。两块板和手机连接同一个手机热点（板载 Microchip WILC 模块，2.4 GHz）：

- 画面：每块板把自己的画面直接发给手机，不经另一块板转发。TX 主程序在 8081 端口提供交给编码器的原图，RX 服务把解码图编码为 JPEG 推送给网页服务，两路都以 MJPEG 流（`multipart/x-mixed-replace`）发送，由浏览器的`<img>`直接显示。
- 地址：热点分配的地址不固定。GUI 的手机页读取 RX 板 WiFi 网口的地址，显示控制页地址和二维码，手机扫码进入；网页服务在 WiFi 子网内广播状态查询，由 TX 的应答得到 TX 板地址，页面再从该地址取 TX 原图。
- 状态与控制：页面经 WebSocket 每秒接收 4 次状态，星座、逐子载波 SNR 和频谱只在物理层页打开时发送。衰减、模式和信源命令以 UDP 直接发给 TX 板；两板连着网线时改由 GUI 转发，触摸屏与手机上的控件保持一致。PSNR 与端到端时延需要把原图和重建图放在一处比较，只在两板连网线时由 GUI 给出，未连线时手机上显示“ - ”。

WILC 模块的上行吞吐只有约 2.5 Mbit/s，一路 30 fps 的 JPEG 流就要占满。若每帧都写入 TCP 连接，来不及发送的数据会在内核缓冲区中排队，画面滞后可达数秒，并随拥塞控制时快时慢。因此每路 MJPEG 限为 15 fps、JPEG 质量 65（每帧约 8 KB，约 1 Mbit/s）；每帧写入前读取套接字中尚未发出的字节数（`SIOCOUTQNSD`），超过 16 KB 时跳过该帧，始终只发最新的一帧；连续 5 s 无法发送则关闭连接。经手机热点实测，一路画面稳定在约 14 fps。WILC 的省电模式会使往返时延升至约 170 ms，开机时将其关闭。



图 2-29　手机控制：RX 板触摸屏的手机页显示控制页地址与二维码（左）；两板未连网线时手机上的控制页（右），PSNR 与时延显示“–”。

2.3.2.6 Mali-400 GPU 离屏三维渲染 `h3d_render.py`

信道 |H| 三维曲面（52 个子载波 × 最近 5 秒）由 ZU5EG 片上的 Mali-400 MP2 GPU（1 个几何处理器、2 个像素处理器，500 MHz）渲染。

驱动：PYNQ-ZU 镜像的内核（6.6.10-xilinx-v2024.1）没有打开开源 Mali 驱动 lima（`CONFIG_DRM_LIMA`）及其依赖 `DRM_SCHED`、`DRM_GEM_SHMEM_HELPER`，而设备树中已有符合 lima 要求的 GPU 节点（`gpu@fd4b0000`），用户态 Mesa 22 也带有 `lima_dri.so`。因此取 Xilinx linux-xlnx 对应标签的 `drivers/gpu/drm/{lima,scheduler}`与 `drm_gem_shmem_helper.c`，在板上以外部内核模块方式编译，开机自动加载。板上内核由 gcc 12 编译而板上只有 gcc 11，所以在一份私有的内核构建树中单独生成 arm64 头文件和 `modpost`（不经过会重跑 kconfig 的顶层 Makefile），并用一个编译器包装脚本去掉 gcc 11 不支持的编译选项。

渲染方式：板上 X 服务的显示驱动（armsoc）不支持 GPU 加速，经 X 的 OpenGL 程序只能得到软件渲染。因此三维曲面由一个独立的渲染进程绕过 X、直接在 lima 的 render 节点上离屏绘制，再把像素读回交给 GUI 当作图片显示，界面其余部分仍由 X 显示（图 2-30）：

1. 初始化 GPU：按驱动名在`/sys/class/drm/renderD*`中找到 lima 的 render 节点，经 `libgbm` 创建 GBM 设备，用 `eglGetPlatformDisplay(EGL_PLATFORM_GBM_KHR)`初始化 EGL，建立 OpenGL ES 2.0 的无表面上下文；渲染目标是一个帧缓冲对象，RGBA 纹理作颜色附件，16 位深度渲染缓冲作深度附件。
2. 生成几何（NumPy，约 7 ms）：T×N 的 |H| 历史（N = 52，T 为最近 5 s 的快照）构成三角网格；法向量由有限差分得到，按 Lambert 模型着色，颜色按 |H| 在三段色图中插值；另有曲面网格线、地面网格和坐标轴的线段。

3. 绘制：一个只做 MVP 变换的顶点着色器加一个直通片元着色器；先画曲面（开启多边形偏移，使网格线浮在曲面上），再画线段；相机参数与 PC 版 pyqtgraph 的视角一致。
4. 读回：`glReadPixels` 读回 RGBA 图像，约 45 ms；投影矩阵预先在 y 方向翻转，读回的数据已是“首行在上”，不需要再翻转。
5. 与 GUI 通信：GUI 中的一个工作线程经标准输入、输出与渲染进程交换数据，请求包含图像尺寸和 |H| 数组，响应包含渲染耗时和像素；同一时刻最多一个请求在途，新数据直接覆盖未发出的旧请求。坐标轴的刻度文字由 Qt 绘制，用同一个 `camera()`和 `project()`函数把三维坐标投影到像素，与 GPU 画出的坐标轴对齐。

没有 lima 驱动时，GUI 自动改用二维瀑布图。每帧渲染加读回约 60 ms，曲面每秒刷新 2.5 次，渲染进程约占一个 CPU 核的 10 %，`rx_server.py` 保持每秒接收 30 帧。

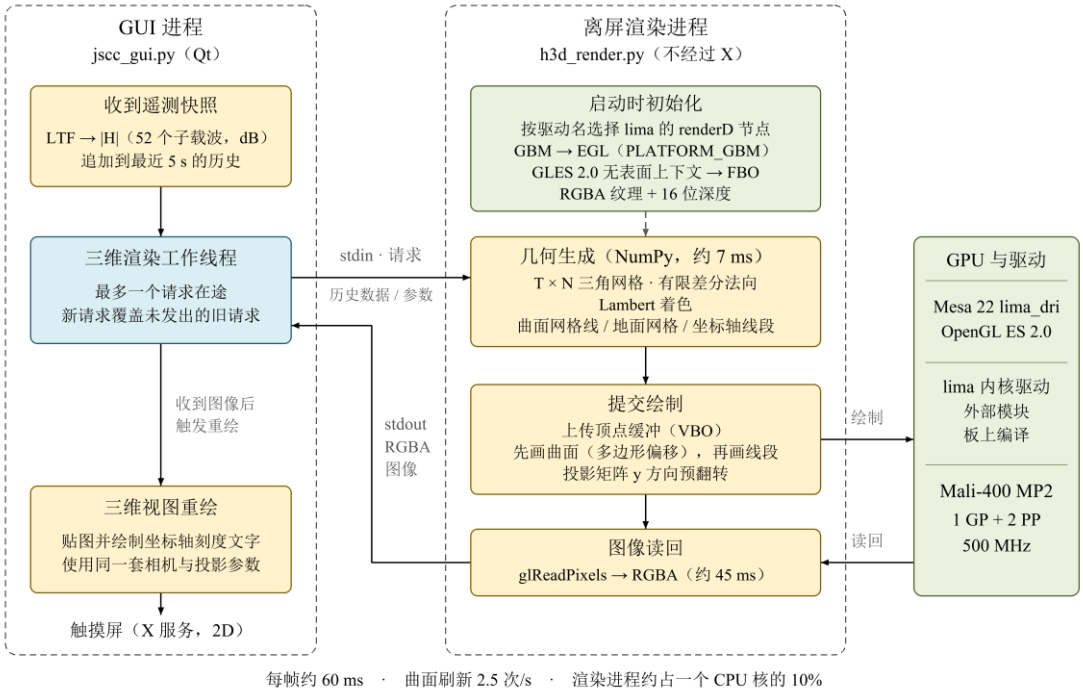


图 2-30　Mali-400 离屏渲染：GUI 进程、渲染进程与 GPU 之间的数据流。

2.3.2.7 PC 端工具链

PC 端工具链从训练一直延伸到上板测量（图 2-31），各环节之间都以文件交接。

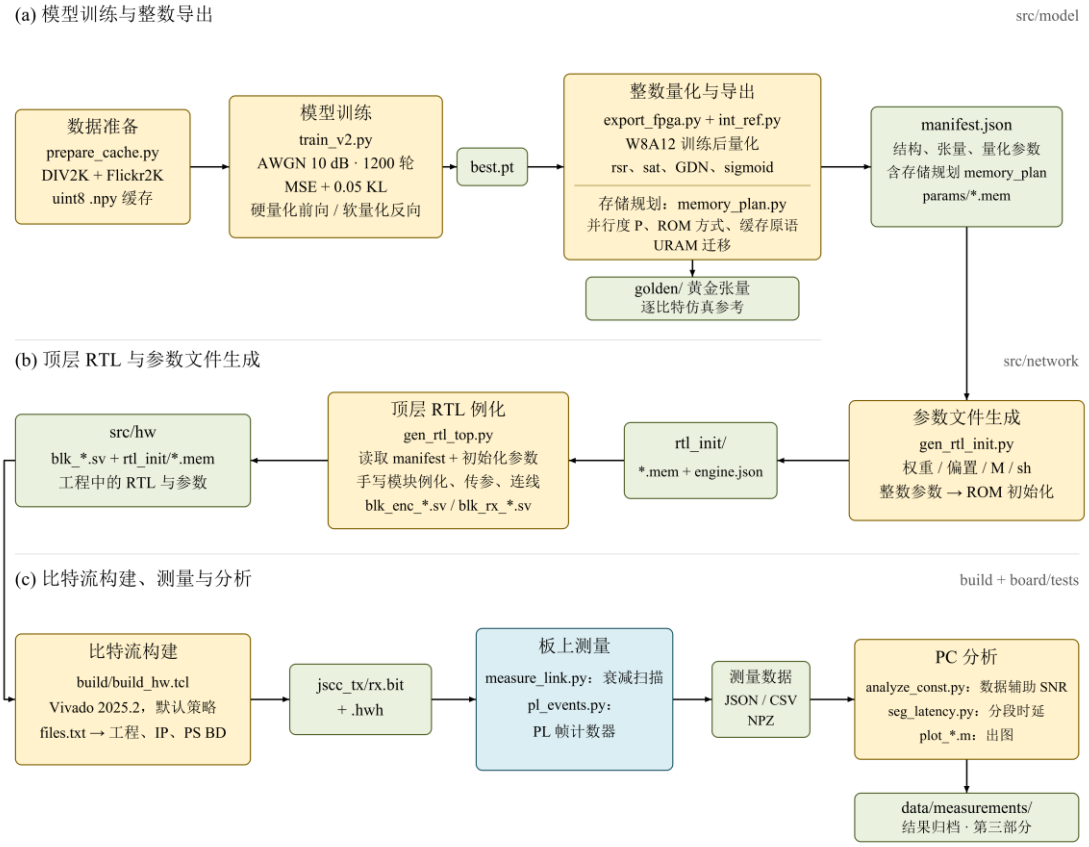


图 2-31　PC 端工具链：模型训练 → 整数量化 → RTL 例化 → 比特流 → 测量。

模型与训练：

- 模型配置：编码器按 DeepJSCC-Q 期刊版本[2]实现，主通道数 C＝32，潜变量通道数 16，总下采样倍率 4。256×256 RGB 图像编码为 64×64×16 的潜变量，相邻通道配为 I/Q，每帧 32,768 个 64QAM 符号，即每个像素 0.5 个复符号。解码器为浅层结构（2.2.3.1 节）。
- 训练目标：在 10 dB AWGN 信道下训练，损失为重构均方误差加权重 0.05 的星座分布 KL 正则，

$$\mathcal{L} = \mathrm{MSE}(\mathbf{x},\mathbf{x}) + 0.05\, D_{\mathrm{KL}}(p \,\|\, u)$$

其中 p 为批次内软分配的平均星座概率，u 为均匀分布。

- 量化方式：前向用最近星座点的硬量化，反向通过距离 softmax 的软量化传递梯度（`hard + soft - soft.detach()`）。训练前向与部署时的符号集合一致，编码器又能得到可用的梯度；KL 项防止符号使用概率过度集中。
- 训练数据：DIV2K[13] train 800 张 ＋Flickr2K 2650 张；优化器 Adam，有效批量 16。
- 训练过程分两段。第一段按论文结构训练完整的编码器与对称解码器（学习率 2×10^{-4}，1200 轮），得到编码器。第二段训练浅层解码器：先冻结

编码器、以无噪声输入训练解码器 300 轮；再在 10 dB AWGN 下联合训练编码器与解码器（解码器学习率 3×10^{-4}、编码器 1×10^{-4}，验证集 PSNR 30 轮不提高时减半，KL 权重不变）共 1400 轮；最后 200 轮做 8 位权重的量化感知训练：前向时卷积权重按部署规则逐输出通道量化为 8 位，梯度加到浮点权重上。部署检查点为量化感知训练的第 172 轮（3.2.3 节）。

整数量化与导出：量化脚本对检查点做 W8A12 量化，卷积权重为逐输出通道缩放的 8 位整数，激活为逐张量缩放的 12 位整数，PWL 输入的定标取 2 的幂次；整数参考模型规定偏置、舍入、饱和与非线性的整数运算，是 RTL 的逐比特参考。导出的网络描述文件包含神经网络结构、张量形状、量化参数和存储规划，顶层生成脚本据此例化手写的 RTL 模块、填写参数并完成顶层连线，另一个脚本把权重与重定标参数写成 ROM 初始化文件（2.2.3.1 节）。量化损失在两个数据集上都不超过 0.03 dB（表 2-18）。

表 2-18　软件模型的量化精度（10 dB AWGN，同一噪声样本）

评估集	浮点 PSNR / dB	W8A12 PSNR / dB	损失 / dB
DIV2K 验证集（256×256 中心裁剪）	31.13	31.13	0.002
Kodak[14]（原分辨率）	32.61	32.58	0.027

构建与测量：构建脚本按源文件清单新建 Vivado 工程、加入 IP、生成 PS 块设计，用 Vivado 2025.2 默认策略完成综合、实现和比特流生成；测量脚本完成链路扫描、数据辅助 SNR 计算和基于 PL 帧计数器的分段时延测量，结果见第三部分。

第三部分 完成情况及性能参数

3.1 整体介绍

系统实物见图 3-1（正面）与图 3-2（斜 45°）：左侧为 RX 板（PYNQ-ZU + AD9361 子卡 + 1024×600 触摸屏），右侧为 TX 板（PYNQ-ZU + AD9361 子卡 + USB 摄像头），两块子卡各接一根胶棒天线（TX 子卡接发射口，RX 子卡接接收口）。两块板上电约 1 分钟后自动建立链路，触摸屏实时显示重建图像与物理层状态；测试在室内进行，信道 SNR 通过 TX 的 AD9361 衰减调节。

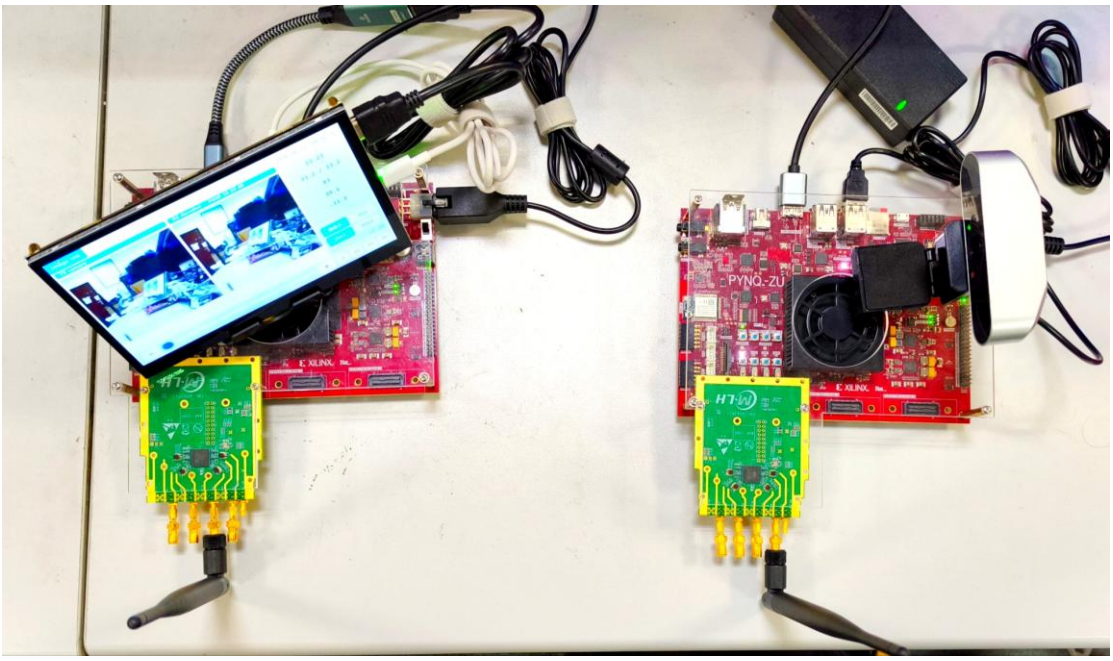


图 3-1　系统实物（正面）。

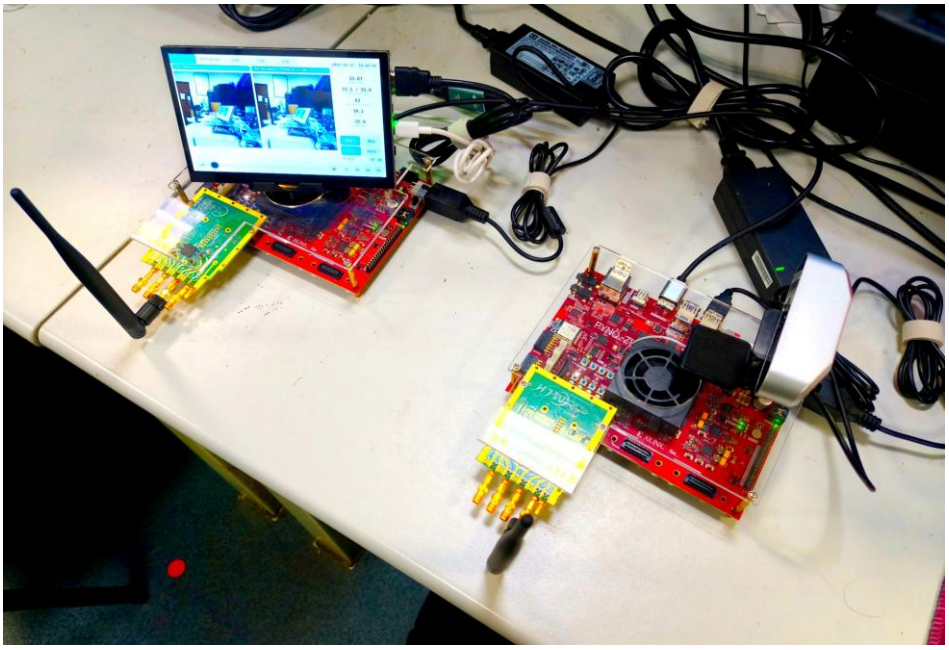


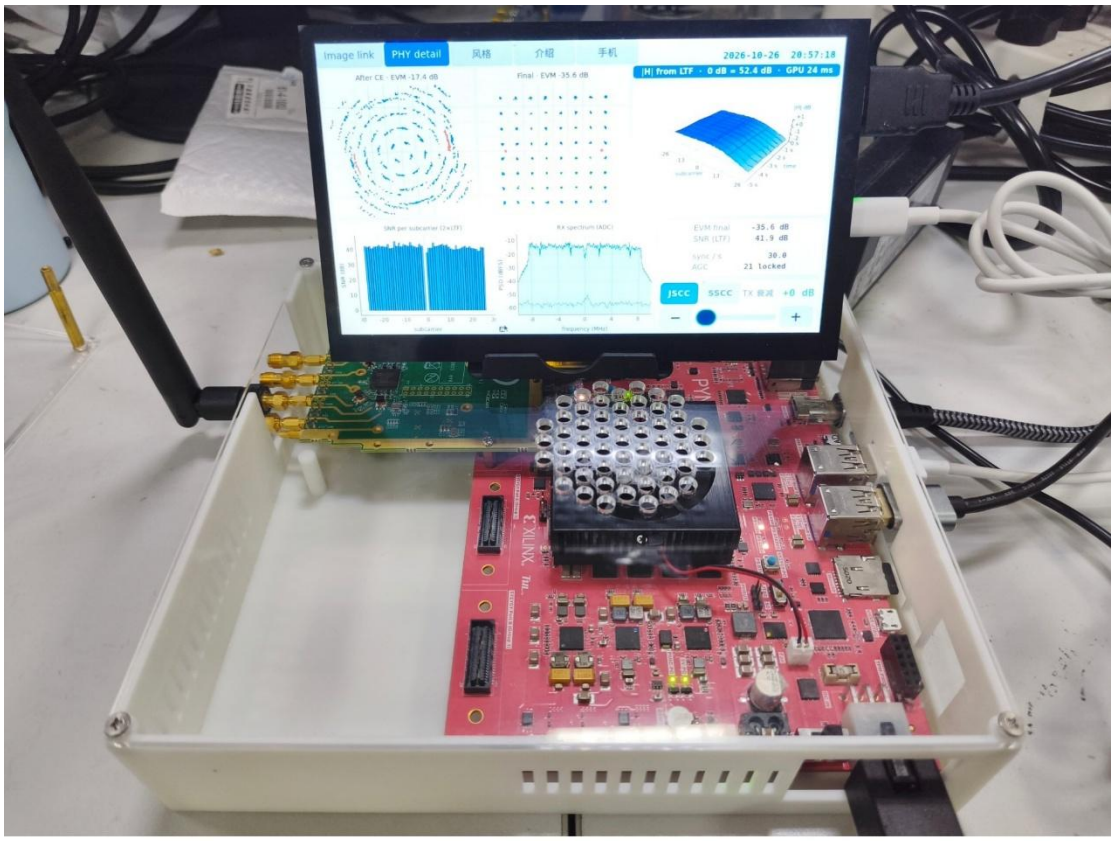
图 3-2　系统实物（斜 45°）。

3.2 工程成果

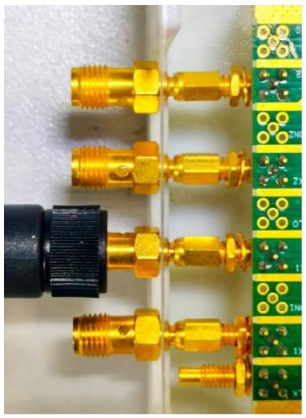
3.2.1 机械成果

两块板分别装入外壳（2.2.2 节），底壳为 3D 打印的光敏树脂件，顶盖为激光切割的 3 mm 透明亚克力。图 3-3 为装入外壳的 RX 板：子卡的 SMA 接口从前侧板的通槽伸出，天线直接接在子卡上（图 3-3(b)）；后侧板的开孔与 Pmod、

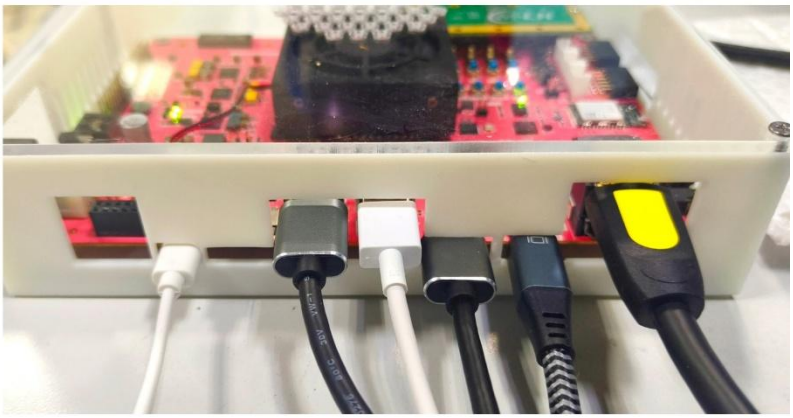
UART、USB、mini DP 和 HDMI 等接口对齐，插上线缆后插头不与侧板干涉（图 3-3(c)）；顶盖的进风孔位于风扇正上方，左、右、前三面的散热缝作为出风口。图 3-3(a)的物理层页为装好外壳后的链路状态：最终 EVM 为−35.6 dB，每秒同步 30 帧。



(a) 整体



(b) 前侧板的SMA通槽



(c) 后侧板的接口开孔

图 3-3　装入外壳的 RX 板：(a) 整体（触摸屏放在顶盖上，显示物理层页）；(b) 前侧板的 SMA 通槽；(c) 后侧板的接口开孔。

3.2.2 电路成果

两块板分别加载 TX、RX 比特流，均用 Vivado 2025.2 的默认策略综合与实

现，时序全部收敛（表 3-1）。

表 3-1　完整工程的资源、时序与功耗（xczu5eg-sfvc784-2-e，布局布线后）

工程	LUT	FF	BRAM36	URAM	DSP	WNS / WHS (ns)	功耗（实测 / 估算）
TX	54,241（46.3 %）	58,470（25.0 %）	104.5（72.6 %）	10（15.6 %）	243（19.5 %）	+0.082 / +0.010	3.07 W / 4.48 W
RX	63,699（54.4 %）	57,600（24.6 %）	65.5（45.5 %）	7（10.9 %）	133（10.7 %）	+0.307 / +0.010	3.44 W / 4.26 W

功耗的估算值是 Vivado 在布局布线后按默认翻转率给出的片上功耗，其中 PS 各占 2.733 W。实测值由板上三片 IRPS5401 稳压器的 PMBus 遥测读出（每 0.5 s 采样一次，取 60 s 平均），为 PL 内核（VCCINT）、PL 与 PS 的 1.8 V 辅助电源、PS 内核和 DDR 四类电源轨之和；DDR 电源轨包含 DDR4 颗粒，射频子卡由 FMC 的 12 V 供电，不在其中，也不含 12 V 输入的转换损耗。各工作状态的实测值见表 3-2。

表 3-2　各工作状态的实测功耗（W，DeepJSCC-Q 与 SSCC 均为 30 fps）

板	工作状态	PL 内核	PS 内核	辅助电源	DDR	合计
TX	DeepJSCC-Q	0.53	1.33	0.73	0.49	3.07
TX	SSCC	0.21	1.35	0.73	0.50	2.80
TX	比特流已加载、服务停止	0.22	1.24	0.72	0.45	2.64
RX	DeepJSCC-Q	0.40	1.55	0.81	0.68	3.44
RX	SSCC	0.40	1.58	0.81	0.68	3.47
RX	比特流已加载、服务停止	0.32	1.31	0.81	0.47	2.91

TX 在 DeepJSCC-Q 模式下的 PL 内核功耗比 SSCC 模式高 0.32 W，即编码器在 30 fps 下的功耗，每帧约 11 mJ。RX 的物理层输出同时送入神经网络解码器和 SSCC 的 LDPC 译码器，两者始终都在运行，两种模式的 PL 内核功耗相同。PL 内核的实测值约为 Vivado 估算的一半以下（TX 0.53 W / 1.21 W，RX 0.40 W / 0.97 W），因为估算采用默认翻转率，实际电路的平均翻转率更低。

资源按模块划分：

- 神经网络核（OOC 综合）：编码器 46,054 LUT、233 DSP、100 BRAM36、4 URAM；浅层解码器 16,351 LUT、51 DSP、43 BRAM36、不用 URAM（与论文同构的对称解码器为 50,178 LUT、312 DSP、100 BRAM36、18 URAM）；
- OFDM 物理层：TX 约 2.9 k LUT、3 BRAM36、6 URAM；RX 约 11.0 k LUT、12 BRAM36、6 URAM、82 DSP；
- SSCC 基线（OOC 综合）：LDPC 编码器 3.8 k LUT，LDPC 译码器 27.4 k LUT（其中 3.6 k LUTRAM），均不用 BRAM 和 DSP；
- 其余为 AD9361 接口、PS 接口与遥测。

神经网络在 250 MHz 下运行，每帧卷积乘累加量为 1.154 GMAC（按 3.3.5 节的方法计数），编码器、解码器分别只用 233 和 51 个 DSP，占器件的 19 %和 4 %。

3.2.3 软件成果

RX 板的触摸屏 GUI 有图像链路页（图 3-4）和物理层页（图 3-5）。图像链路页并排显示 TX 原图与 RX 重建图，右侧给出实时 PSNR、端到端时延、接收帧率和 EVM，下方是 TX 衰减调节条，并可切换信源（摄像头 / 预设图 / 预设视频）和传输方案（JSCC / SSCC）；图 3-4 为 TX 额外衰减 20 dB 时的屏幕截图。物理层页实时显示信道估计后与最终均衡输出的星座图、信道 |H| 三维曲面（Mali-400 渲染）、逐子载波 SNR、接收频谱和链路状态。

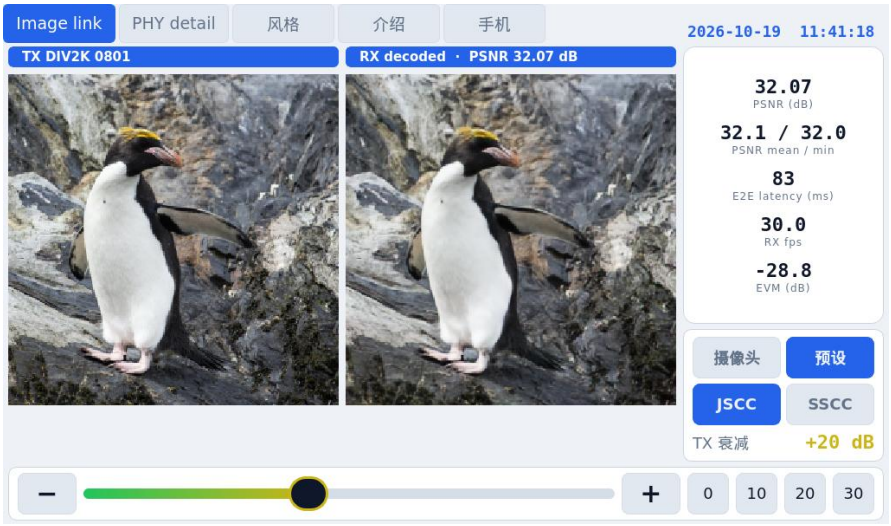


图 3-4　RX 板触摸屏 GUI（图像链路页，DIV2K 0801 预设图，TX 额外衰减 20 dB）。

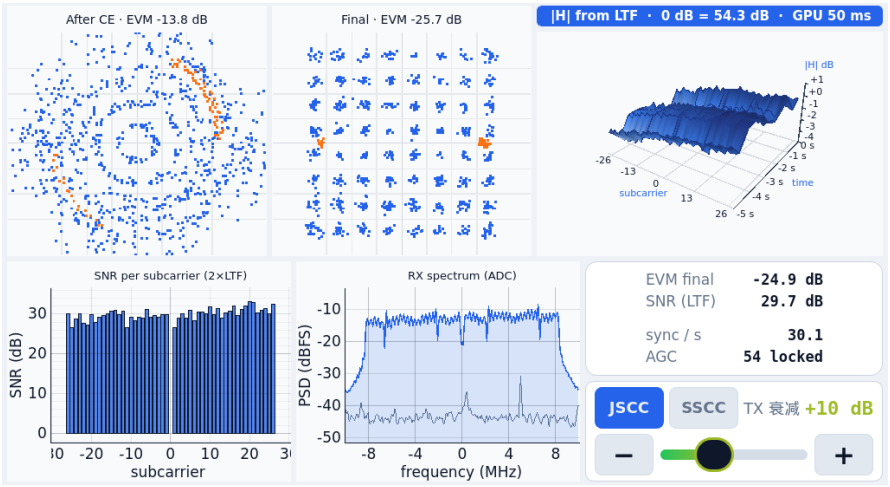


图 3-5　RX 板触摸屏 GUI（物理层页）。

PC 端训练程序的结果见图 3-6 与图 3-7。第一段按论文结构训练编码器与对

称解码器（图 3-6）：在 10 dB AWGN 信道下训练 1200 轮，训练集与验证集（DIV2K 验证集 100 张，256×256 中心裁剪）的 MSE 同步下降，没有过拟合；验证集指标连续 20 轮不提升时学习率降为原来的 0.8 倍，从 2×10^{-4} 逐步降到 1.7×10^{-5}。验证集 PSNR 在第 1186 轮达到最高的 31.28 dB（Kodak 32.65 dB），这一检查点的编码器作为第二段的起点。

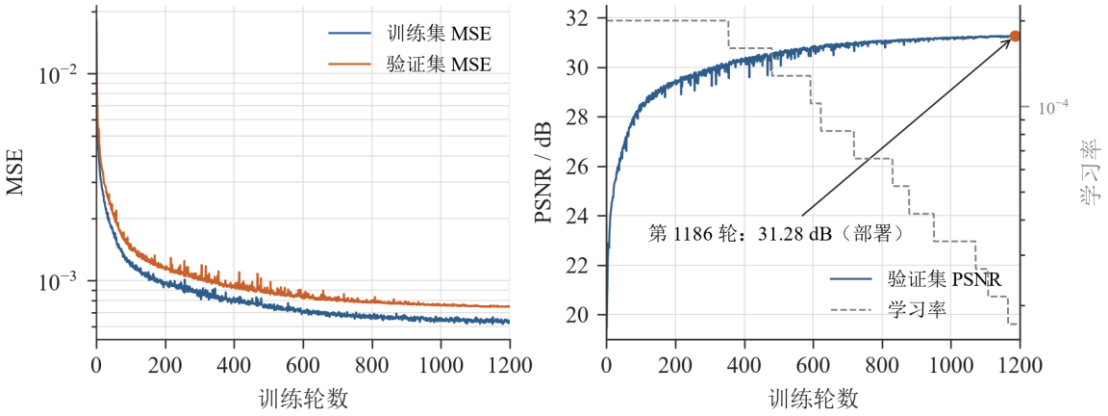


图 3-6　第一段训练曲线（编码器与对称解码器，10 dB AWGN）：左为训练集与验证集 MSE，右为验证集 PSNR 与学习率。

第二段训练浅层解码器（图 3-7）：冻结编码器、无噪声输入训练 300 轮后，编码器与浅层解码器在 10 dB AWGN 下联合训练，验证集 PSNR 在 1400 轮内从 30.1 dB 升到 31.19 dB；最后的量化感知训练使 PSNR 略降，第 172 轮的 31.15 dB（Kodak 32.61 dB）为部署检查点，比对称解码器低 0.12 dB，W8A12 整数模型低 0.09 dB。

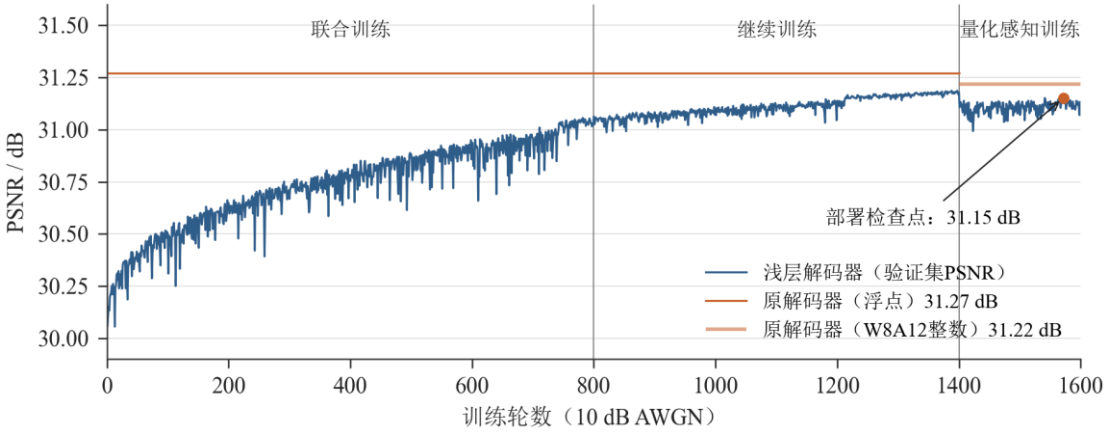


图 3-7　浅层解码器的联合训练与量化感知训练（10 dB AWGN，DIV2K 验证集 PSNR）。

3.3 特性成果

3.3.1 功能验证

表 3-3　功能验证结果

对象	方法	结果
编码器、解码器 RTL	与整数参考模型逐拍比较数据与边界标记，输入 valid 概率 90 %、输出 ready 概率 80 %	各 2 帧逐比特一致；解码器帧周期 6,494,259 个时钟周期（26.0 ms）
神经网络量化	W8A12 与浮点模型对比（10 dB AWGN，表 2-18）	PSNR 损失 0.002 dB（DIV2K）/ 0.027 dB（Kodak）
OFDM 物理层 RTL	RTL 发射机输出 + ±40 ppm SFO、20 kHz CFO、多径、30 dB SNR	683 个符号的长帧内 SFO 跟踪与理论漂移一致
同步 RTL	帧检测与定时同步单独仿真：随机半采样定时偏移、±36.6 kHz 随机频偏、AWGN	4 dB 下 60 帧：59 帧定时正确，1 帧帧检测漏检，没有落到 LTF2 的帧，频偏估计均方根误差 12.5 kHz；20 dB 下 20 帧全部正确，均方根误差 1.1 kHz；板上实采的 ADC 采样回放（TX 额外衰减 44～48 dB，共 36 帧）：全部定在 LTF1，5 万个只含噪声的采样中没有误触发
SSCC 编译码 RTL	TX → AWGN → RX 环回，与逐位参考模型比较	无噪声时 0 字节错误；15.5 dB、14.5 dB 下各 202 个码字的译出比特、收敛标志、迭代次数全部一致
OFDM 物理层（板上）	PRBS 64QAM 源代替编码器，板上满速连续运行	7.15×10^{8} 个未编码比特 0 误码；EVM 帧头 −36.6 dB、帧尾 −34.9 dB
整机	摄像头 / 预设视频 / 预设图经空口传输，GUI 实时显示	30 fps 稳定运行（3.3.3 节）

帧尾与帧头的 EVM 只差 1.7 dB，说明 SFO / CPE 跟踪在 683 个 OFDM 符号的长帧内有效。

3.3.2 画质与 SNR

测试条件：

- TX 播放 DIV2K 0802 预设图（256×256），DeepJSCC-Q 与 SSCC 使用同一物理层、同一星座电平与 TX 衰减设置，每幅图同样 32,768 个符号；
- 从最大衰减往小扫描，每点稳定 3 s 后测 8 s；

- 横轴为数据辅助 SNR：静止图像下每帧发送的符号相同，以最强点所有快照平均后的判决作为参考星座，$\mathrm{SNR} = 10\log_{10} \overline{|\mathrm{ref}|^2} / \overline{|\mathrm{rx} - \mathrm{ref}|^2}$；
- SSCC 帧 CRC 错误但 JPEG 仍可解码时按实际重建图计分，无法解码时按中灰图计分；
- 数据辅助 SNR 低于约 3 dB（TX 额外衰减 47 dB 以上）时帧同步开始漏检，超过 5 %的快照来自错误同步、数据辅助 SNR 不可信，这些点不计入曲线；DeepJSCC-Q 接收帧率低于 25 fps 的点也不计入。

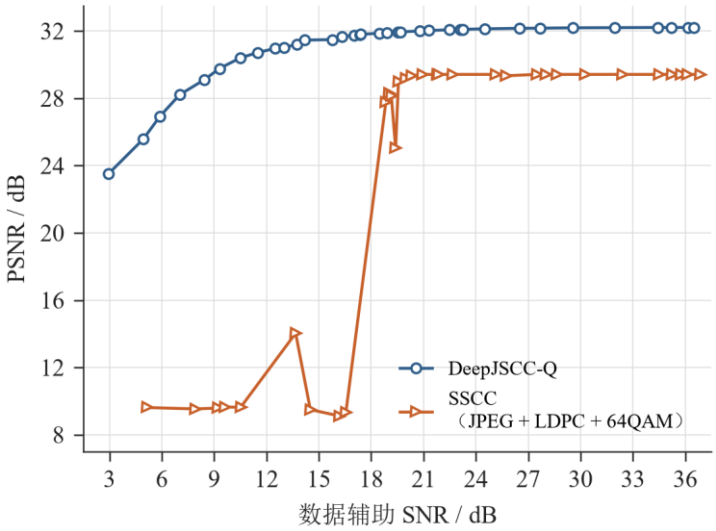


图 3-8　PSNR – SNR 实测曲线（DIV2K 0802）。

表 3-4　DeepJSCC-Q 的实测 PSNR

TX 额外衰减 / dB	**0**	**32.5**	**34**	**37**	**40**	**43**	**45**	**46**
数据辅助 SNR / dB	36.1	17.4	15.8	13.0	10.5	7.0	4.9	2.9
PSNR / dB	32.21	31.79	31.49	31.01	30.40	28.22	25.58	23.51

表 3-5　SSCC 基线的实测 PSNR 与 CRC 正确率

TX 额外衰减 / dB	**0**	**31**	**32**	**33**	**33.5**	**34**	**35**	**36**
数据辅助 SNR / dB	36.1	20.3	19.6	19.0	18.8	19.4	13.7	16.2
PSNR / dB	29.42	29.34	28.99	28.30	27.78	25.04	14.04	9.12
CRC 正确率	1.00	1.00	0.97	0.93	0.91	0.80	0.16	0.00

- 在相同的每图 32,768 个符号下，DeepJSCC-Q 在高 SNR 时 PSNR 比 SSCC 高 2.8 dB（32.21 dB 对 29.42 dB）；
- DeepJSCC-Q 从 36 dB 到 2.9 dB 平缓退化，PSNR 共下降 8.7 dB，SNR 低于 15 dB 后仍持续输出可辨认的图像，2.9 dB 时 PSNR 仍有 23.5 dB；
- SSCC 在 SNR 约 19.6 → 16.2 dB 的区间内，PSNR 从 29.0 dB 跌到 9.1 dB（CRC 正确率 0.97 → 0），再往下重建图严重损坏或无法解码，PSNR 只有 9 – 10 dB。悬崖附近的数据辅助 SNR 只来自少数快照，散布较大，表 3-5 按 TX 衰减排列；
- AWGN 仿真中 LDPC 2/3 在 15 dB 达到 99 %的整帧正确率（图 2-21），

板上 CRC 正确率 0.97 对应约 19.6 dB 的数据辅助 SNR。数据辅助 SNR 是 48 个数据子载波的平均，频率选择性衰落下弱子载波上的误码决定了码字能否纠正，因此悬崖比 AWGN 晚约 4.5 dB。

相同 TX 衰减下两种方案的代表帧见图 3-9，DeepJSCC-Q 在三个 SNR 下解码器收到的软符号见图 3-10。低 SNR 时星座点已无法分辨，解码器仍能从未经判决的 I/Q 重建出可辨认的图像。

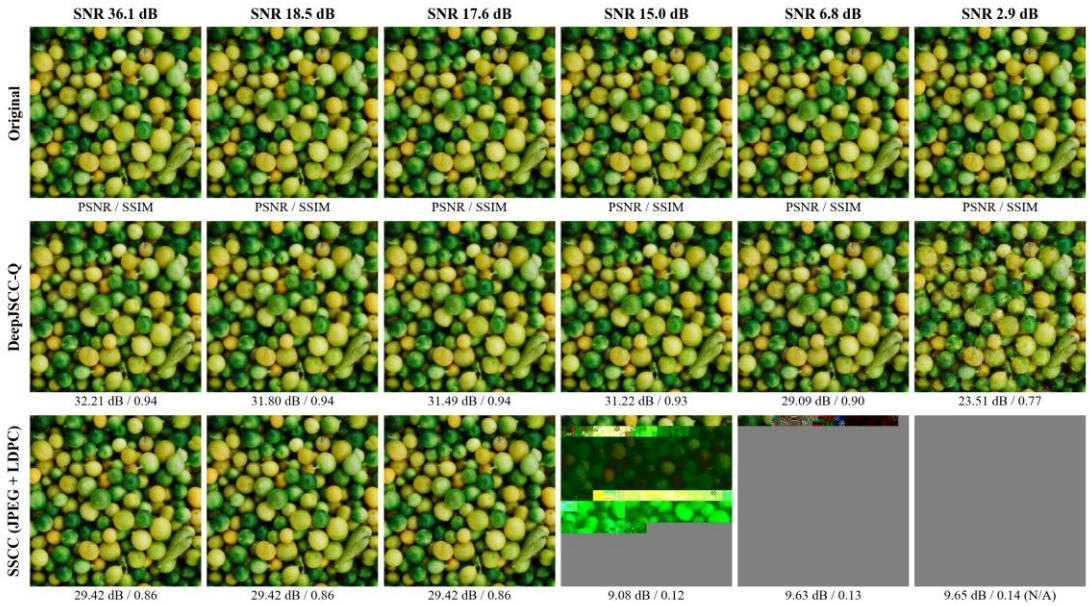


图 3-9　相同 TX 衰减下的代表解码图像（上：原图；中：DeepJSCC-Q；下：SSCC），列标题为两种方案实测 SNR 的均值，图下为单帧 PSNR / SSIM，N/A 表示无法解码、按中灰图计分。

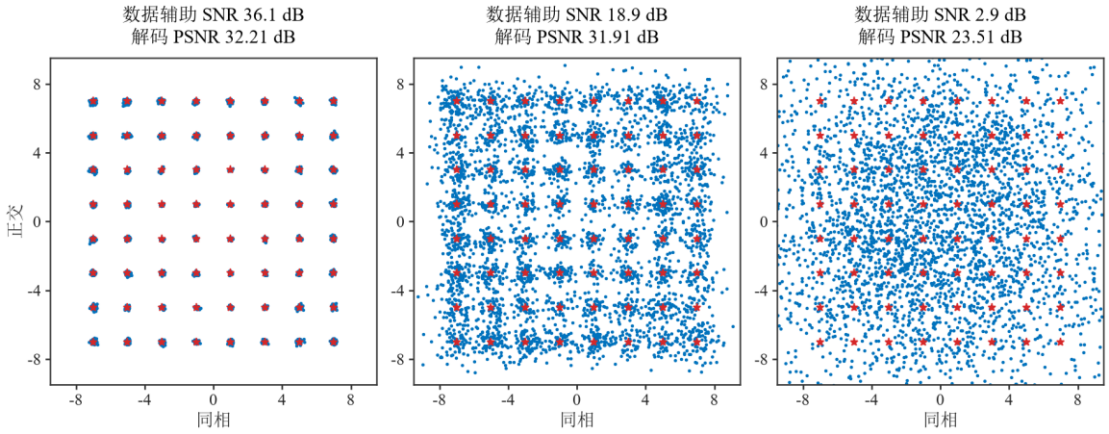


图 3-10　DeepJSCC-Q 解码器输入端的接收星座（均衡输出，48 个数据子载波）。

3.3.3 帧率与时延

帧率：DeepJSCC-Q 播放预设视频连续运行 10 min，RX 的解码完成帧计数器在 600 s 内增加 18,020 帧，平均 30.0 fps，每 5 s 的帧率在 29.8~30.2 fps 之间（图 3-11），帧缓存没有整帧丢弃。RX 服务以同样的速率把图像转发给 GUI。

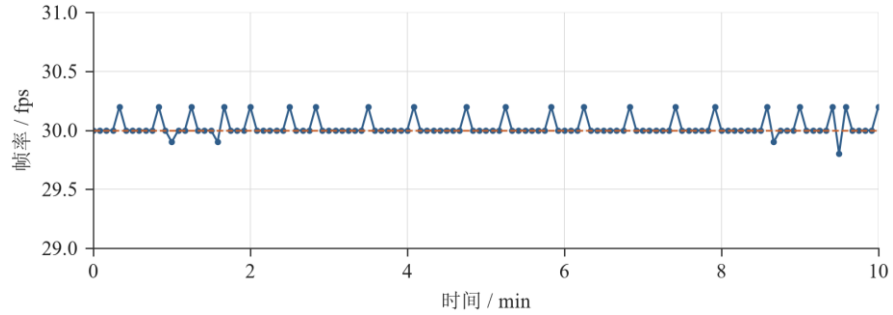


图 3-11　连续 10 min 的接收帧率（DeepJSCC-Q，预设视频；PL 解码完成帧计数器，每 5 s 一点）。

分段时延：两块板的 PS 高频轮询 PL 中按数据流位置分布的帧计数器（TX：编码器输入字节、编码器输出帧、发射帧；RX：同步、PHY 输出帧、解码完成帧），按帧配对，两板时钟偏差用往返时间估计并线性插值校正（往返时间中位数 0.70 ms）。按文献[3]的三段划分，结果见表 3-6 与图 3-12。

表 3-6　分段时延（PL 计数器）

段	**起点 → 终点**	**中位数**	**1 % / 99 % 分位**
编码	第一个图像字节进入编码器 → 最后一个符号输出	38.20 ms	38.14 / 38.25 ms
传输	最后一个编码符号 → RX 最后一个 PHY 符号输出（帧缓存、空口、同步、均衡）	2.90 ms	2.82 / 3.93 ms
解码	RX 最后一个 PHY 符号 → 最后一个解码字节	26.39 ms	25.54 / 27.68 ms
PL 合计		67.49 ms	67.28 / 69.23 ms

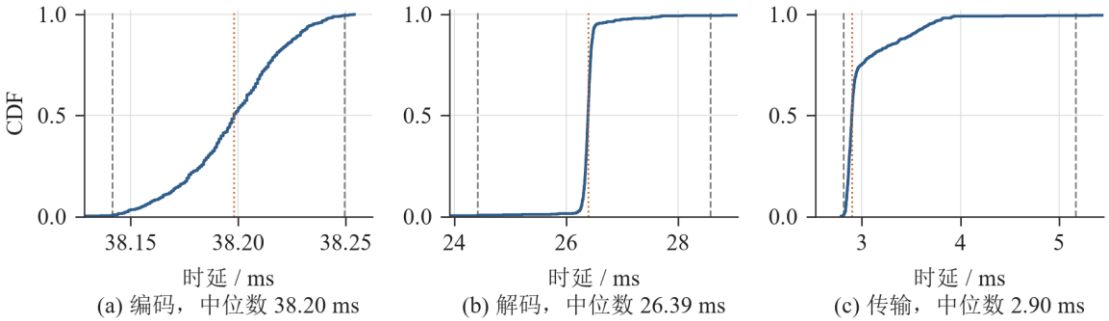


图 3-12　分段时延的累积分布（编码段 666 个样本，其余各段 665 个）。传输段的下限 2.748 ms 即一帧的空中时间。

编码段 38.2 ms 长于帧间隔 33.3 ms，是因为编码器是逐层流水的：下一帧的输入在上一帧尚未输出完时就已开始，吞吐由最慢的一层决定（30 fps），单帧时延则是各层流水填充时间之和。解码段比对称解码器（30.39 ms）缩短 4.0 ms，

与 RTL 仿真的帧周期（26.0 ms）一致。从 TX 应用取得源帧到 RX 的 PS 收到完整重建图（不含屏幕显示），端到端时延中位数为 77.20 ms（555 个样本）。

3.3.4 射频前端相对开源基线的改进

AD9361 的接口与初始化部分来自开源工程与 ADI 配置工具（2.2.3 节），本作品在其基础上做了三项修改：修正 SPI 握手、在 PL 中实现 TX 正交校准相位扫描、重新设计可编程 FIR。表 3-7 为单板环回的测量结果（物理层以 16QAM 传输随机比特）。

表 3-7　单板环回 EVM 与镜像抑制

配置	EVM	镜像（I/Q 失衡）
开源工程 + 配置工具默认设置	−22.0 dB	−25.7 dBc
修正 SPI 握手	−29.7 dB	−36.5 dBc
+ TX 正交校准相位扫描	−31.2 dB	−38.7 dBc
+ 可编程 FIR 重新设计（最终版本，另含数字功率回退）	−32.0 ~ −32.2 dB	−38 dBc

FIR 重新设计后，冲激响应在 ±8 个采样之外的能量从 −24.7 dB 降到 −44.8 dB（配置工具默认的 128 抽头 FIR 振铃超出 CP）。快速 AGC 的寄存器取值参考 openwifi[12]，其中锁定电平改为 −14 dBFS：SSCC 波形的 PAPR 尾部比 DeepJSCC-Q 高约 0.8 dB，在 openwifi 的 −12 dBFS 锁定电平下会触发 ADC 大过载，使已锁定的快速 AGC 在帧内改变增益（表 3-8）。

表 3-8　AGC 锁定电平对 SSCC 的影响

指标	−12 dBFS（openwifi 取值）	−14 dBFS（本作品）
SSCC 帧内增益跳变的帧	9 / 45	0 / 44
数据辅助 SNR（各衰减点）	—	提高 0.1 ~ 1.9 dB

PN 扰码使 OFDM 的 PAPR（99 %分位）从 10.1~10.7 dB 降到 9.4~9.8 dB，与随机 64QAM 相当；三张测试图的 TX 时域峰值最大为 1765（满量程 2047），不削顶。

3.3.5 与 DeepJSCC-Q 原网络的对比

DeepJSCC-Q 论文[2]的网络（Fig. 3）主通道数为 256，编码器有 4 个步长为 2 的残差块，总下采样 16 倍。本作品为了在单片 xczu5eg 上以 30 fps 运行，在保持潜变量元素数（即每幅图的信道符号数）不变的前提下修改了网络规模，修改及理由见表 3-9。

表 3-9　部署网络相对 DeepJSCC-Q 原网络的修改

项	原网络[2]	本作品	理由
主通道数 C / 输出通道数 Cout	256 / 256	32 / 16	权重与乘加数随通道数平方增长，原网络的权重放不进片上存储（见下文）
下采样倍率	16（4 个步长 2 的残差块）	4（前 2 个步长 2，后 2 个步长 1）	通道数减少后，增大潜变量的空间尺寸，保持潜变量元素数不变
解码器	与编码器对称，10 个块共 59 个卷积	非对称浅层解码器：线性 3×3 卷积、4 个 PWL 残差块、2 个 3×3 亚像素上采样，共 12 个卷积	乘加量降为对称结构的 1/6，与编码器联合训练（2.2.3.1 节）
潜变量	16×16×256	64×64×16	二者都是 65,536 个实数，即每幅图 32,768 个复信道符号
LeakyReLU 斜率	论文未注明	1/128	负半轴只需右移 7 位，不用乘法器
数值精度	浮点	W8A12，解码器做量化感知训练	片上存储与 DSP 位宽；量化损失不超过 0.03 dB（表 2-18）

编码器的残差块、GDN、注意力块与软硬量化训练方法均与原网络相同。按同一计数方法（每个卷积输出元素计 K • K • C_in 次乘加，含零填充位置），256×256 输入时两种网络的规模见表 3-10。

表 3-10　原网络与部署网络的规模（256×256 输入，计算值）

项	原网络（C = 256）	本作品（C = 32）	比值
编码器乘加 / 帧	47.70 GMAC	0.913 GMAC	1/52
解码器乘加 / 帧	70.48 GMAC	0.241 GMAC	1/292
编码器参数	10,833,408	141,584	1/77
解码器参数	23,599,224	53,332	1/442
编码器卷积权重（8 位存储）	10.6 MB	137 KB	—
解码器卷积权重（8 位存储）	23.4 MB	50 KB	—

xczu5eg 的片上存储共约 2.95 MB（144 块 BRAM36 与 64 块 URAM），原网络仅编码器的 8 位权重就有 10.6 MB，只能放在片外 DDR 中逐层读取；原网络编码器在 30 fps 下需要 1.43 TMAC/s，而全部 1248 个 DSP 在 250 MHz 下每秒只能完成 0.31 TMAC。缩小后，编码器、解码器的全部权重与激活缓存都放在片上，乘加需求降到原网络的约 1/102。

缩小网络使画质有所下降。论文 Fig. 12 给出了与本作品完全相同的工作点：

Kodak 测试集、64-QAM、λ＝0.05、训练与测试 SNR 均为 10 dB、每幅图 32,768 个复信道符号，原网络的 PSNR 约为 34.9 dB（逐图 PSNR 的平均）。本作品的浮点模型在相同条件下逐图平均 PSNR 为 33.24 dB（24 张图，每张 10 次噪声），低约 1.7 dB；其中解码器换成浅层结构只带来 0.03 dB 的差别（对称解码器为 33.27 dB）。这一差值还包含训练数据的差异：原网络用 ImageNet（约 120 万张）训练，本作品用 DIV2K 与 Flickr2K（3450 张）训练。

浅层解码器与对称解码器在不同 SNR 下的对比见图 3-13（W8A12 整数模型，同一组噪声）：15 dB 以上浅层解码器高 0.05~0.18 dB，4~12 dB 低 0.01~0.13 dB，2 dB 以下又高 0.05 dB 以上，两者的差在整个 0~30 dB 范围内不超过 0.2 dB。

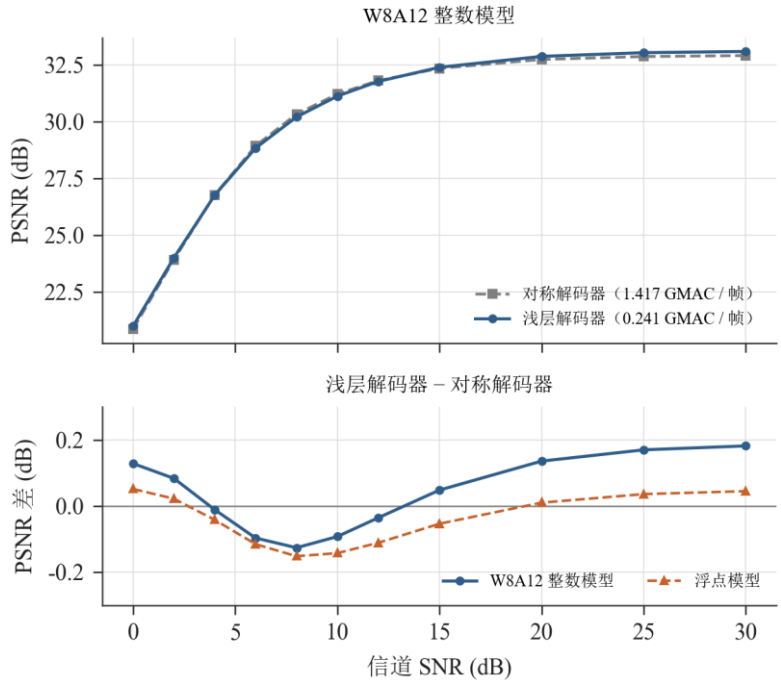


图 3-13　浅层解码器与对称解码器的 PSNR－SNR（DIV2K 验证集 100 张，AWGN，同一组噪声）。上：W8A12 整数模型；下：浅层解码器减对称解码器，实线为整数模型，虚线为浮点模型。

3.3.6 与 FPGA 实现的 DeepJSCC 对比

与本作品最接近的公开 FPGA 工作是 Isobe 等人在 GLOBECOM 2025 发表的 DeepJSCC 实时 5G 图像传输系统[3]，对比见表 3-11。

表 3-11　与文献[3]的对比

项	Isobe 等[3]	本作品
神经网络	5 层卷积 / 5 层转置卷积	编码器：残差、GDN、注意力，55 个卷积算子；解码器：PWL 残差块、IGDN、亚像素卷积，12 个卷积算子
每像素复符号数	0.25	0.5
每帧卷积乘累加量	0.222 GMAC/帧	1.154 GMAC/帧（5.2 倍）
数值精度 / 实现	INT8，HLS	W8A12，参数化 RTL
器件	ZCU111（XCZU28DR）	2 × xczu5eg
编码器资源	77,902 LUT / 1,597 DSP / 1,038.5 BRAM	46,054 LUT / 233 DSP / 100 BRAM36 + 4 URAM
解码器资源	28,179 LUT / 874 DSP / 1,011.5 BRAM	16,351 LUT / 51 DSP / 43 BRAM36
物理层	5G 系统（外部设备）	自行设计的 OFDM 物理层，与神经网络在同一 PL 中
帧率	约 33 fps	30 fps
时延	编码 39.6 / 解码 52.9 ms（含 PS 处理）	编码 38.2 / 解码 26.4 ms（PL 计数器）
功耗	编码器 7.757 W / 解码器 5.689 W	TX 3.07 W / RX 3.44 W（实测，含 PS、物理层与 DDR）

按 3.3.5 节的同一方法计数，本作品编码器、解码器的工作量分别是[3]的 8.23 倍和 2.17 倍，而编码器 DSP 只用[3]的 15 %、解码器用 5.8 %，BRAM 分别约为[3]的 1/10 和 1/24。[3]的时延含 PS 处理，本作品的分段时延由 PL 计数器测得。

第四部分 总结

4.1 可扩展之处

1. 信道自适应：目前模型只在 10 dB 下训练。可以针对多个 SNR 或不同的信道符号数训练模型并按 GUI 测得的 SNR 切换，或把 SNR 作为神经网络的输入，进一步提高高 SNR 时的画质。
2. 更高分辨率：神经网络是全卷积的，可以把大图分块传输，或在更大器件上增加行缓存宽度。

3. 移动与多天线：利用均衡后逐子载波的 SNR 做加权，或扩展到 AD9361 的 2×2 MIMO，提高衰落信道下的鲁棒性。
4. 双向链路：两块板都有完整收发通道，可以实现双向实时图像传输。

4.2 心得体会

这个作品让我们体会最深的是，“把神经网络放到 FPGA 上”和“让它在真实无线链路上工作”是两件难度相当的事。

神经网络部分，最初按论文的 256 通道结构估算，权重有三千多万个，片上存储差了一个数量级。我们没有选择把权重放在 DDR 里，而是保持潜变量数不变、减少通道和下采样，让整网权重和缓存都留在片上，再按层选择最小并行度。这一步的关键是在写 RTL 之前就逐个实例计算 BRAM：不同层的宽深比、端口数都不同，按总容量估算会严重低估实际用量。整数参考模型从一开始就规定了每一步的舍入和饱和，RTL 逐个模块与它逐比特比较，问题能在仿真阶段定位到具体模块。

解码器的精简是在系统跑通之后做的。按论文与编码器对称实现的解码器是整个系统中最大的模块，我们参考 Yang 与 Mandt 的非对称思路，但直接照搬以线性上采样为主的浅层结构，10 dB 下比原解码器低 1.6 dB。用直线插值逐级检查后发现，DeepJSCC-Q 编码器的非线性比图像压缩的编码器强得多，反解只需要在潜变量分辨率上完成，上采样部分几乎是线性的；于是把 PWL 残差块放在 64×64 上、上采样只用 3×3 亚像素卷积，再与编码器联合训练，解码器的计算量和 DSP 都降到原来的 1/6，画质基本不变。每个部件是否保留都由消融实验决定。

物理层部分，DeepJSCC 的接收端不做判决，帧尾的相位旋转、定时漂移会直接进入解码器，所以对同步和跟踪的要求比先判决再纠错的比特传输更高。683 个 OFDM 符号的长帧让我们必须按晶振标称精度的最坏情况设计 SFO 跟踪，而不能只看两块板实测的频偏。射频前端的问题往往不在算法里：SPI 握手错位使校准提前结束，快速 AGC 在信号变弱时停在低增益，SSCC 的峰值触发 ADC 过载。这些问题都是靠抓取原始 ADC 数据、读取寄存器状态、一次只改一个参数逐步定位的。

测量口径也很重要。最初用 PS 的时间戳测时延，结果混入了 Linux 调度的抖动；改为在 PL 中放置帧计数器、按数据流位置分段统计后，各段时延的分布都非常集中。

整个项目是参赛者与两个 AI 编程代理（Claude Code 与 Codex）协作完成的。AI 代理承担了大量代码编写、调试脚本、数据分析和文档工作；参赛者负责提出目标与约束、做关键技术判断、决定取舍，并完成接线、上电、观察等物理操

作。协作中 AI 反复出现的问题是：没有证据就下结论、把自己定的经验值当成规格、没有问清约束就增加设计、只读了部分资料就引用。针对这些问题，我们约定了“改动要小而有判据、一次只改一处、先定验证标准再动手、不做不必要的 RTL 修改”等规则，并写进代理的记忆和交接文档。协作记录与从中提炼的技能见附录 A、附录 B。

第五部分 参考文献

[1] E. Bourtsoulatze, D. B. Kurka, D. Gündüz. Deep joint source-channel coding for wireless image transmission[J]. IEEE Transactions on Cognitive Communications and Networking, 2019, 5(3): 567-579. DOI: 10.1109/TCCN.2019.2919300.

[2] T.-Y. Tung, D. B. Kurka, M. Jankowski, D. Gündüz. DeepJSCC-Q: Constellation constrained deep joint source-channel coding[J]. IEEE Journal on Selected Areas in Information Theory, 2022, 3(4): 720-731. DOI: 10.1109/JSAIT.2022.3231042.

[3] T. Isobe, et al. FPGA-based deep joint source-channel coding for real-time 5G image transmission[C]// GLOBECOM 2025 - 2025 IEEE Global Communications Conference. Taipei, 2025: 1232-1237. DOI: 10.1109/GLOBECOM59602.2025.11432434.

[4] J. Ballé, V. Laparra, E. P. Simoncelli. Density modeling of images using a generalized normalization transformation[C]// International Conference on Learning Representations (ICLR). 2016.

[5] Z. Cheng, H. Sun, M. Takeuchi, J. Katto. Learned image compression with discretized Gaussian mixture likelihoods and attention modules[C]// IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2020: 7939-7948. DOI: 10.1109/CVPR42600.2020.00796.

[6] W. Shi, J. Caballero, F. Huszár, et al. Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network[C]// IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016: 1874-1883.

[7] IEEE Std 802.11-2020. IEEE Standard for Information Technology—Telecommunications and Information Exchange between Systems—Local and Metropolitan Area Networks—Specific Requirements—Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications[S]. New York: IEEE, 2021.

[8] T. M. Schmidl, D. C. Cox. Robust frequency and timing synchronization for

OFDM[J]. IEEE Transactions on Communications, 1997, 45(12): 1613-1621. DOI: 10.1109/26.650240.

[9] M. Speth, S. A. Fechtel, G. Fock, H. Meyr. Optimum receiver design for wireless broad-band systems using OFDM—Part I[J]. IEEE Transactions on Communications, 1999, 47(11): 1668-1677.

[10] Analog Devices. AD9361 reference manual: UG-570 Rev. A[Z/OL]. (2015-06)[2026-10-11]. https://www.analog.com/media/en/technical-documentation/user-guides/ad9361.pdf.

[11] Analog Devices. no-OS: software drivers for systems without an operating system[CP/OL]. (2026-10-09)[2026-10-11]. https://github.com/analogdevicesinc/no-OS.

[12] X. Jiao, W. Liu, M. Mehari, M. Aslam, I. Moerman. openwifi: a free and open-source IEEE802.11 SDR implementation on SoC[C]// 2020 IEEE 91st Vehicular Technology Conference (VTC2020-Spring). Antwerp, Belgium, 2020: 1-2. DOI: 10.1109/VTC2020-Spring48590.2020.9128614.

[13] E. Agustsson, R. Timofte. NTIRE 2017 challenge on single image super-resolution: Dataset and study[C]// IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW). 2017: 126-135.

[14] Eastman Kodak. Kodak lossless true color image suite[DB/OL]. [2026-10-11]. http://r0k.us/graphics/kodak/.

[15] G. K. Wallace. The JPEG still picture compression standard[J]. IEEE Transactions on Consumer Electronics, 1992, 38(1): xviii-xxxiv.

[16] lzk2211. Zedboard_AD9361_radar[CP/OL]. (2025-01-11)[2026-10-11]. https://github.com/lzk2211/Zedboard_AD9361_radar.

[17] Analog Devices. HDL libraries and projects[CP/OL]. (2026-10-09)[2026-10-11]. https://github.com/analogdevicesinc/hdl.

[18] AMD. AMD Ross agentic AI assistant[CP/OL]. [2026-10-11]. https://github.com/xilinx/ross-ai-assistant.

[19] Y. Yang, S. Mandt. Computationally-efficient neural image compression with shallow decoders[C]// IEEE/CVF International Conference on Computer Vision (ICCV). Paris, 2023: 530-540. DOI: 10.1109/ICCV51070.2023.00055.

[20] F. Tosato, P. Bisaglia. Simplified soft-output demapper for binary interleaved COFDM with application to HIPERLAN/2[C]// IEEE International Conference on Communications (ICC). New York, 2002: 664-668.

附录

附录 A 大模型协作记录

本作品由参赛者与两个 AI 编程代理协作完成：Claude Code（Anthropic）与 Codex（OpenAI）。代理在本机与两块板卡上读写代码，运行 Vivado、仿真和板上程序，采集数据并出图；参赛者提出目标和约束、做关键技术判断、决定取舍，并完成接线、上电、观察 LED / 屏幕 / 天线等全部物理操作。完整记录见仓库 `report/llm_collab/`。

表 A-1　分工与时间线（2026 年）

时间	工作	代理
09-01~09-26	DeepJSCC 原型；DeepJSCC-Q 按期刊版重写、训练；类 802.11a OFDM 发射机	Codex
09-25~09-28	AD9361 配置调试；OFDM 自环到两板空口；AGC、定时门限、长帧 SFO 跟踪、遥测 GUI	Claude Code
09-28~09-29	1200 轮训练；定点方案；存储规划；行缓存与卷积引擎 RTL	Claude Code
09-29~09-30	神经网络顶层例化脚本、FIFO 定尺寸、逐比特验证、编 / 解码器工程	Codex
09-30~10-05	物理层与神经网络对接；PYNQ 上板；摄像头、GUI、Mali-400 GPU；SSCC 基线	Claude Code
10-05~10-07	演示视频（代理控制两块板卡、配音、剪辑）；AGC 看门狗；实测；开源仓库	Claude Code
10-07~10-11	设计报告：Codex 起草神经网络部分初稿并交叉审查；Claude Code 撰写、绘图，参赛者逐条批注修改	两者
10-15~10-19	浅层解码器：结构探索、联合训练与量化感知训练、PWL 的 RTL、逐比特仿真、消融实验、上板测试与报告修改	Claude Code
10-20~10-21	残差块个数扫描与对照实验；SSCC 基线改用 LDPC（AWGN 仿真、Vivado LDPC IP 的资源评估、编译码器 RTL 与逐位参考模型）；定时同步改为两点度量、频偏估计改为 64 点累加；重新上板测试	Claude Code

与本项目相关的会话共 40 个，用户消息 766 条。`report/llm_collab/cases.md`

整理了 17 个典型案例，每个案例写明背景、模型做了什么、用户做了什么和结果，代表性的几类见表 A-2。

表 A-2　协作案例（摘选）

类型	案例	结果
参赛者给出关键判断，AI 计算与实现	LTF 定时门限按半采样相关峰损失确定；SSCC 在悬崖区的增益跳变来自 PAPR 与 ADC 过载；低 SNR 定时偏移时参赛者提出累加 LTF 两段的四个相关点，AI 仿真发现四点和首次越限会提前 64 个采样	门限 0.375 用移位相加实现；AGC 锁定电平改为−14 dBFS；改为相邻两点之和，低 SNR 下落到 LTF2 与漏检的帧消失
参赛者追问依据，纠正 AI 的错误前提	并行度不必受 BRAM 位宽限制；对 openwifi 配置的引用不完整；残差块个数缺少依据	加入位宽转换后 DSP 与 BRAM 同时下降；补齐漏抄的 AGC 寄存器；补做 1~6 个残差块的扫描与对照实验
参赛者约束模块边界	AGC 属于射频模块，不依赖基带；帧缓存归物理层，神经网络不重复设置	AGC 不与基带耦合；神经网络两端改为带反压的 AXI-Stream
参赛者纠正测量口径	PS 调度抖动不应计入 FPGA 的时延与帧率	改为 PL 计数器分段时延
AI 发现问题	SPI 握手错位；量化器多乘了一个系数（由参赛者观察到的训练曲线异常引出）	校准恢复正常；重新训练
两个代理交叉审查	Codex 审查报告初稿，提出 21 条意见，其中包括 Claude Code 写下的过度结论和无出处数值	逐条处理，处理记录见 `report/llm_collab/claude_codex/`

AI 反复出现的问题有四类：没有证据就下结论；把自己定的经验值当成规格；没有问清约束就增加设计；只读了部分资料就引用。对应的协作约定是：改动要小而有判据，一次只改一处，先定验证标准再动手，不做不必要的 RTL 修改。这些约定写进了代理的记忆与交接文档，在后续工作中持续生效。会话全文的脱敏转写及生成脚本见 `report/llm_collab/transcripts/` 与 `report/llm_collab/tools/`。

附录 B 技能包

从开发过程中提炼了 5 个面向 PYNQ / Zynq UltraScale+用户的通用技能（仓库 `skill/`），不依赖本作品的硬件设计。格式参照 AMD 官方的 AMD Ross Agentic AI Assistant[18]中的技能：Agent Skills 格式（`SKILL.md`），正文是写给 AI 代理的操作手册，包括触发条件、适用与不适用场景、DO / DON’T 与取自真实失败的反例、带检查点的分步流程，以及暂停点：加载内核模块、安装或停止服务、重启、修改系统配置、只要求检查时修改用户代码，这些操作之前代理必须等待用户确认；用户已经明确要求、可撤销且不影响整个系统的操作则直接执行，事后说明（表 B-1）。只适用于本系统的经验（AD9361 配置、杂散排查、时延测量等）和踩坑清单另存于 `report/notes/project_skills/`。

表 B-1　技能包

技能	解决的问题	附带文件
`mali400-offscreen-gles`	ZU EG/EV 片上 Mali-400 GPU 在 PYNQ 下无驱动、X 中只有软件渲染：判断能否使用开源 lima 驱动，排除走不通的渲染路径，用 GBM + EGL 在 Python 中离屏渲染 GLES2	探测脚本、渲染模板
`pynq-out-of-tree-kernel-module`	镜像内核未开启的驱动在板上补编为外部模块；内核与板上 gcc 版本不一致时不破坏系统构建树	头文件与 modpost 生成脚本、编译器包装
`pynq-systemd-autostart`	程序上电自启：排在加载 base.bit 的 `bootpy.service` 之后、使用 PYNQ venv、等待 X 与外设、overlay 自检重载、SIGTERM 干净退出	service 模板、程序骨架
`pynq-local-display-gui`	在板载 DP / HDMI 屏和 USB 触摸屏上运行全屏 PyQt 界面：显示栈探测、在真实 X 服务器上用不显示的窗口截图并检测文字溢出与窗口超出可用区域、CPU 预算、触摸长按与拖动	探测、截图、触摸脚本
`pynq-dma-stale-transfer`	AXI DMA 接收在超时或进程被杀后遗留传输，导致下一次接收卡死；PYNQ 的 `stop()`无法撤销，改用 DMACR 软复位	—

每个技能都从本作品开发中的一次真实失败或观察提炼而来，适用边界写在各 `SKILL.md` 的不适用场景一节（表 B-2）。

表 B-2　技能的来源与失效条件

技能	从哪些失败或观察中总结	失效条件
`mali400-offscreen-gles`	RX 触摸屏 GUI 的信道三维曲面用 CPU 软件渲染太慢；改用 Mali-400 时，曾把属于 `zocl` 的渲染节点 `renderD128` 误认作 GPU，lima 发生 GP MMU 页错误后不报错、每帧只有清屏色	使用 Xilinx 专有 `libmali` 驱动的 PetaLinux；没有 Mali-400 的器件（Zynq-7000、ZU CG、Versal）；CPU 已足够快的二维界面
`pynq-out-of-tree-kernel-module`	补编 lima 模块时在系统构建树中运行 `make modules_prepare`：内核由 gcc 12 编译而板上为 gcc 11，板上时钟落后，kconfig 重新运行，改写了 `.config` 并删除 `autoconf.h`	选项只能编进内核、不能编为模块；强制模块签名；已有对应当前内核的发行版软件包
`pynq-systemd-autostart`	两板改为开机自启后断电重启，链路不通：PYNQ 自带的 `bootpy.service` 晚十几秒执行，加载 `base.bit` 覆盖了已加载的 overlay	一次性实验；Jupyter notebook（应先转为脚本）
`pynq-local-display-gui`	在板上用 Qt 的 `offscreen` 平台检查排版，所用字体与 X 会话不同，文字宽约 40 %，报出不存在的截断；固定尺寸的离屏渲染也看不出 600 像素高的窗口超出 578 像素的可用区域	界面运行在 PC 上；视频输出完全在 PL 中、不经过 X；三维 OpenGL 绘制（配合 `mali400-offscreen-gles`）
`pynq-dma-stale-transfer`	RX 服务停止时 ADC 采集 DMA 仍在等待下一帧，之后每个采集进程的第一次传输都卡在 `wait()`；PYNQ 的 `stop()`撤不掉等待中的传输，改用 DMACR 软复位	只用 MM2S 发送且 PL 总能接收；scatter-gather 模式；PL 从不产生 TLAST（应修改 PL）

每个技能都在全新会话、全新工作区中由另外的代理和模型执行测试，共两套环境：Cursor CLI + Grok 4.7，CodeBuddy CLI + DeepSeek V4.1 Flash。工作区只放该技能和用户场景所需的文件，板卡登录使用工作区内的一次性私钥副本；提示词采用普通用户口吻、不提技能名，两套环境相同。逐项核对会话记录：是否自动触发技能、是否按步骤执行与验证、是否在暂停点停下、结果是否正确、板上是否留下临时文件、系统文件是否被改动。5 个技能在两套环境中均通过，其中内核模块技能换用另一个驱动（hid-multitouch）测试通用性。

对照实验采用同样的场景和提示词，工作区中不提供技能；每次运行前后对板子做状态快照，运行后恢复到基线再进行下一次（表 B-3）。

表 B-3　有无技能的完成时间（秒）

场景	Grok 无技能	Grok 有技能	加速	DeepSeek 无技能	DeepSeek 有技能	加速
DMA 遗留传输	1199	479	2.5×	143	24	6.0×
Mali-400	1321	132	10×	401	96	4.2×
板载屏排版	514	295	1.7×	218	81	2.7×
开机自启	1404	295	4.8×	202	137	1.5×
补编内核模块	1783	392	4.5×	2530	160	16×

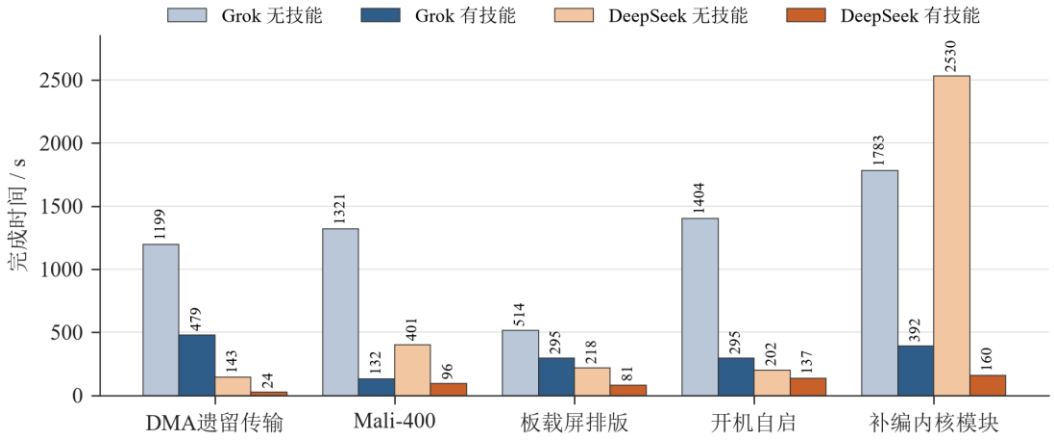


图 B-1　有无技能的完成时间。

1. 有技能时各场景都更快（1.5～16 倍），技能直接给出了根因、做法和可用的脚本；无技能时修复也常不完整，例如 DMA 修复缺接收超时或缺启动时复位。
2. 差别最大的是安全。无技能的代理大多未经询问就改动系统：安装软件包、加载内核模块并设为开机加载、安装并启用服务、为验证自启而自行重启板子、在系统内核构建树中重跑 kconfig 并改写配置，还有一次在 SSH 断开时尝试用默认密码登录 Jupyter；有技能的运行都在暂停点停下。

技能内容与测试记录见 `skill/README.md`。

附录 C 复现说明

硬件：2 块 PYNQ-ZU、2 块 AD9361 FMC 子卡、2 根胶棒天线；TX 可选 USB UVC 摄像头，RX 接 1024×600 DisplayPort 触摸屏；两板之间一根网线。

使用部署版比特流：见仓库根目录 `README.md`。把 `board/bitstreams/` 与 `src/sw/` 拷到两块板上，安装 `src/sw/systemd/` 中的服务，上电约 1 分钟后触摸屏显示 GUI。比特流的 SHA-256 见 `board/bitstreams/SHA256SUMS`。Mali-400 驱动与 PyQt5 的安装见 `src/sw/board_setup/`。

从源码构建比特流（Vivado 2025.2）：

```
vivado -mode batch -source build/build_hw.tcl -tclargs tx 8
vivado -mode batch -source build/build_hw.tcl -tclargs rx 8
```

脚本按 src/hw/<tx|rx>/files.txt 新建工程、加入 IP、生成 PS 块设计，用默认策略完成综合与实现。在干净目录中重建的资源与时序报告与部署版一致（表 3-1）。

从模型重新生成神经网络：仓库中带有部署模型的检查点与 W8A12 量化导出（data/model_shallow/，编码器与浅层解码器；data/model/为第一段训练得到的编码器与对称解码器），python src/network/regen_hw.py --export data/model_shallow --gen src/network/gen_shallow --files-txt 由它重新生成两端的神经网络 RTL 与权重初始化文件并写入 src/hw/，git diff 为空即与入库文件一致。重新训练与导出见 src/model/README.md，需要 CUDA 与训练数据。

仿真：python sim/network/make_golden.py 用 NumPy 整数参考模型生成各层参考数据，再运行 sim/network/中的脚本做单层、块级与整网的逐比特比对；SSCC 基线的仿真见 sim/sscc/。各脚本的命令与期望结果见 sim/README.md。

复现测量：board/tests/中各脚本开头有用法说明。PSNR–SNR：RX 板上运行 measure_link.py，PC 上运行 analyze_const.py 计算数据辅助 SNR；帧率：measure_link.py --src video --dwell 600；分段时延：两板同时运行 pl_events.py，PC 上用 seg_latency.py 配对；功耗：两板在各工作状态下运行 power_rails.py（PMBus 遥测，60 s 平均）。报告中的图由 report/figures/visio/（Visio 脚本）与 report/figures/sim/（Python 脚本，实测数据图为 meas_plots.py）生成。

复现验证：用与技能测试相同的方法（附录 B），由与开发无关的代理和模型在全新工作区中执行。工作区中只有从 GitHub 克隆的仓库，提示词要求完全按仓库文档操作、不修改仓库，最后提交复现报告。PC 部分（重新生成神经网络 RTL、仿真、比特流构建）由 CodeBuddy CLI + DeepSeek V4.1 Flash 执行；板上部分（把部署版比特流与程序部署到两块已装有旧版本的板卡，运行链路）由 Cursor CLI + Grok 4.7 与 CodeBuddy CLI + DeepSeek V4.1 Flash 分别执行，每次部署前板卡还原到同一旧版本，触摸屏画面由参赛者在现场确认。结果见表 C-1。表 C-1 的验证在解码器换成浅层结构之前完成；换用浅层解码器后，参赛者一方重新执行了神经网络 RTL 的重新生成（编码器 378 个、解码器 119 个初始化文件与两个顶层，与入库文件逐字节相同）、两端整网逐比特仿真（各 2 帧，编码器 17,717,420 个周期、浅层解码器 13,767,645 个周期，全部逐比特一致）与两端比特流构建（表 3-1）。

表 C-1　复现验证结果（对称解码器版本）

项目	结果
重新生成神经网络 RTL	编码器 378 个、解码器 404 个初始化文件与两个顶层，与入库文件逐字节相同
单层与块级仿真	行缓存 7 个用例、卷积层 8 个、GDN / IGDN 4 个、块级顶层 1 个，全部逐比特一致
整网仿真（2 帧，输入 valid 90 %、输出 ready 80 %）	编码器 17,717,420 周期、解码器 14,803,719 周期，逐比特一致，周期数与 `sim/network/results/`中的记录相同
SSCC 基线仿真	TX → AWGN → RX 环回：无噪声时 32,768 个符号与 Python 参考逐个相同、0 字节错误；15.5 dB、14.5 dB 下各 202 个码字的译出比特、收敛标志与迭代次数与逐位参考模型（`src/model/ldpc_ref.py`）完全相同
比特流构建	TX、RX 的资源、时序与功耗与表 3-1 相同
板上部署	两套环境均按 `README.md` 部署成功，三个服务正常运行；RX 在 60 s 内收到 1801 帧（30.0 fps），无丢帧；GUI 显示预设视频

附录 D 浅层解码器的设计细节

本附录给出 2.2.3.1 节第 2 小节所依据的分析，完整的说明与脚本见 `report/notes/shallow_decoder_design.md` 与 `src/model/shallow_analysis/`。

D.1 Yang 与 Mandt 浅层解码器中的大核

Yang 与 Mandt[19]的浅层解码器以一层大核转置卷积为主（两层合成：核尺寸 13、步长 8；JPEG 式合成：核尺寸 18、步长 16）。论文没有专门说明大核的作用，一种理解是：深层解码器各层的激活函数基本工作在近似线性的区间，若干层小卷积核就可以跨过激活函数合并成一个大核。用论文公开的模型（$\lambda = 0.005$）检验，见图 D-1：

- 大核的能量并不集中在中心、向外递减，而是在一个步长大小的块内大致均匀（两层合成主路的中心 8×8 块含 81 %的能量，JPEG 式合成的中心 16×16 块含 96 %），出了块才迅速衰减；大核更像 JPEG 的块基函数，核尺寸大的直接原因是上采样倍数大。
- 用 3 层 3×3、步长 2 的线性转置卷积（中间 128 通道）按最小二乘拟合 13×13、步长 8 的大核，剩余误差只占核能量的 0.11 %（主路）和 0.38 %（残差路）；4 层级联拟合 18×18、步长 16 的 JPEG 式大核，剩余 0.24 %。同尺寸的随机核即使保留训练核的空间能量分布，剩余误差仍在 16 %以上；把训练核每个位置上的通道顺序打乱后同样为 16 %。训练出的大核能被小核级联近似，靠的是各位置之间通道的一致结构，而不是能量分布本身。

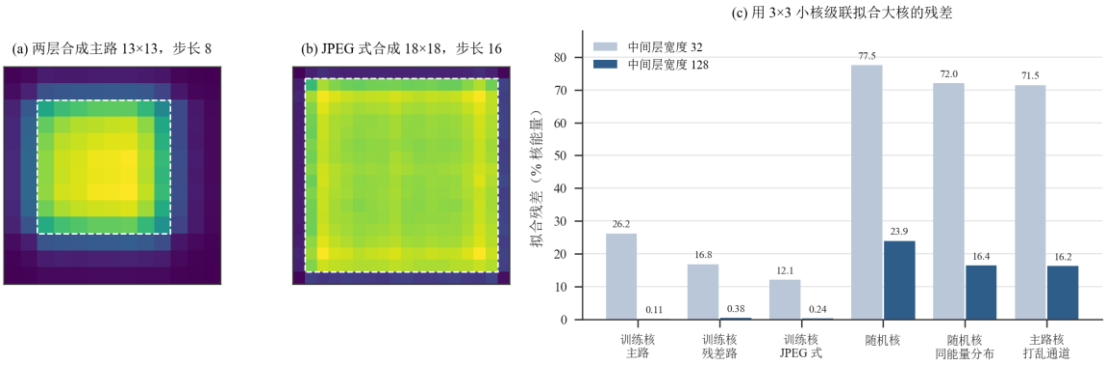


图 D-1　Yang 与 Mandt 公开模型的合成大核。(a)、(b) 各空间位置的能量（对通道求和，虚线框为一个步长大小的块）；(c) 用 3×3、步长 2 的线性转置卷积级联拟合大核时的剩余误差（相对核能量），对照为同尺寸的随机核与打乱通道后的训练核。

D.2 直线插值的线性程度

在两幅图像（或两组特征）之间取 $x(t)=(1-t)x_0+t\cdot x_1$，比较输出沿 t 的路径与两端连线（弦）的偏离（DIV2K 验证集 100 对图像，无噪声），见图 D-2 与表 D-1。编码器把图像空间的直线映射成明显弯曲的路径，编码器与解码器串联后重建图像的路径又几乎是直线（不经过 64QAM 判决时为 0.025 与 1.005），即编码器的非线性在解码器中被反解；解码器内部，前段的弯曲程度与整个解码器相当，后段在 sigmoid 之前几乎是直线。

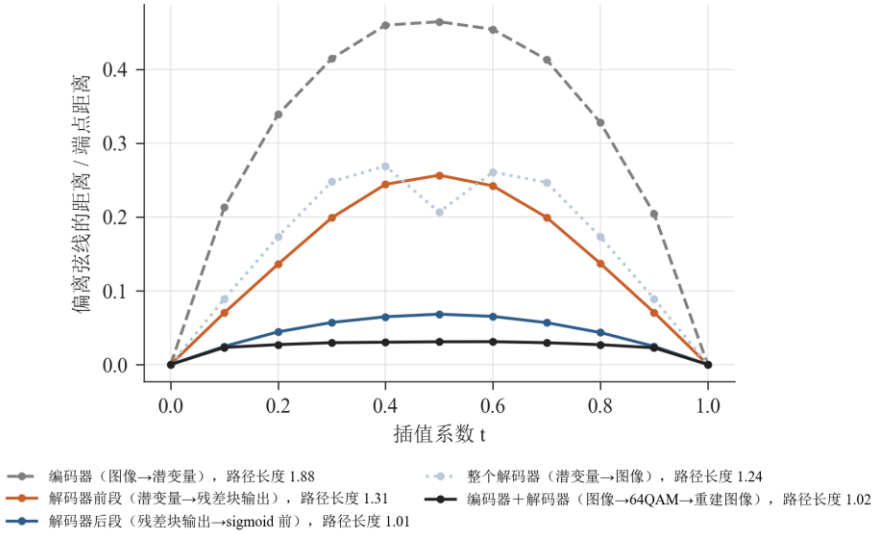


图 D-2　沿直线插值时输出偏离弦线的距离（相对端点距离，100 对图像的中位数）。

表 D-1　各部分沿直线插值的线性程度（中位数；完全线性时偏离为 0、路径长度为 1）

部分	最大偏离 / 端点距离	路径长度 / 端点距离
编码器：图像 → 潜变量	0.474	1.88
解码器前段：潜变量 → 第 4 个 RP 输出	0.261	1.31
解码器后段：第 4 个 RP 输出 → sigmoid 前	0.068	1.01
解码器后段：第 4 个 RP 输出 → 图像	0.097	1.02
整个解码器：潜变量 → 图像	0.297	1.24
编码器与解码器串联：图像 → 64QAM → 重建图像	0.032	1.02

D.3 各级的噪声

同一组编码符号分别以无噪和加噪（AWGN）两种形式送入解码器，在每一级比较无噪输出（去掉各通道均值）的功率与噪声引起的偏差（有噪输出减无噪输出）的功率，得到各级输出的 SNR（图 D-3）。对线性的一级，这就是该级输出的信噪比；对非线性的一级，它表示噪声引起的偏差相对信号的大小，两种解码器内部特征不同，中间各级不能逐级对比，输出图像处可以直接比较。

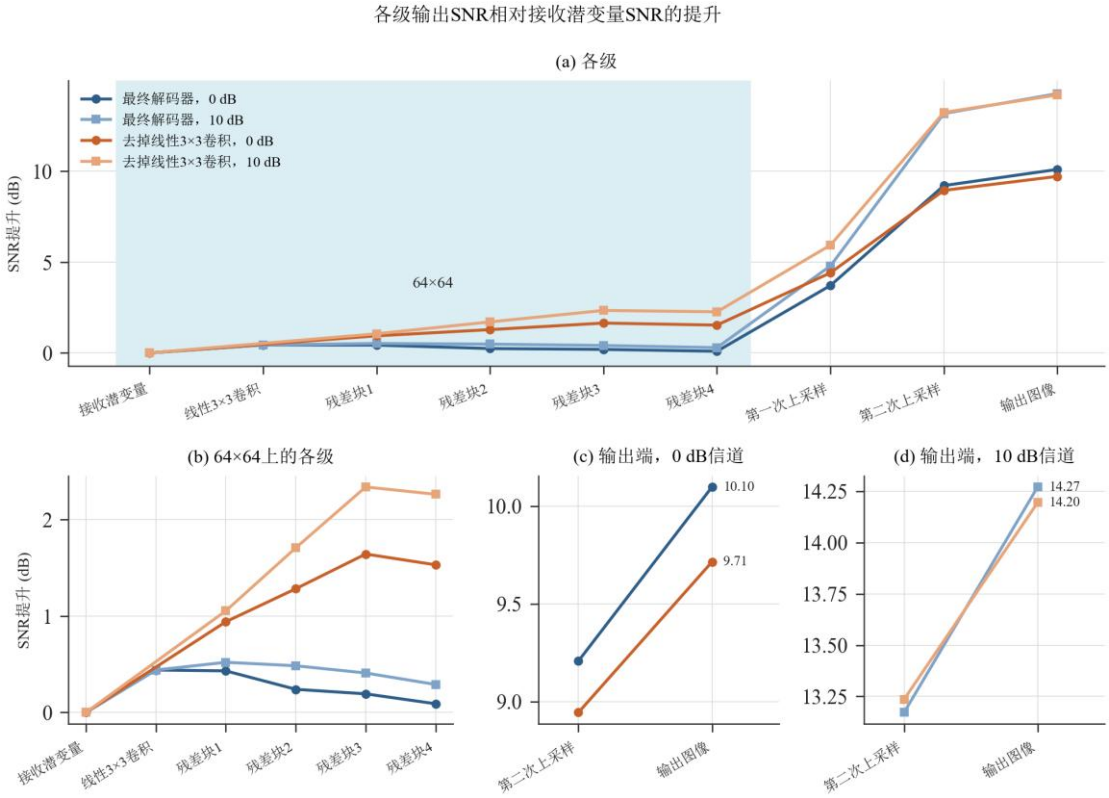


图 D-3　各级输出 SNR 相对接收潜变量 SNR 的提升（DIV2K 验证集 100 张，同一组噪声）。蓝色为最终解码器，橙色为去掉线性 3×3 卷积的解码器；圆点为 0 dB 信道，方块为 10 dB 信道。(b)为 64×64 部分的放大；(c)、(d)为输出端的放大，数字为输出图像处的值。

最终解码器中，线性 3×3 卷积在各 SNR 下都把 SNR 提高 0.44 dB；之后 64×64 上各级与输入相差不到 0.6 dB，噪声主要在两次上采样中被抑制，10 dB 信道下第一次上采样后高 4.8 dB，第二次上采样后高 13.2 dB：上采样卷积的每个输出由 3×3×C 个输入加权组合而成，把噪声中不属于图像结构的分量投影掉。去掉线性 3×3 卷积后，残差块自身把偏差压得更小，64×64 上最多高出输入 2.3 dB，但这部分没有保留到输出：输出图像处 0 dB 信道下比最终解码器低 0.38 dB，10 dB 下低 0.08 dB，与表 2-8 的 PSNR 差（−0.34 dB、−0.04 dB）一致。残差块的非线性激活在压低噪声偏差的同时也改变了信号，入口的线性滤波则在不改变信号结构的前提下提高 SNR，对低 SNR 更有效。

D.4 PWL 学到的激活

4 个 RP 中 8 个激活共 128 个通道学到的曲线见图 D-4。按左侧（输入−2～−0.5）与右侧（0.5～2）的平均斜率分类：57 个通道为 V 形，左侧斜率为负、右侧为正，拐点在 0 附近；59 个通道单调上升，但左侧斜率多为 0.1～0.6，远大于 LeakyReLU 的 1/128；12 个通道右侧不上升。所有通道的平均形状在 0 附近平滑下凹，左侧斜率略为负，右侧斜率约 0.5，与初始的 LeakyReLU 明显不同。

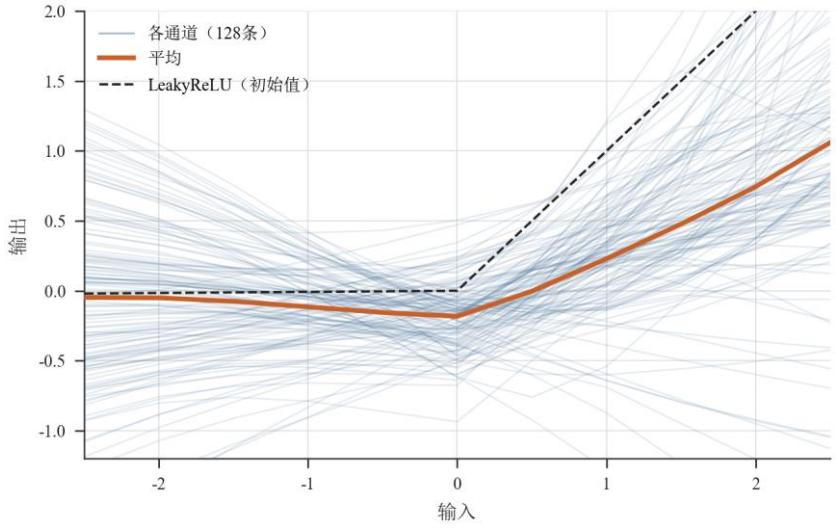


图 D-4　RP 中 PWL 激活学到的曲线：128 个通道（浅色半透明，重叠处颜色加深）与其平均形状，虚线为初始的 LeakyReLU。各激活在 10 dB 信道下的输入有 98 %落在约−2～2.3 之间。

附录 E 基于本作品的通信算法验证平台

E.1 意义与基本组成

本作品把 DeepJSCC-Q 从论文做成了实时空口系统，大部分工作量却不在算法本身，而在射频接口、PL 与 PS 之间的数据搬运、驱动、上位机和测量手段上。这些部分是围绕 DeepJSCC-Q 写成的，换一个算法就要重做。课题组研究的通信算法大多停留在 MATLAB 仿真，要在真实信道、实时条件下验证，面临同样的门槛：在电脑上处理，采样率受电脑限制；放进 FPGA，又要从零搭建整套系统，FPGA 实现出错时也难以与仿真对照定位。

为此，我们在本作品的两块板卡上设计了一个通用的通信算法验证平台。射频、数据搬运、驱动和上位机由平台提供；算法先在 MATLAB 或 GNU Radio 中接真实射频验证，再把计算量大的部分写成 PL 模块，逐个移入 PL，每移入一个模块都与它的软件实现逐样点核对，最终脱离电脑运行。平台的 PL 部分参考了 Ettus Research 的 RFNoC 框架（RFNoC Specification 1.0）：把处理拆成可以任意连接的模块，模块之间以带时间戳的数据包通信，连接关系在运行中由软件设定。

平台的组成见图 E-1。PL 中有四类部件：

- 射频收发流：在 AD9361 的采样与平台内部的数据流之间转换，并维护硬件时间；
- 模块槽位：放置用户模块，每个槽位一个输入、一个输出；
- 主机端口：PL 与 PS 之间的数据出入口，经 DMA 读写内存，收、发各两个；
- 流交换：把以上各部件的输出接到任意部件的输入。

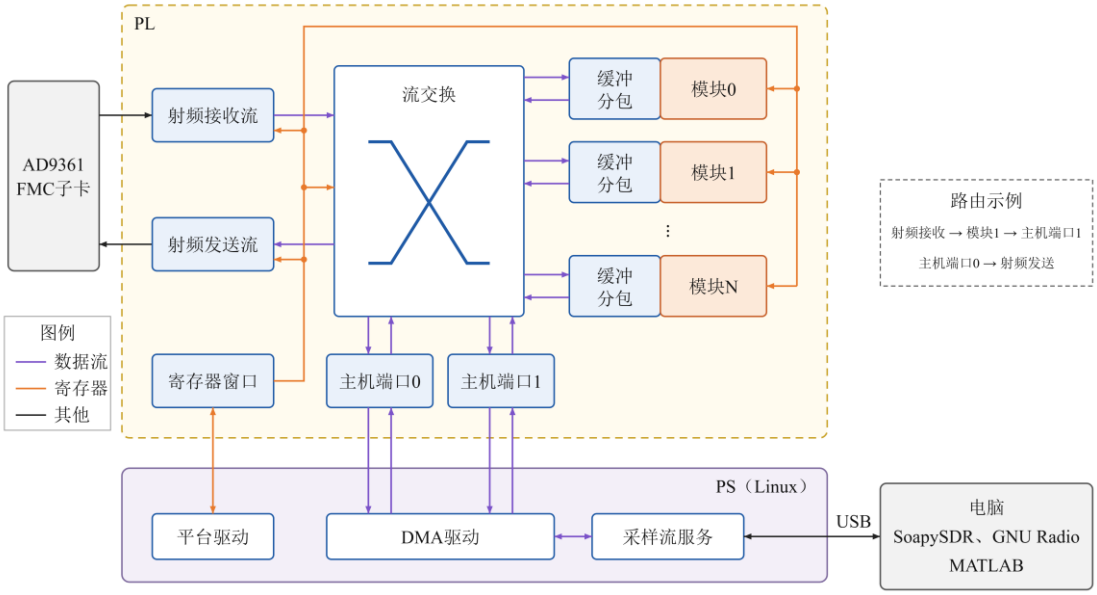


图 E-1　平台的组成：每个槽位由平台生成的缓冲与分包部分和用户模块组成；紫色为数据流（AXI4-Stream），橙色为寄存器访问（AXI4-Lite）；右上方为一组路由的例子。

拓扑分两级确定。槽位里放哪些模块、每个槽位用什么时钟，在生成比特流时由一份镜像配置决定；部件之间怎样连接，在运行中由流交换的路由寄存器决定。同一个比特流可以组成多种处理链，例如先把一个模块的输入、输出都接到主机端口单独测试，再把它接进射频链路；图 E-1 右上方的路由让射频接收经模块 1 送往电脑，同时电脑发出的数据直接送往射频发送。

与 RFNoC 相比，平台只在一片 FPGA 内工作，数据经一级流交换即可到达任何部件，因此没有沿用 RFNoC 的 CHDR 包格式与多级路由，各部分的对应与区别见表 E-1。

表 E-1　RFNoC 的部件与本平台的对应

RFNoC	**本平台**	**区别**
NoC 块（NoC Shell + 用户逻辑）	模块槽位 + 用户模块	用户模块只有 `tdata`、`tvalid`、`tready`，包由槽位处理
CHDR 包	片内包与主机端口包头	包头只有时间戳与标志，不含路由信息；片内包头放在首拍的侧带信号中，不占数据拍
CHDR 交叉开关、静态路由	流交换	一级交换，全部连接都可在运行中更改
流端点、传输适配器	主机端口	不跨 FPGA，主机端口直接接 DMA
控制面（控制包、控制交叉开关）	寄存器窗口（AXI4-Lite）	PS 直接按地址读写，不经数据网络
`axi_rate_change`	槽位中的分包模块	作用相同
NoC Shell 的清理（flush）	槽位清理	做法相同

不用控制包，是因为控制方只有片上的 PS：PS 可以按地址直接读写 PL 中的寄存器，不需要像 RFNoC 那样把控制命令打包、经交叉开关送到远端的 FPGA。这样省去了每个模块的控制端点与控制交叉开关，模块的寄存器也成为普通的读写端口。

E.2 包格式

E.2.1 片内包

平台内部的所有数据流都是带 `tlast` 与 `tuser` 的 AXI4-Stream。一个包是一段连续的样点，`tlast` 标记最后一拍；包头放在第一拍的 `tuser` 中，其余各拍的 `tuser` 不使用。各字段的位置见图 E-2。

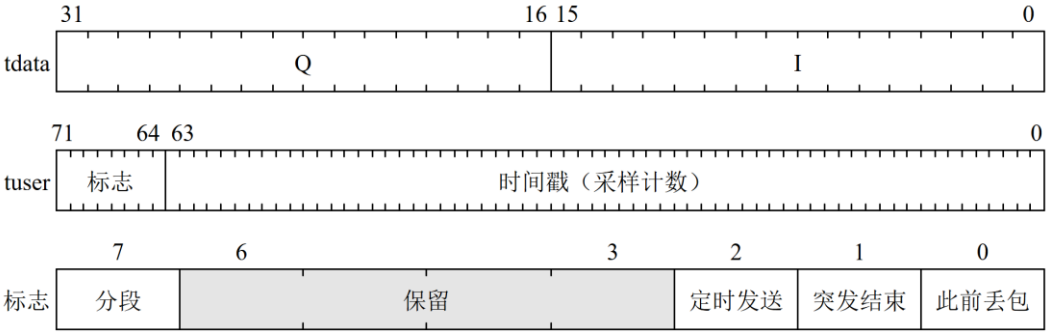


图 E-2　片内包的字段：tdata 为一个复样点，I、Q 各为 16 位补码；tuser 为 8 位标志与 64 位时间戳，只在包的第一拍有效；标志的位 7 只出现在主机端口的包头中。

每拍一个复样点，与 SoapySDR 的 CS16 格式相同，送到电脑的数据不需要换算。AD9361 的 12 位 ADC 样点在 RX 流化时左移 4 位，PL 中收、发两个方向都是 16 位满量程，模块直接接射频时两个方向的幅度一致。模块也可以在包中传比特、软比特或字节，格式由模块说明文件声明，电脑按声明解释。

包头放在侧带而不是数据拍中，是这一格式最主要的取舍。RFNoC 的 CHDR 把包头作为包的前几拍传输，处理包的每个部件都要识别并跳过它；平台的包头与第一个样点同拍出现，用户模块只需要 tdata 一路，包头由槽位记下、再贴回输出（E.4.4 节）。代价是每个端口多出 73 根线（72 位 tuser 与 tlast），流交换中每路选择的宽度由 32 位增至 105 位。平台的端口在十几个以内，这部分逻辑不大，换来的是每个用户模块都不必处理包。

时间戳是 64 位采样计数器的值。计数器在采样时钟下每个样点加 1，PS 可以读出，也可以写入设定值（对时）。时间戳用样点个数而不用纳秒表示，换算由电脑上的驱动按采样率完成，PL 中的比较与加减都是整数运算，不随采样率产生舍入误差；50 MSPS 下 64 位计数约 11700 年才回绕。RX 方向的时间戳是包中第一个样点的计数值，TX 方向是包的发送时刻。经过插值、抽取等模块后，输出包携带的仍是对应输入包的时间戳，所以时间戳始终以射频采样为单位，与模块改变了多少速率无关。

标志位的含义见表 E-2。前三个标志在所有端口、两个方向上用同一套编号：一个包从主机端口发出，经过若干模块再回到主机端口，标志的含义不变。

表 E-2　包头中的标志位

位	名称	由谁置位	由谁使用
0	此前有丢包	RX 流化：下游来不及接收而整包丢弃后，在下一个包置位	电脑：SoapySDR 驱动据此报告溢出
1	突发结束	电脑（发送时标记突发结束），或模块（如 OFDM 发射在帧尾）	TX 流化：此包之后的空闲不计为欠载
2	定时发送	电脑（发送时给出时刻）	TX 流化：采样计数器到达时间戳时才开始发送；已经迟到的包按配置丢弃或立即发送，并计数
7	分段	主机端口（送往电脑）：包长超过 8192 个样点时切成多段，除最后一段外置位	电脑：把各段拼回一个包

“此前有丢包”与时间戳一起，使接收数据的不连续可以被准确定位。RX 流化只在缓冲能放下整个包时才开始一个包，否则整包丢弃，因此送出的每个包内部都是连续的样点，时间戳与内容一一对应，丢失的样点数等于两个包的时间戳之差减去前一个包的长度。整包丢弃也便于接收端处理：OFDM 等按帧同步的接收机只需在不连续处重新捕获，不会在一个包中间遇到断点。

一个包的时序见图 E-3。包内可以随时被反压，包与包之间可以有空闲；图中第二个包的标志为“此前有丢包”，时间戳比第一个包多 8，说明中间丢了一个包。

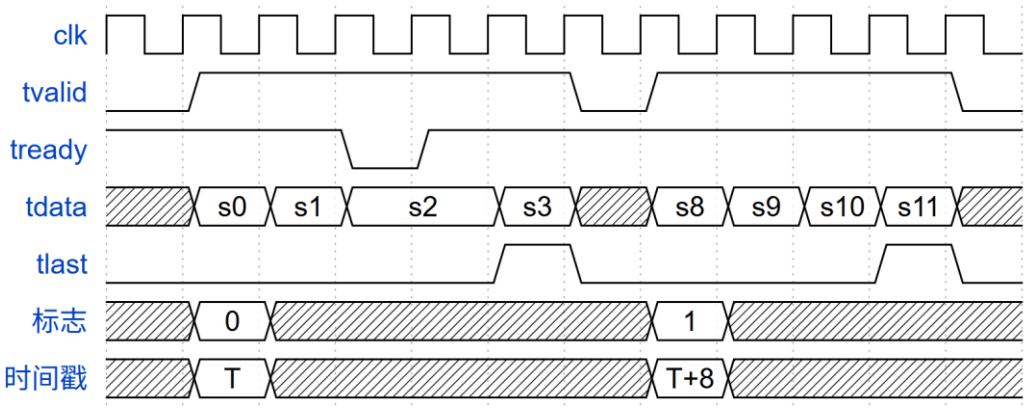


图 E-3　包的时序（为便于作图，包长取 4 个样点）：`tuser` 只在每个包的第一拍有效，第二个包的标志为 0x01（此前有丢包）。

包长由寄存器设定，RX 方向默认 4096 个样点，最长 8192 个。包越长，主机端口包头的开销越小（4096 个样点时为 0.1 %）；但流交换只在包边界改路由，丢包也以包为单位，包越短，改路由的等待和溢出时丢失的样点越少。4096 个样点在 50 MSPS 下为 82 μs，两方面都可以接受。模块输出的包长由模块决定，例如 OFDM 接收输出的一帧有 32768 个软符号。

E.2.2 主机端口包头

在主机端口，侧带信号要与样点一起写入内存，因此每个包前加 4 个 32 位字的包头（图 E-4），之后是包中的样点。

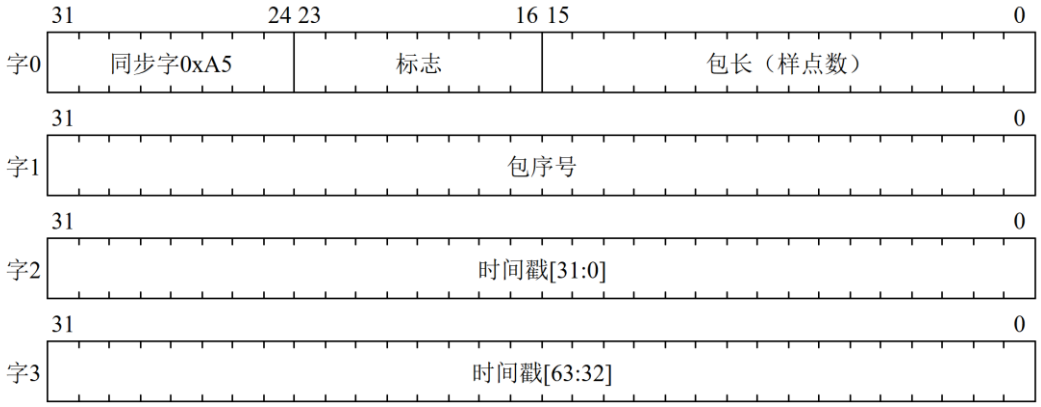


图 E-4　主机端口的包头，字 0 在前。

- 同步字 0xA5：TX 方向，主机端口在应当是包头的位置读到的不是同步字（或包长为 0）时，丢弃这个字并计数，再从下一个字重新寻找包头，一处写错不会使后续数据全部错位；
- 包长：以样点为单位。RX 方向的包长要到包尾才知道，主机端口先把一段样点放进缓冲，再把包头写在样点之前；
- 包序号：每个端口独立计数，电脑据此检出 DMA 或传输过程中丢失的包；
- 标志、时间戳：与片内包相同，TX 方向的时间戳是发送时刻。

包头使时间戳与标志随数据一起走，不需要另外的通道传送。主机端口另有原始模式，不加包头，供不认识包头的工具（如 libiio 的命令行工具）使用；模式只在端口空闲时切换。

E.3 流交换

E.3.1 端口与路由寄存器

流交换的源端口与接收端口按表 E-3 编号。路由寄存器为 4 位，0xF 表示不连接，因此最多 11 个槽位、14 对端口。

表 E-3　流交换的端口编号

编号	源端口（数据来自）	接收端口（数据送往）
0	射频接收	射频发送
1	主机端口 0（电脑发出的数据）	主机端口 0（送往电脑）
2	槽位 0 输出	槽位 0 输入
3	主机端口 1（电脑发出的数据）	主机端口 1（送往电脑）
3+k（k≥1）	槽位 k 输出	槽位 k 输入

每个接收端口 j 有一个路由寄存器 ROUTE[j]，写入源端口的编号，表示接收端口 j 的数据来自这个源（图 E-5）。每个接收端口前是一个选择器，按自己的

寄存器从全部源中选一个。复位后的路由为射频接收接主机端口 0、主机端口 0 接射频发送，平台上电即可作为普通的软件无线电使用。

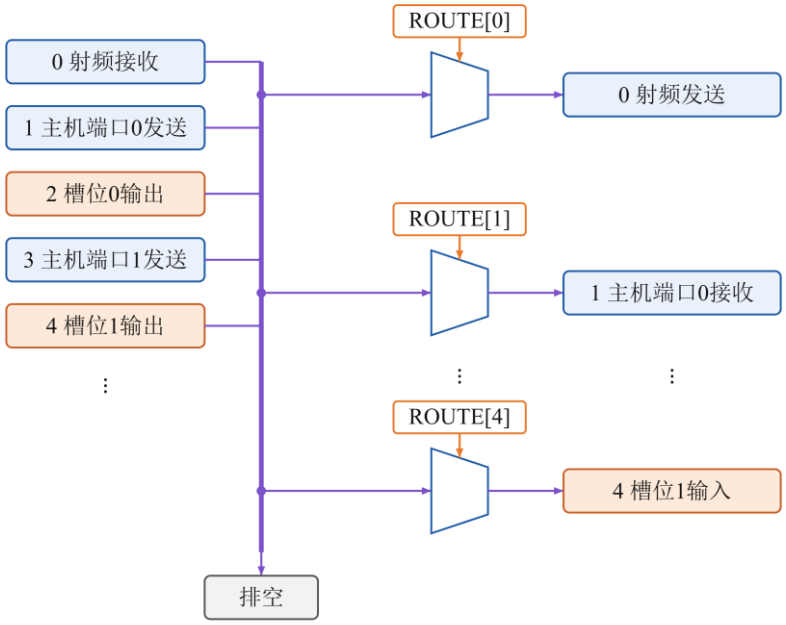


图 E-5　流交换的结构：每个接收端口一个选择器，由各自的路由寄存器选择数据来源；没有被任何接收端口选中的源接到排空支路。

由接收端口选择来源，而不是由源端口指定去向，是因为一个接收端口在同一时刻只能接收一路数据，每个选择器只受一个寄存器控制，逻辑最简单。一个源在同一时刻只送往一个接收端口，若几个接收端口选了同一个源，编号最小的得到它；需要把一路数据复制成多路时，可以写一个复制模块放进槽位。

E.3.2 切换规则

路由寄存器在 PS 的时钟下写入，在流交换的时钟下使用。新值经两级触发器同步后，连续 4 个周期不变才生效，几位同时变化时不会用到中间状态。生效后的切换遵循以下规则：

- 只在包边界切换：接收端口正在转发一个包时，等这个包的最后一拍送出后才换到新的源，不会切出半个包；
- 源有数据时才占用：接收端口空闲时，只有当新的源送出一个包的第一拍时才与它接通。如果接收端口在源还没有数据时就占住它，那么在源发出一个包之前，这条路由就再也改不掉；
- 不从包中间接入：源在发一个包的中途被选中时，这个包的剩余部分被排空，接收端口从它的下一个包开始接收。接收端口收到的总是完整的包，包头总在第一拍；
- 未被选中的源被排空：没有任何接收端口选中的源，数据直接丢弃，而不是反压。射频接收没有接到任何地方时不会被计为溢出；一条链路断

开时，上游的模块也不会因此停住。

图 E-6 是一次切换的时序。路由由 A 改为 B 时，接收端口仍在转发 A 的包，转发完 a5 后断开；B 此时正处在包 b0～b3 的中间，剩余部分被排空；B 的下一个包 b4～b7 要等接收端口空闲后才被接收，期间 B 被反压。

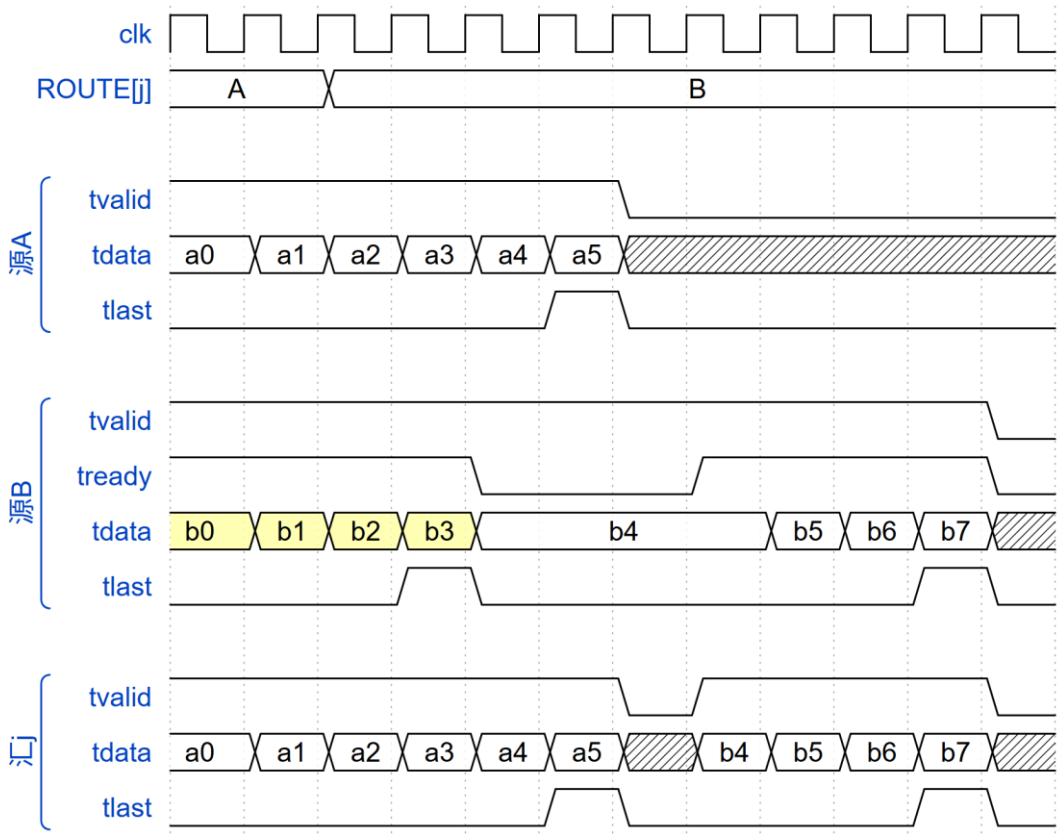


图 E-6　流交换切换路由的时序：ROUTE[j]为同步后生效的值，黄色为被排空的数据。

E.3.3 端口清理

只在包边界切换带来一个约束：一个停在包中间的端口永远到不了包边界。例如电脑上的程序在接收中途退出，DMA 停在一次传输的中间，主机端口里留下半个包，与它相连的接收端口就一直等这个包的最后一拍，路由无法再改。为此平台为每个主机端口、每个槽位设一个清理位（写 1 后自动清零）。清理时，该端口在流交换中的连接立即断开，流交换不再记着它正在发一个包，端口自身的缓冲清空；对槽位，还执行 E.4.4 节的模块清理。电脑上的驱动每次开始一条数据流之前，沿路由依次清理经过的端口和槽位（接收方向向上游、发送方向向下游），每条数据流都从干净的状态开始。

E.4 模块槽位

E.4.1 用户模块的接口

用户模块的端口见表 E-4。数据流只有 `tdata`、`tvalid`、`tready`，按 AXI4-Stream 握手收发，两侧都可能被反压；包、时间戳、跨时钟域、总线握手都由平台处理。

表 E-4　用户模块的端口

端口	方向	说明
clk、rstn	输入	槽位时钟与复位（复位寄存器与处理状态）
flush	输入	开始新的数据流时为 1：模块按需清除处理状态，寄存器保留
s_axis_tvalid、s_axis_tready、s_axis_tdata[31:0]	输入流	每拍一个样点
m_axis_tvalid、m_axis_tready、m_axis_tdata[31:0]	输出流	同上
寄存器端口	输入、输出	按模块说明文件中的寄存器表生成，见 E.4.2 节

一个输入不对应一个输出的模块，在说明文件中声明速率：每 M 个输入对应 L 个输出（插值、抽取），或输出个数不定（如符号同步每个符号输出一个点）。L、M 可以写成含寄存器的表达式，例如模块关闭时为 1。按帧工作、需要自己处理 tuser 与 tlast 的模块（如 OFDM 发射），可以声明由模块自己分包，这时槽位把包头原样交给模块。

槽位的结构见图 E-7，参照 RFNoC 的 NoC 块，分成平台生成的部分与用户模块两部分。平台生成的部分对外接平台的时钟、复位、寄存器与流交换，对内向用户模块给出槽位时钟、复位、清理、寄存器端口与只有样点的数据流，内部分为三块：时钟与复位、寄存器（跨时钟桥与寄存器文件）、数据（输入缓冲、分包与输出缓冲）。

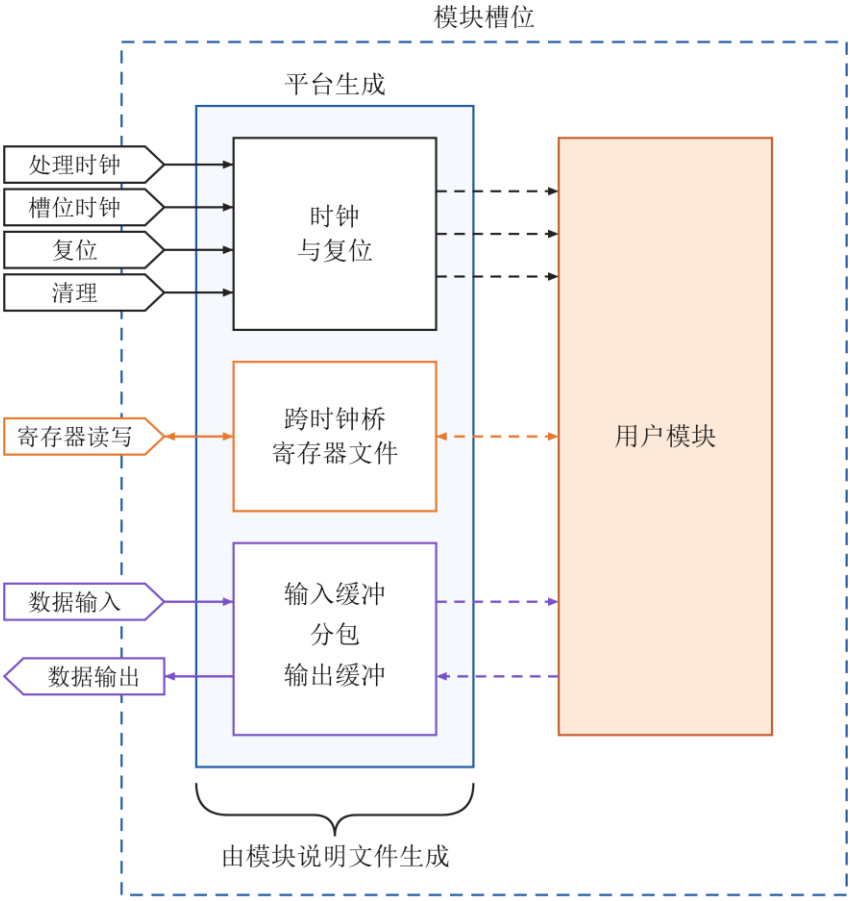


图 E-7　模块槽位的结构：左侧为接平台的端口，虚线为平台生成的部分给用户模块的端口；黑色为时钟与复位，橙色为寄存器，紫色为数据流。

E.4.2 寄存器

每个槽位有 4 KB 的寄存器窗口，槽位 k 位于平台寄存器空间的 0x2000 + 0x1000 • k。窗口中 0x000、0x004 固定为模块 ID 与版本，其余寄存器在说明文件中列出名字、位宽、默认值与含义，从 0x008 起依次排列。寄存器文件由说明文件生成，模块看到的是普通端口：

- 读写寄存器：一个输入端口，值为写入的值，复位后为默认值；
- 脉冲位：每次写 1 时一个周期的脉冲，用于软复位等操作；
- 带写选通的寄存器：另有一个端口在每次写入时为 1，用于向系数表中逐个写入；
- 只读寄存器：一个输出端口，由模块给出数值，用于状态与环路的读数。

读写都在一个周期内完成，模块不写任何总线握手与地址译码。电脑上的程序按名字访问寄存器，按模块 ID 找到它所在的槽位，所以换比特流、调整槽位顺序都不影响上层程序。

E.4.3 时钟与反压隔离

每个槽位可以选择 100、200 或 250 MHz 的时钟，平台的处理时钟为 200

MHz。槽位时钟与处理时钟不同时，输入、输出各经一个 32 深的异步 FIFO（用 LUTRAM 实现），寄存器经 AXI4-Lite 跨时钟桥，复位与清理信号经同步器。已有的 RTL 可以按原来的时钟前提放入槽位：本作品的编码器按 250 MHz 设计，物理层按 100 MHz 设计，迁移时都不需要改动。

槽位时钟与处理时钟相同时，输入前放一个两级缓冲。AXI4-Stream 允许模块的 `s_axis_tready` 组合依赖于 `m_axis_tready`，而槽位的输出与输入在流交换的同一组选择器中相遇，一个这样的模块就会经流交换形成组合环路。两级缓冲的就绪信号是寄存器输出，切断了这条路径，代价是一个周期的延迟和每个槽位约 20～60 个 LUT、212 个触发器。异步 FIFO 的满标志同样是寄存器输出，跨时钟的槽位不需要另加缓冲。这样，用户模块只要符合 AXI4-Stream 标准，不需要遵守标准以外的规则。

E.4.4 分包与清理

分包模块的作用与 RFNoC 的 `axi_rate_change` 相同：记下每个输入包的包头，对用户模块的输出计数，把输出重新分成包，每个输出包带上对应输入包的包头。

- 固定速率 L/M：第 n 个输入包对应的输出包，结束于第⌊N_n • L/M⌋个输出，N_n 为复位以来到这个包结束为止的输入个数。一个包短于抽取倍数、没有自己的输出时，它的包头随之丢弃；
- 输出个数不定：一个输出包结束于输入包结束之后的第一个输出，在此之前结束而没有输出的输入包，包头一并丢弃。

图 E-8 是 2 倍抽取的例子：每个 4 样点的输入包对应 2 个输出，输出包的第一拍带着对应输入包的时间戳。

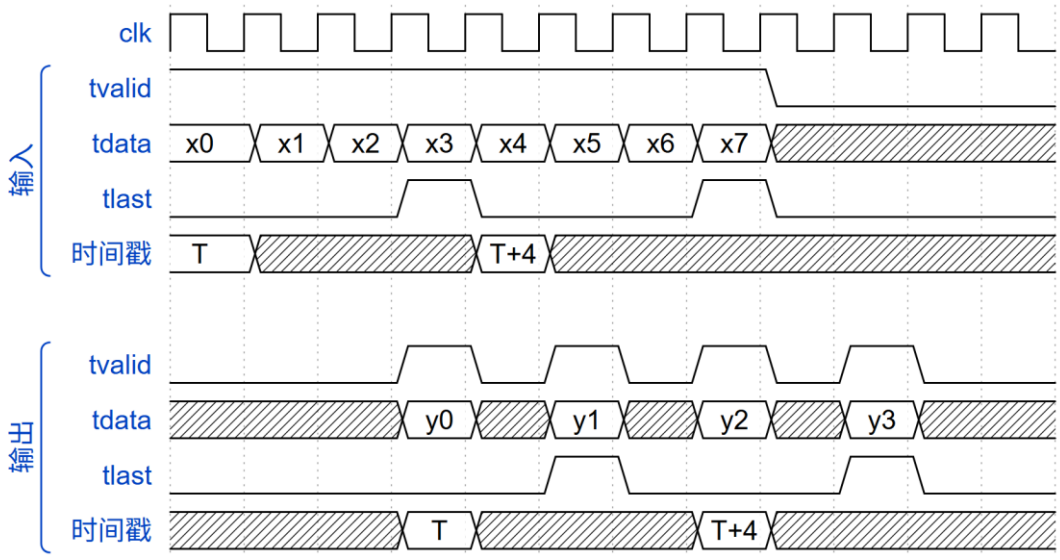


图 E-8　2 倍抽取时的重新分包：上为槽位的输入，下为槽位的输出。

分包模块从不因为等包头而扣住模块的输出。模块的输出先进入一个 16 深

的 FIFO，一个输出若还没有对应的包头（例如抽取器提前一个样点给出结果），就在 FIFO 中等下一个包头；包头 FIFO 最多记 64 个在模块中尚未输出的包。AXI4-Stream 允许模块在输出被反压时停止接收输入，如果平台在输出侧等包头，就可能与这样的模块互相等待，所以输出侧总是接收。

清理的过程见表 E-5，与 RFNoC 的 NoC Shell 清理一个块的做法相同。

表 E-5　槽位清理的步骤

步骤	平台的动作	结束条件
1	断开槽位在流交换中的两侧连接，清空输入、输出缓冲与分包模块，向模块发出 `flush`	一个周期
2	不再向模块送输入，模块仍在送出的数据全部接收并丢弃	模块连续 256 个周期没有输出
3	恢复接收输入，新的数据流从此开始	—

模块收到 `flush` 后只需清除自己愿意清除的状态，寄存器保留，模块的配置不会因每次开始数据流而丢失。模块流水线中尚未送出的结果由第 2 步丢弃，新的数据流中不会混入清理前的样点。Xilinx FIR Compiler 等 IP 的复位既不丢弃已经算出的结果，也不清除延迟线，由平台统一处理后，用这些 IP 的模块不必另写逻辑。

E.5 电脑端软件

SoapySDR 是开源的软件无线电硬件抽象接口，GNU Radio、GQRX 等软件都通过它访问硬件。我们为平台写了 SoapySDR 驱动，控制分三层映射到 SoapySDR 的标准接口：标准参数（频率、采样率、带宽、增益）；AD9361 驱动的配置参数（如快速 AGC 的各项门限），写入后重新初始化芯片，初始化前后自动保持频率、采样率、带宽与增益；AD9361 与平台的寄存器。数据流支持时间戳、突发结束与定时发送，与 E.2 节的包头一一对应。现有的 GNU Radio 流程图、GQRX 等不用修改即可使用这两块板卡。

GNU Radio 中，每个 PL 模块有一个 PL 块和一个软件块，都由模块说明文件生成。PL 块把参数换算成寄存器写给板上的槽位，软件块把同样的寄存器写给模块的 Python 参考模型，两者共用一份“参数 → 寄存器”的换算，在流程图中可以直接互换，用来逐样点核对 PL 模块。PL 块的端口在 GNU Radio 中单独成一类，两个 PL 端口之间的连线由 GNU Radio 转换成一次路由寄存器的写入，数据不经过电脑；程序结束时，它建立的路由自动断开（图 E-9）。在 GNU Radio 中连接 PL 模块，因此与连接普通的软件块相同。

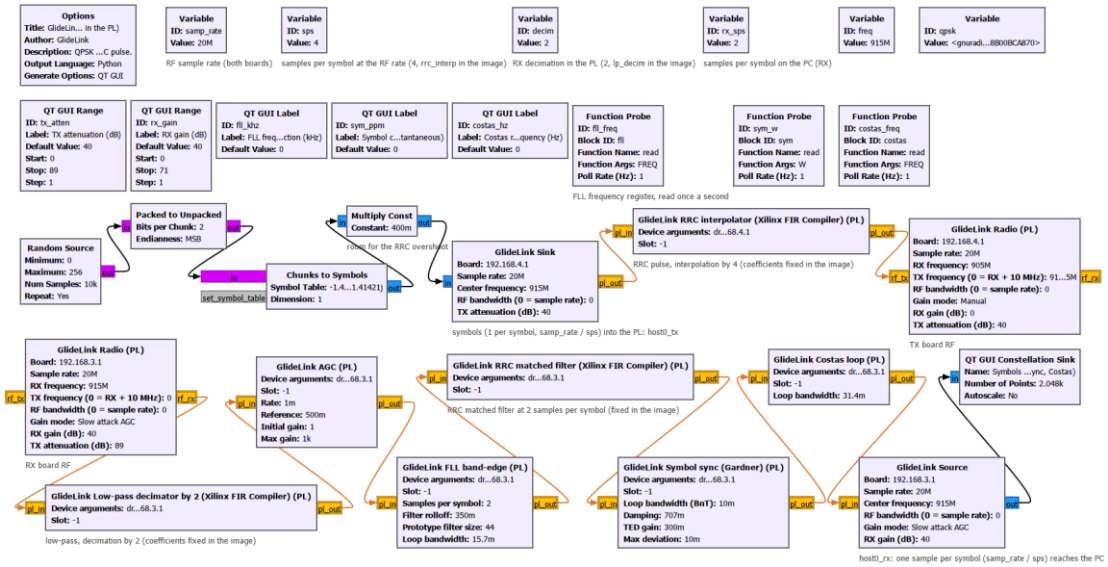


图 E-9　QPSK 接收机全部在 PL 中的 GNU Radio 流程图：橙色端口与连线为 PL 中的模块与路由。

模块的开发与验证由平台的脚本完成：用户写只含数据处理的 RTL、说明文件与参考模型，脚本生成寄存器文件、槽位与 GNU Radio 块。仿真时测试平台随机插入间隙与反压，输出与参考模型逐拍比较，读回全部寄存器，并在数据流中途清理一次，检查清理后的输出回到与模型一致；上板后同一段信号分别经软件块和 PL 模块处理，逐样点比较。路由可以把任一模块单独接到主机端口，链路中的每个模块都能这样单独核对。

MATLAB 经板上的桥接服务使用平台：采集在板上完成后再发回，任意采样率都不丢样点；循环发送的波形放在 DMA 的循环缓冲中由硬件重复；定时突发可以提前排定多个。全部处理都在 PL 中时，板上的守护程序按配置文件加载比特流、配置 AD9361、写模块寄存器、建立路由，板卡可以脱离电脑运行。

E.6 板卡与电脑之间的传输

板卡经 USB 3 连接电脑，以 USB 网卡的方式通信。50 MSPS 时收发同时每个方向约 200 MB/s，超过千兆网口的速率，因此我们参照 Ettus Research 对网口设备的做法，在板上写了采样流服务：TCP 连接建立并管理数据流，数据走 UDP，每个包带序号，两个方向都有流量控制；服务只搬运 DMA 中的数据，主机端口的包头原样通过。为达到这一速率，修改了内核中 IIO DMA 缓冲的缓存方式、AXI DMA 驱动的队列处理以及 USB 网卡的传输块大小与帧长。实测 Windows 下的程序收发同时各 50 MSPS，接收与发射均无丢失。

E.7 平台上的两个应用

QPSK 链路。我们在平台上实现了 QPSK 链路的全部 PL 模块：根升余弦插值、低通抽取、匹配滤波（用 Xilinx FIR Compiler），以及 AGC、锁频环频偏校

正、Gardner 符号同步、Costas 载波同步（自行设计），每个模块都与参考模型逐比特一致。两块板之间 5 Msymbol/s 的 QPSK（射频 20 MSPS），从全部在电脑上处理的 GNU Radio 流程图出发，把处理逐步换成 PL 模块：全部在电脑上时电脑来不及处理，持续丢样点；整个接收机移入 PL 后，电脑每个符号只收一个点，链路稳定运行，符号 EVM 约−19.6 dB，与电脑处理时相同（图 E-10）。槽位时钟 200 MHz 而采样率只有几到 25 MSPS，每个样点有十几个以上的周期，环路模块因此分时复用乘法器，镜像中的全部 PL 模块共用 42 个 DSP。

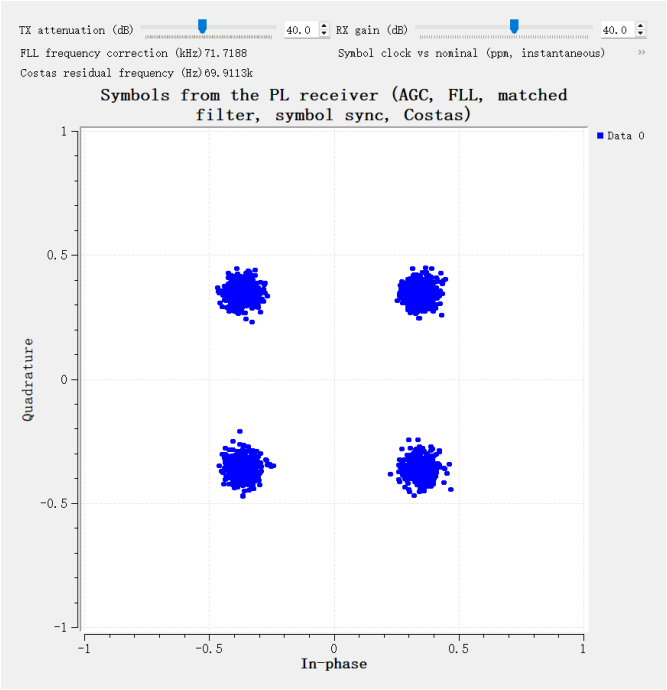


图 E-10　PL 接收机输出的 QPSK 符号，上方为从模块寄存器读出的锁频环与 Costas 环的频率。

本作品。本作品的编码器、OFDM 发射、OFDM 接收和解码器作为平台的四个用户模块，RTL 不变：编码器与解码器放在 250 MHz 的槽位，物理层放在 100 MHz 的槽位；TX 板为编码器与 OFDM 发射，RX 板为 OFDM 接收与解码器。编码器与解码器的输出与整数参考模型逐比特一致；物理层经 AD9361 数字环回，前导逐比特一致，数据符号全部判决正确。两块板由守护程序脱离电脑运行，空口连续 600 s 平均 29.97 fps，DIV2K 0802 在强信号下的 PSNR 为 32.21 dB，与本作品相同。